The Ultimate JSBSim Reference

Architecture, Aerodynamics, and Aircraft Modelling — Under the Hood

A practical engineering manual covering JSBSim's XML configuration language, equations of motion, integration methods, aerodynamic function/table framework, propulsion subsystem, flight control components, and the workflow for generating CFD-derived aerodynamic data tables for a custom aircraft model.

Generated 2026-05-24 · Reverse-engineered from JSBSim v2.0 source

Table of Contents

Chapter 1: Introduction to JSBSim

JSBSim is an open-source, multi-platform, object-oriented C++ Flight Dynamics Model (FDM) used as the physics core behind a wide spectrum of aerospace applications: FlightGear, Unreal Engine's Antoinette Project, ArduPilot and PX4 Software-in-the-Loop, reinforcement-learning gyms such as *gym-jsbsim*, and academic research running into thousands of citations. It implements a nonlinear, six-degree-of-freedom rigid-body simulation with an accurate Earth model (WGS-84 ellipsoid, rotation, Coriolis), the ISA 1976 standard atmosphere, configurable wind and turbulence, propulsion (piston, turbine, turboprop, rocket, electric, rotor), ground reactions and a fully scriptable XML-based flight control system.

1.1 What this document covers

This reference is organised as a top-down dive into JSBSim. We start with the architecture and execution loop, then walk every section of the aircraft XML, then go deep on aerodynamics (the math framework, tables, and conventions), and finish with a practical CFD-to-JSBSim workflow for building your own aircraft model. Every claim is rooted in source-code citations of the form `file:line`.

1.2 How JSBSim differs from a typical sim engine

- **XML-only modelling:** aircraft are described in declarative XML — no recompilation is needed to add a new airframe.
- **Function/table aerodynamics:** aerodynamic coefficients are expressed as *functions* that compose tables, properties and arithmetic operators. This makes the model fully data-driven and easy to populate from CFD or wind-tunnel data.
- **Property tree:** every state variable, control input, FCS output and aero coefficient is exposed under a hierarchical string path (`aero/qbar-psf`, `velocities/p-aero-rad_sec`, etc.). Everything is observable and most things are writable.
- **Round-Earth physics:** equations of motion are integrated in the ECI frame, including the Coriolis and centrifugal accelerations from Earth's rotation. Optional gravitational-torque modelling supports spacecraft.
- **Trim algorithm:** a robust bracketed root-finder trims the vehicle into steady flight (longitudinal, full, turn, pull-up, ground, custom).

1.3 Conventions used in this document

Distances and inertia use US customary units by default (feet, inches, slug·ft²) — JSBSim's default — but all elements accept an explicit `unit` attribute. Math symbols follow Etkin/Stevens-Lewis convention: α = angle of attack, β = sideslip, *p,q,r* = body angular rates, $\bar{q} = \frac{1}{2}\rho V^2$ = dynamic pressure, *S, b,* $\bar{c}$ = wing area, span, mean aerodynamic chord.

Chapter 2: Architecture and Execution Flow

JSBSim is structured around a central executive class (`FGFDMExec`) that owns a fixed-order list of *models* and drives them through a discrete-time loop. Each model reads its inputs from the property tree (filled by upstream models in the same tick) and writes its outputs back. Integrating the equations of motion is itself one of those models.

2.1 The model execution order

Models are enumerated in `FGFDMExec.h:225-241` and executed in this order every tick:

```
enum eModels {
  ePropagate=0,        // Integrate state from previous tick's accels
  eInput,              // Telnet/socket inputs
  eInertial,           // Gravity and Earth rotation vector
  eAtmosphere,         // T, p, rho, speed of sound at h
  eWinds,              // Steady wind + gusts + turbulence
  eSystems,            // FCS, autopilot, custom systems
  eMassBalance,        // CG, inertia tensor, total mass
  eAuxiliary,          // alpha, beta, qbar, Vt, Mach, ...
  ePropulsion,         // Engine thrust and torque
  eAerodynamics,       // Aero forces and moments
  eGroundReactions,    // Gear forces and friction
  eExternalReactions,  // User-defined external forces
  eBuoyantForces,      // Lighter-than-air models
  eAircraft,           // Sum forces and moments at CG
  eAccelerations,      // Solve F = m a, tau = I omega-dot
  eOutput              // CSV, sockets, FlightGear, ...
};
```

Why this order matters. *Propagate* runs first because it advances the state using the accelerations computed at the end of the *previous* tick. Then atmosphere and winds give pressure, density and the airmass-relative velocity that *Auxiliary* needs to compute α, β, $\bar{q}$, M — which the aerodynamics functions consume. Mass balance runs after FCS so a control law can request a pointmass shift (think: fuel transfer, stores release). Accelerations runs last; its outputs become the inputs to Propagate on the next tick.

2.2 The Run loop in pseudocode

```
FGFDMExec::Run() {                                    // FGFDMExec.cpp:404
    for child in childFDMs: child->Run()
    sim_time += dt; Frame++
    if (script) script->RunScript()
    for i in eModels:
        LoadInputs(i)        // copy upstream outputs to this model's inputs
        Models[i]->Run(holding)
}
```

Default integration step is `1.0 / 120.0` s (see `FGFDMExec.cpp:99`). At 120 Hz JSBSim runs faster than real time on a single CPU core even for complex aircraft. The step is set with `SetDeltaT()` or via the simulation script.

2.3 Modules at a glance

Module	Source	Responsibility
FGPropagate	models/FGPropagate.*	Integrate position, velocity, quaternion
FGAccelerations	models/FGAccelerations.*	Newton-Euler equations; ground friction LCP
FGAerodynamics	models/FGAerodynamics.*	Sum function-driven aero forces/moments
FGPropulsion	models/FGPropulsion.*	Engines (piston/turbine/rocket/electric)
FGGroundReactions	models/FGGroundReactions.*	Tires, struts, brakes, steering
FGAtmosphere	models/FGAtmosphere.*	ISA 1976 (T, p, ρ, a)
FGWinds	models/atmosphere/FGWinds.*	Steady wind, shear, Dryden/Karman turbulence
FGInertial	models/FGInertial.*	Gravity (spherical or WGS-84), planet rotation
FGMassBalance	models/FGMassBalance.*	Mass, CG, inertia tensor, pointmasses
FGAuxiliary	models/FGAuxiliary.*	α, β, q̄, Vt, Mach, gamma, load factors
FGFCS	models/FGFCS.*	Flight control system; channel/component graph
FGOutput	models/FGOutput.*	CSV, FlightGear binary, TCP, telnet
FGTrim	initialization/FGTrim.*	Bracketed root-finder for trim
FGPropertyManager	input_output/FGPropertyManager.*	Hierarchical property tree, all I/O via paths

Chapter 3: Coordinate Frames, Axes, and Units

Getting the frames right is the single most common source of bugs when building a new aircraft. JSBSim uses several frames at once and each section of the XML implicitly chooses one of them.

3.1 The structural frame (XML geometry)

All `<location>` elements — landing gear, CG, AERORP, pointmasses, engine thruster location — are expressed in the *structural* reference frame. This is an aircraft-fixed frame with the X axis usually pointing aft along the fuselage, Y to the right, Z up. The origin (0,0,0) is conventionally near the firewall or the nose datum. Inches are the default unit.

Inside the FDM these positions are converted to the body-fixed frame used by the equations of motion (X forward, Y right, Z down) — but you do **not** see that conversion in the XML.

3.2 The body-fixed (body) frame

All angular rates, accelerations, body-frame velocities and inertia components are in the body frame:

- **X** — forward, out through the nose.
- **Y** — out through the right wing.
- **Z** — downward, completing the right-handed triad.

Body-frame translational velocity is the property triple `velocities/u-fps, v-fps, w-fps`. Angular rates are `velocities/p-rad_sec, q-rad_sec, r-rad_sec`. The inertia tensor is given in the structural frame in the XML but is rotated to the body frame at load.

3.3 Wind (aerodynamic) and stability frames

The *wind* frame is rotated from the body frame by sideslip β and angle of attack α: its X axis points along the relative wind, drag acts in –X, lift acts in –Z (up relative to airflow). When you write `<axis name="LIFT">` in the aerodynamics section JSBSim assumes you are in wind axes. The *stability* frame is an intermediate frame rotated from body only by α (no β); some classical stability derivatives are tabulated in that frame and you can opt into it with `frame="STABILITY"`.

JSBSim handles the conversion automatically: forces computed in *LIFT/DRAG/SIDE* are rotated by T_{w2b} into the body frame and moments are added at the AERORP, then transferred to the CG by $r \times F$.

3.4 Local NED and ECI frames

Local NED (North-East-Down) is the local-tangent frame centered at the aircraft. JSBSim uses it for winds, ground track (γ, ψ_{gt}), and the local Euler angles ϕ, θ, ψ. **ECI** (Earth-Centred Inertial) is the non-rotating frame in which the equations of motion are actually integrated; `FGPropagate` maintains $v_{inertial}$ and $r_{inertial}$ internally and transforms them to body/local for output.

3.5 Sign conventions

- **Elevator deflection**: positive trailing-edge-down (creates negative pitching moment), output to `fcs/elevator-pos-rad`.
- **Aileron deflection**: differential — the C-172 uses `left-aileron-pos-rad` with positive = down (rolls right).
- **Rudder deflection**: positive trailing-edge-left (yaws nose left).
- **Inertia cross products**: classical convention is I_{xz} as the integral of *+xz dm*. Some references use the negated convention; set `negated_crossproduct_inertia="true"` on the *<mass_balance>* element to

switch.

• **Up vs. down**: Z_{body} is positive down. The structural Z given in the XML is usually positive up — JSBSim does the flip internally.

3.6 Units

```
Length              FT (default), IN (default for <location>), M, KM, CM
Area                FT2 (default), M2, IN2, CM2
Mass                LBS (default), KG, SLUG  (1 slug = 14.594 kg)
Inertia             SLUG*FT2 (default), KG*M2
Force               LBS, N
Pressure            PSF (default), PSI, INHG, ATM, PA, N/M2
Velocity (output)   FPS, KTS, M/S  (per-property)
Angle               RAD (aero default), DEG (FCS default)
Spring              LBS/FT, N/M
Damping             LBS/FT/SEC, N/M/SEC
```

Always declare units explicitly on every element that accepts a `unit` attribute — silent default mismatches are responsible for the majority of "my aircraft launches itself sideways" bug reports.

Chapter 4: The Aircraft XML, Section by Section

An aircraft is a single XML file with the root element `<fdm_config>`. The order of sections is enforced by the XSD; the canonical sequence is shown below. Below the cover material follow ~10 sub-chapters, one per major section.

4.1 Root: fdm_config

```
<?xml version="1.0"?>
<fdm_config name="MyAircraft" version="2.0" release="PRODUCTION"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:noNamespaceSchemaLocation="http://jsbsim.sourceforge.net/JSBSim.xsd">

    <fileheader>          ...  </fileheader>
    <planet>              ...  </planet>      <!-- optional -->
    <metrics>             ...  </metrics>
    <mass_balance>        ...  </mass_balance>
    <ground_reactions>    ...  </ground_reactions>
    <external_reactions>  ...  </external_reactions>
    <buoyant_forces>      ...  </buoyant_forces>
    <propulsion>          ...  </propulsion>
    <system  .../>                          <!-- 0..N -->
    <autopilot ...>       ...  </autopilot>   <!-- 0..1 -->
    <flight_control ...>  ...  </flight_control>
    <aerodynamics>        ...  </aerodynamics>
    <input ...>           ...  </input>       <!-- 0..N -->
    <output ...>          ...  </output>      <!-- 0..N -->
</fdm_config>
```

Attributes: `name` (required) — display name; `version` (required, currently 2.0); `release` = PRODUCTION | ALPHA | BETA (ALPHA/BETA print a warning at load time).

4.2 fileheader: authorship and provenance

Pure metadata. JSBSim ignores everything inside but tools that scan a fleet of models read it. Use it.

```
<fileheader>
    <author>Ada Lovelace</author>
    <email>ada@example.com</email>
    <organization>Acme Aircraft</organization>
    <license licenseName="GPL"
             licenseURL="http://www.gnu.org/licenses/gpl.html"/>
    <filecreationdate>2026-05-24</filecreationdate>
    <version>$Revision: 1.0 $</version>
    <description>Acme A-1 light sport aircraft.</description>
    <note>Stall hysteresis modelled per NACA TN-3909.</note>
    <limitation>Gear aero drag not modelled.</limitation>
    <reference title="AGARD-AR-265"
               author="ESDU"
               date="1988"
               URL="https://example.org/AR265.pdf"/>
</fileheader>
```

4.3 metrics: aircraft geometry

`<metrics>` establishes the non-dimensionalisation: *S* (wing area), *b* (wing span), $\bar{c}$ (mean aerodynamic chord), tail areas/arms, and the three *reference points* AERORP, EYEPOINT and VRP.

```
<metrics>
    <wingarea  unit="FT2"> 174.0  </wingarea>
    <wingspan  unit="FT" >  35.8  </wingspan>
    <chord     unit="FT" >   4.9  </chord>
```

```
    <htailarea unit="FT2">  21.9  </htailarea>
    <htailarm  unit="FT" >  15.7  </htailarm>
    <vtailarea unit="FT2">  16.5  </vtailarea>
    <vtailarm  unit="FT" >   0    </vtailarm>
    <wing_incidence unit="DEG"> 1.5 </wing_incidence>  <!-- optional -->

    <location name="AERORP" unit="IN">  <!-- 25% MAC -->
        <x>43.2</x> <y>0</y> <z>59.4</z>
    </location>
    <location name="EYEPOINT" unit="IN">
        <x>37</x> <y>0</y> <z>48</z>
    </location>
    <location name="VRP" unit="IN">
        <x>42.6</x> <y>0</y> <z>38.5</z>
    </location>
</metrics>
```

The three reference points have specific meaning:

- **AERORP** — aerodynamic reference point. All aerodynamic moments are first summed about this point, then transferred to the CG by $M_{cg} = M_{arp} + r_{arp \to cg} \times F$. Conventionally placed at the 25% mean aerodynamic chord on the wing.
- **EYEPOINT** — pilot eye position. Used by FlightGear and visual systems; affects the load factor at the pilot's head (`accelerations/n-pilot-{x,y,z}-norm`).
- **VRP** — visual reference point, origin used by external viewers.

4.4 mass_balance: weight, CG, inertia

The empty weight plus a list of *pointmasses* is what JSBSim uses to compute total mass, CG location, and the inertia tensor at the CG. Pointmasses include crew, passengers, cargo, and user-defined items that may move at runtime (slung loads, transferable fuel). Tank mass is added automatically by the propulsion subsystem and consumed during flight.

```
<mass_balance negated_crossproduct_inertia="false">
    <ixx unit="SLUG*FT2">  948 </ixx>
    <iyy unit="SLUG*FT2"> 1346 </iyy>
    <izz unit="SLUG*FT2"> 1967 </izz>
    <ixy unit="SLUG*FT2">   0  </ixy>
    <ixz unit="SLUG*FT2">   0  </ixz>
    <iyz unit="SLUG*FT2">   0  </iyz>
    <emptywt unit="LBS"> 1500 </emptywt>
    <location name="CG" unit="IN">
        <x>41</x> <y>0</y> <z>36.5</z>
    </location>
    <pointmass name="Pilot">
        <weight unit="LBS"> 180 </weight>
        <location unit="IN"> <x>36</x><y>-14</y><z>24</z> </location>
    </pointmass>
    <pointmass name="Baggage">
        <weight unit="LBS">  0  </weight>
        <location unit="IN"> <x>95</x><y>0</y><z>24</z> </location>
        <form shape="cylinder">  <!-- optional inertia shape -->
            <radius unit="IN">  6 </radius>
            <length unit="IN"> 24 </length>
        </form>
    </pointmass>
</mass_balance>
```

Inertia about the CG: the values you put in `ixx, iyy, izz, ixy, ixz, iyz` are the inertia of the empty-weight structure about the CG, in the *structural* frame. JSBSim rotates them to the body frame, then adds the parallel-axis contributions of each pointmass. The `negated_crossproduct_inertia` attribute exists because half the textbooks use the opposite sign convention for products of inertia.

4.5 ground_reactions: gear, skids and contacts

`<contact>` elements describe anything that can touch the ground. There are two types:

- **BOGEY** — a wheel. Has a strut (spring + damper), tire friction (static, dynamic, rolling), optional steering, optional brakes, and optional retraction.
- **STRUCTURE** — a non-rolling contact such as a wing tip, skid, fuselage or tail bumper. Typically very stiff with low friction.

```
<ground_reactions>
    <contact type="BOGEY" name="NOSE">
        <location unit="IN">
            <x>-6.8</x> <y>0</y> <z>-19.5</z>
        </location>
        <static_friction>  0.80 </static_friction>
        <dynamic_friction> 0.50 </dynamic_friction>
        <rolling_friction> 0.02 </rolling_friction>
        <spring_coeff  unit="LBS/FT">     1800 </spring_coeff>
        <damping_coeff unit="LBS/FT/SEC"> 600  </damping_coeff>
        <damping_coeff_rebound unit="LBS/FT/SEC"> 1200 </damping_coeff_rebound>
        <max_steer unit="DEG"> 10 </max_steer>
        <brake_group> NONE </brake_group>
        <retractable>0</retractable>
    </contact>
    <contact type="BOGEY" name="LEFT_MAIN">
        <location unit="IN"> <x>58.2</x><y>-43</y><z>-15.5</z> </location>
        <static_friction>0.8</static_friction>
        <dynamic_friction>0.5</dynamic_friction>
        <rolling_friction>0.02</rolling_friction>
        <spring_coeff  unit="LBS/FT">     5400 </spring_coeff>
        <damping_coeff unit="LBS/FT/SEC"> 1600 </damping_coeff>
        <brake_group> LEFT </brake_group>
    </contact>
    <contact type="STRUCTURE" name="LEFT_WINGTIP">
        <location unit="IN"> <x>43.2</x><y>-214.8</y><z>59.4</z> </location>
        <static_friction>0.2</static_friction>
        <dynamic_friction>0.2</dynamic_friction>
        <spring_coeff  unit="LBS/FT">     20000 </spring_coeff>
        <damping_coeff unit="LBS/FT/SEC">  2000 </damping_coeff>
    </contact>
</ground_reactions>
```

Friction forces are computed by a projected-Gauss-Seidel iterative Lagrange-multiplier solver (Catto 2005) inside `FGAccelerations::CalculateFrictionForces()`, so multi-wheel contacts respect the no-interpenetration and Coulomb-friction constraints simultaneously.

4.6 external_reactions: bolt-on forces

Anything that applies a force or moment that is neither aero, nor propulsion, nor ground contact goes here: parachutes, towlines, JATO bottles, magnetic launchers.

```
<external_reactions>
    <property>fcs/parachute_reef_pos_norm</property>  <!-- declare -->

    <force name="parachute" frame="WIND">
        <function>
            <product>
                <property>aero/qbar-psf</property>
                <property>fcs/parachute_reef_pos_norm</property>
                <value>1.0</value>
                <value>10000.0</value>     <!-- chute area in sq-ft -->
            </product>
        </function>
        <location unit="FT"> <x>1</x><y>0</y><z>0</z> </location>
        <direction> <x>-1</x><y>0</y><z>0</z> </direction>
    </force>
</external_reactions>
```

4.7 propulsion: engines, tanks, thrusters

Propulsion is two-tier: an *engine* XML file (loaded from `$JSBSIM_ROOT/engine/`) describes the engine; the aircraft XML references it and adds a *thruster* (the body that applies the actual force) and one or more *tanks*. Engine types are:

- **piston_engine** — internal-combustion piston engine with power tables vs. RPM, throttle, mixture, MAP, altitude.
- **turbine_engine** — turbojet/turbofan with idle/military/AB tables.
- **turboprop_engine** — turboprop with shaft horsepower tables.
- **rocket_engine** — solid or liquid rocket with thrust/Isp.
- **electric_engine** — DC motor, power × throttle.
- **brushless_dc_motor** — torque vs. RPM curve.
- **rotor** — helicopter rotor (induced flow + flapping).

```
<propulsion>
    <engine file="eng_io320">
        <feed>0</feed>          <!-- consume from tank 0 -->
        <feed>1</feed>
        <thruster file="prop_75in2f">
            <location unit="IN"> <x>-37.7</x><y>0</y><z>26.6</z> </location>
            <orient   unit="DEG">
                <pitch>0</pitch><roll>0</roll><yaw>0</yaw>
            </orient>
            <sense>1</sense>      <!-- 1 = right-hand prop -->
            <p_factor>5</p_factor> <!-- effective offset for P-factor -->
        </thruster>
    </engine>

    <tank type="FUEL" number="0">
        <location unit="IN"> <x>56</x><y>-112</y><z>59.4</z> </location>
        <type>AVGAS</type>
        <capacity unit="LBS"> 185 </capacity>
        <contents unit="LBS"> 100 </contents>
        <priority>1</priority>
    </tank>
    <tank type="FUEL" number="1">
        <location unit="IN"> <x>56</x><y> 112</y><z>59.4</z> </location>
        <capacity unit="LBS"> 185 </capacity>
        <contents unit="LBS"> 100 </contents>
    </tank>
</propulsion>
```

An example engine file (`engine/eng_io320.xml`):

```
<piston_engine name="IO320">
    <minmp     unit="INHG">  10.0 </minmp>
    <maxmp     unit="INHG">  28.5 </maxmp>
    <displacement unit="IN3"> 320.0 </displacement>
    <maxhp>                 160.0 </maxhp>
    <bsfc>                    0.32 </bsfc>
    <cycles>                  4.0  </cycles>
    <idlerpm>               550.0  </idlerpm>
    <maxrpm>               2700.0  </maxrpm>
    <maxthrottle>             1.0  </maxthrottle>
    <minthrottle>             0.1  </minthrottle>
    <sparkfaildrop>           0.1  </sparkfaildrop>
</piston_engine>
```

4.8 flight_control: command shaping

The FCS is a directed graph of components arranged into named *channels*. Inputs come from pilot commands (`fcs/elevator-cmd-norm`, etc., set by the input device or script) and outputs flow into the aerodynamic deflection properties (`fcs/elevator-pos-rad`, etc.) that the aerodynamics functions read.

```
<flight_control name="FCS: c172">

    <channel name="Pitch">
        <summer name="Pitch Trim Sum">
            <input>fcs/elevator-cmd-norm</input>
            <input>fcs/pitch-trim-cmd-norm</input>
            <clipto> <min>-1</min><max>1</max> </clipto>
        </summer>

        <aerosurface_scale name="Elevator Control">
            <input>fcs/pitch-trim-sum</input>
            <gain>0.01745</gain>           <!-- deg -> rad -->
            <range> <min>-28</min><max>23</max> </range>
            <output>fcs/elevator-pos-rad</output>
        </aerosurface_scale>
    </channel>

    <channel name="Flaps">
        <kinematic name="Flaps Control">
            <input>fcs/flap-cmd-norm</input>
            <traverse>
                <setting><position>0</position> <time>0</time></setting>
                <setting><position>10</position><time>2</time></setting>
                <setting><position>20</position><time>2</time></setting>
                <setting><position>30</position><time>2</time></setting>
            </traverse>
            <output>fcs/flap-pos-deg</output>
        </kinematic>
    </channel>
</flight_control>
```

Available FCS components include `summer`, `pure_gain`, `scheduled_gain`, `aerosurface_scale`, `kinematic`, `deadband`, `switch`, `fcs_function`, `pid`, `lag_filter`, `lead_lag_filter`, `washout_filter`, `second_order_filter`, `integrator`, `sensor`, `actuator` and the generic `fcs_function` which lets you embed any function-language expression as a block. PIDs and filters discretise their continuous-time form using the simulation *dt*.

4.9 autopilot and system: same toolbox, different scope

`<autopilot>` and `<system>` use the same FCS components and channel structure. They are split out because they execute as separate sub-groups: typical practice is to put basic stick/pedal shaping in *flight_control*, classical autopilot loops (altitude-hold, heading-hold, autothrottle) in *autopilot*, and avionics or electrical-system logic in *system* (which may be loaded from external files in `$AC/Systems/`).

4.10 aerodynamics: the core of the model

Aerodynamics is large enough to warrant its own chapter — see Chapter 5. The skeleton is:

```
<aerodynamics>

    <alphalimits unit="RAD">          <!-- physical alpha clamp -->
        <min>-0.087</min>
        <max> 0.280</max>
    </alphalimits>

    <hysteresis_limits unit="RAD">    <!-- stall hysteresis -->
        <min>0.09</min>
        <max>0.36</max>
    </hysteresis_limits>

    <!-- Helper functions (often ground effect, induced velocity) -->
    <function name="aero/function/kCLge"> ... </function>

    <!-- Six axes: three forces and three moments -->
    <axis name="DRAG">  ...  </axis>
    <axis name="SIDE">  ...  </axis>
    <axis name="LIFT">  ...  </axis>
```

```
    <axis name="ROLL">  ...  </axis>
    <axis name="PITCH"> ...  </axis>
    <axis name="YAW">   ...  </axis>
</aerodynamics>
```

4.11 input and output

<input> enables an external interface, typically a TCP/UDP socket for telnet-style command/query interaction. <output> is much more common: a logging configuration for CSV files, FlightGear binary, sockets, etc.

```
<output name="flight.csv" type="CSV" rate="60">
    <property> aero/qbar-psf </property>
    <property> velocities/vt-fps </property>
    <rates>     ON </rates>
    <velocities>ON </velocities>
    <forces>    ON </forces>
    <moments>   ON </moments>
    <position>  ON </position>
    <fcs>       OFF</fcs>
    <aerosurfaces>OFF</aerosurfaces>
</output>

<output type="FLIGHTGEAR" rate="60" protocol="UDP" port="5500"/>

<input port="1138"/>     <!-- telnet on TCP 1138 -->
```

Chapter 5: Aerodynamics — Deep Dive

5.1 The six axes and the function/table model

Inside `<aerodynamics>` the heart of the model is an arbitrary number of `<function>` elements grouped into `<axis>` blocks. JSBSim sums every function inside an axis to obtain that axis's total force or moment *in physical units* (lbf, ft·lbf). It does **not** do any additional non-dimensionalisation — you embed the $\bar{q}\cdot S$ factor yourself, by convention.

Axis names map to indices (FGAerodynamics.cpp:57-69):

```
Forces (axis index 0..2)   Moments (axis index 3..5)
  0  DRAG                     3  ROLL  (l)
  1  SIDE                     4  PITCH (m)
  2  LIFT                     5  YAW   (n)

Alternative force axes:        Alternative moment frames:
  X, Y, Z       (body)         frame="WIND"
  AXIAL, NORMAL (axial/norm)   frame="STABILITY"
```

5.2 How a single function evaluates

Every function returns a scalar each tick. A canonical drag term looks like:

```
<function name="aero/coefficient/CDo">
    <description>Drag at zero lift</description>
    <product>
        <property>aero/qbar-psf</property>     <!-- q-bar -->
        <property>metrics/Sw-sqft</property>   <!-- S -->
        <value>0.027</value>                   <!-- CD0 -->
    </product>
</function>
```

$$F_{D,0} = C_{D0} \cdot \bar{q} \cdot S$$

Inside `FGAerodynamics::Run()` the values returned by all `<axis name="DRAG">` children are summed into the DRAG slot of `vFnative`; the LIFT and SIDE slots are filled the same way; then a rotation by T_{w2b} lifts wind-axis forces into the body frame, the moments computed by ROLL/PITCH/YAW are added at the AERORP, and the moments are finally transferred to the CG by $M_{cg} = M_{arp} + r_{arp\to cg} \times F$.

5.3 The function language

JSBSim's function language is a small Lisp-ish XML expression tree. The leaves are `<value>` (constant) and `<property>` (reference to the property tree); the internal nodes are operators.

```
Arithmetic      sum, difference, product, quotient, pow, sqrt,
                abs, min, max, avg
Trigonometric   sin, cos, tan, asin, acos, atan, atan2
Exponential     exp, ln, log2, log10
Logical         lt, le, gt, ge, eq, nq, and, or, not,
                ifthen, switch
Modular         mod, floor, ceil, fmod, roundmultiple
Unit            toradians, todegrees
Random          random      (Gaussian),  urandom (uniform)
Constants       pi, value/v, integer
Table           table   (1-D, 2-D, 3-D, n-D)
Property        property/p
```

A more interesting example — induced drag, $C_{Di} = C_L^2 / (\pi \cdot AR \cdot e)$ — turned into a JSBSim function:

```
<function name="aero/coefficient/CDi">
    <description>Induced drag</description>
    <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
```

```
        <quotient>
            <pow>
                <property>aero/cl-squared</property>     <!-- CL^2 -->
                <value>0.5</value>                       <!-- *not* powed -->
            </pow>
            <product>
                <pi/>
                <property>metrics/aspectratio</property>
                <value>0.85</value>                       <!-- e -->
            </product>
        </quotient>
    </product>
</function>
```

5.4 Tables: 1-D, 2-D, 3-D, and higher

Tables are the workhorse of any non-trivial model — they let you encode the CFD-derived lookup of any coefficient against any combination of state variables. JSBSim uses **linear interpolation** in every dimension; outside the tabulated range the boundary value is clamped (no extrapolation), so it is your job to cover the operational envelope.

1-D table (a vector lookup against one variable):

```
<table>
    <independentVar lookup="row">aero/alpha-rad</independentVar>
    <tableData>
        -0.0900  -0.2200
         0.0000   0.2500
         0.0900   0.7300
         0.1500   1.0000
         0.2800   1.1500
    </tableData>
</table>
```

2-D table (e.g. drag vs alpha and flap angle):

```
<table>
    <independentVar lookup="row">aero/alpha-rad</independentVar>
    <independentVar lookup="column">fcs/flap-pos-deg</independentVar>
    <tableData>
                 0.0     10.0    20.0    30.0
        -0.0873  0.0041  0.0000  0.0005  0.0014
        -0.0349  0.0003  0.0057  0.0108  0.0141
         0.0000  0.0052  0.0168  0.0251  0.0299
         0.0524  0.0240  0.0452  0.0583  0.0655
         0.1571  0.0962  0.1353  0.1573  0.1690
    </tableData>
</table>
```

3-D table (e.g. pitching moment vs alpha, elevator, Mach). You stack 2-D tables, each with a `breakPoint` attribute for the third dimension:

```
<table>
    <independentVar lookup="row">aero/alpha-rad</independentVar>
    <independentVar lookup="column">fcs/elevator-pos-rad</independentVar>
    <independentVar lookup="table">velocities/mach</independentVar>

    <tableData breakPoint="0.30">           <!-- Mach = 0.30 -->
             -0.4  -0.2   0.0   0.2   0.4
       -0.1   0.20  0.11  0.04 -0.06 -0.16
        0.0   0.15  0.07  0.00 -0.07 -0.15
        0.1   0.11  0.04 -0.04 -0.12 -0.21
    </tableData>

    <tableData breakPoint="0.80">           <!-- Mach = 0.80 -->
             -0.4  -0.2   0.0   0.2   0.4
       -0.1   0.18  0.10  0.03 -0.05 -0.14
        0.0   0.13  0.06  0.00 -0.06 -0.13
        0.1   0.10  0.03 -0.03 -0.10 -0.19
    </tableData>
```

```
</table>
```

Higher dimensions are supported recursively. In practice 3-D is the limit of what you should build by hand; for 4-D and above use a script to generate the XML from a CFD database.

5.5 Properties commonly used in aerodynamics

```
aero/qbar-psf                q-bar = 0.5 * rho * Vt^2  (psf)
aero/alpha-rad               angle of attack (radians)
aero/beta-rad                sideslip angle (radians)
aero/alphadot-rad_sec        alpha-dot
aero/betadot-rad_sec         beta-dot
aero/mag-beta-rad            |beta|  (useful in drag terms)
aero/bi2vel                  b / (2 V)   --- roll-rate non-dim factor
aero/ci2vel                  cbar / (2 V)--- pitch-rate non-dim factor
aero/h_b-mac-ft              height above ground / chord, for ground effect
aero/cl-squared              CL^2 (computed from previous tick's lift)
velocities/p-aero-rad_sec    roll rate in aero frame  (rad/s)
velocities/q-aero-rad_sec    pitch rate in aero frame
velocities/r-aero-rad_sec    yaw rate in aero frame
velocities/mach              Mach number
velocities/vt-fps            true airspeed (ft/s)
metrics/Sw-sqft              wing area
metrics/bw-ft                wing span
metrics/cbarw-ft             mean aerodynamic chord
fcs/elevator-pos-rad         elevator deflection from FCS output
fcs/aileron-pos-rad          aileron
fcs/rudder-pos-rad           rudder
fcs/flap-pos-deg             flap (commonly in degrees)
gear/gear-pos-norm           gear position (0..1)
fcs/speedbrake-pos-rad       speedbrake
```

The aero-frame angular rates `p-aero`, `q-aero`, `r-aero` are the body-frame rates rotated into the wind frame (rotated by α only — the stability frame). Always use these for the damping derivatives, not the raw `p-rad_sec`.

5.6 Non-dimensionalisation by hand

Because JSBSim does not non-dimensionalise for you, you wear the responsibility of writing the dimensional formula correctly. These are the textbook formulas you embed in `<product>`:

$$\text{Lift} = C_L \cdot \bar{q} \cdot S$$

$$\text{Drag} = C_D \cdot \bar{q} \cdot S$$

$$\text{Side} = C_Y \cdot \bar{q} \cdot S$$

$$\text{Roll moment} = C_l \cdot \bar{q} \cdot S \cdot b$$

$$\text{Pitch moment} = C_m \cdot \bar{q} \cdot S \cdot \bar{c}$$

$$\text{Yaw moment} = C_n \cdot \bar{q} \cdot S \cdot b$$

For damping derivatives multiply by the appropriate *b/(2V)* or *c̄/(2V)* factor:

$$\text{Roll-rate moment} = C_{lp} \cdot \bar{q} \cdot S \cdot b \cdot [b/(2V)] \cdot p$$

$$\text{Pitch-rate moment} = C_{mq} \cdot \bar{q} \cdot S \cdot \bar{c} \cdot [\bar{c}/(2V)] \cdot q$$

$$\text{Yaw-rate moment} = C_{nr} \cdot \bar{q} \cdot S \cdot b \cdot [b/(2V)] \cdot r$$

$$\dot{\alpha}\ \text{moment} = C_{m\_\dot{\alpha}} \cdot \bar{q} \cdot S \cdot \bar{c} \cdot [\bar{c}/(2V)] \cdot \dot{\alpha}$$

5.7 Worked example: rolling moment due to roll rate

From `aircraft/c172p/c172p.xml:733`:

```
<function name="aero/coefficient/Clp">
    <description>Roll moment due to roll rate (roll damping)</description>
    <product>
        <property>aero/qbar-psf</property>           <!-- q-bar -->
        <property>metrics/Sw-sqft</property>         <!-- S    -->
        <property>metrics/bw-ft</property>           <!-- b    -->
        <property>aero/bi2vel</property>             <!-- b/(2V) -->
        <property>velocities/p-aero-rad_sec</property> <!-- p   -->
        <value>-0.4840</value>                         <!-- Clp  -->
    </product>
</function>
```

Cl_p for the C-172 is constant at −0.484 — the wing's roll damping. Multiplying out gives a body-frame rolling moment in ft·lbf, which is what the ROLL axis expects.

5.8 Static derivatives — getting them from CFD

Static derivatives (∂C/∂α, ∂C/∂β, ∂C/∂δ) characterise the *steady* aerodynamic response of the aircraft to changes in α, β and control deflections. They are obtained from CFD by running the aircraft as a rigid body in steady-state at a grid of (α, β, δ, M) points and reading off the resulting forces and moments. The list below is a typical minimum sweep for a subsonic conventional aircraft:

Derivative	Independent vars	CFD sweep
$C_L(\alpha, M, flap)$	α, M, flap	steady cruise over α ∈ [−6°, 18°], stalled = clip
$C_D(\alpha, M, flap, gear)$	α, M, flap, gear	Same matrix; CD = drag/q̄S
$C_m(\alpha, M, \delta_e)$	α, M, δ_e	α-sweep at each δ_e and Mach
$C_Y(\beta, \delta_r)$	β, δ_r	β-sweep ∈ [−15°, 15°]
$C_l(\beta, \alpha, \delta_a)$	β, α, δ_a	β-sweep + δ_a increments
$C_n(\beta, \alpha, \delta_r)$	β, α, δ_r	β-sweep + δ_r increments
$C_{L\alpha}$	α	slope of CL-α curve
$C_{m\alpha}$	α	slope of Cm-α curve (negative → stable)
$C_{n\beta}$	β	slope of Cn-β curve (positive → stable)
$C_{l\beta}$	β	dihedral effect

5.9 Dynamic (damping) derivatives — running CFD with motion

Dynamic derivatives describe the moment created by a rotation rate (p, q, r, $\dot{\alpha}$). They cannot be obtained from a steady-state snapshot of the geometry; you need *moving* CFD or the classical engineering analogues.

- **Forced oscillation**: in URANS or LES, run the aircraft oscillating sinusoidally about the relevant axis at a known amplitude and reduced frequency. Extract the in-phase and out-of-phase moment components — these give the static and damping derivatives respectively. Industry-standard tools: ANSYS Fluent, OpenFOAM with overset, STAR-CCM+, NASA OVERFLOW.
- **Quasi-steady method**: run several steady CFD cases with a constant rotation rate *p* imposed on the inertial frame; extract the rolling moment from each. The slope $\Delta C_l/\Delta(pb/2V)$ is C_{lp}. Fast but limited to small angles.
- **Strip-theory / lifting-line**: a cheap engineering fallback. For early model bring-up, take handbook estimates (Roskam, Etkin, USAF DATCOM) — they are within ±30% of high-fidelity data for conventional aircraft.
- **System ID**: if you already have flight-test data, fit the damping derivatives by minimising the simulation residual on a doublet manoeuvre. This is what AIAA SciTech "flight-validated model" papers actually do.

Damping derivatives you should obtain at minimum:

Symbol	Meaning	Typical sign	Method
C_{lp}	Roll damping	negative	forced oscillation about X
C_{lr}	Roll due to yaw rate	positive	forced oscillation about Z
C_{mq}	Pitch damping	negative	forced oscillation about Y
$C_{m\_\dot{\alpha}}$	Pitch due to $\dot{\alpha}$	negative	forced plunge oscillation
C_{nr}	Yaw damping	negative	forced oscillation about Z
C_{np}	Yaw due to roll	small, signed	forced oscillation about X
C_{Yp}	Side force due to roll	small	from same X-oscillation run
C_{Yr}	Side force due to yaw	small	from same Z-oscillation run

5.10 Control derivatives

Control derivatives ($Cm_{\delta e}$, $Cl_{\delta a}$, $Cn_{\delta r}$, ...) are the slopes of moment vs. control deflection. They are static, so you obtain them in the same steady CFD batch as the static derivatives, simply by including control deflection as a sweep axis. For non-linear control authority (very common at high α or high deflection) prefer a 2-D table (α, δ) over a single coefficient.

```
<function name="aero/coefficient/Cmde">
    <description>Pitch moment due to elevator</description>
    <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <property>metrics/cbarw-ft</property>
        <table>
            <independentVar lookup="row">aero/alpha-rad</independentVar>
            <independentVar lookup="column">fcs/elevator-pos-rad</independentVar>
            <tableData>
                      -0.40  -0.20   0.00   0.20   0.40
              -0.10    0.22   0.11   0.00  -0.11  -0.22
               0.00    0.21   0.10   0.00  -0.10  -0.21
               0.10    0.18   0.09   0.00  -0.09  -0.18
               0.20    0.12   0.06   0.00  -0.06  -0.12
            </tableData>
        </table>
    </product>
</function>
```

5.11 Ground effect and Reynolds effects

Ground effect is conventionally implemented as a *multiplier* on lift and drag, evaluated as a 1-D table against height-above-ground normalised by mean chord (`aero/h_b-mac-ft`):

```
<function name="aero/function/kCLge">
    <description>Lift multiplier due to ground effect</description>
    <table>
        <independentVar>aero/h_b-mac-ft</independentVar>
        <tableData>
            0.00  1.203
            0.10  1.127
            0.20  1.073
            0.50  1.019
            1.00  1.002
            1.10  1.000
        </tableData>
    </table>
</function>
```

Use `aero/function/kCLge` as a factor in the lift function. The same approach handles Mach effects (table vs. `velocities/mach`) and Reynolds (table vs. `velocities/reynolds`) if you have the CFD data to populate them.

5.12 Stall and stall hysteresis

`<alphalimits>` clips the *physical* α used in aero calculations. `<hysteresis_limits>` defines a switching band — JSBSim exposes `aero/stall-hyst-norm` which is 0 below the low limit and 1 above the high limit. Use it as a second axis of a lift table to encode the difference between the un-stalled and stalled lift curves and get correct stall-recovery behaviour.

Chapter 6: The Equations of Motion Under the Hood

All the XML and tables eventually feed into a small set of ordinary differential equations. This chapter walks the math as JSBSim implements it.

6.1 State vector

```
VehicleState {                                    // FGPropagate.h:100
  FGLocation     vLocation;          // ECEF position (ft)
  FGColumnVector3 vUVW;              // body-frame velocity rel. ECEF
  FGColumnVector3 vPQR;              // body-frame angular rate rel. ECEF
  FGColumnVector3 vPQRi;             // body-frame angular rate rel. ECI
  FGQuaternion   qAttitudeLocal;     // body w.r.t. local NED
  FGQuaternion   qAttitudeECI;       // body w.r.t. ECI  <-- primary
  FGColumnVector3 vInertialVelocity; // ECI frame velocity
  FGColumnVector3 vInertialPosition; // ECI frame position
  // ... history deques for multi-step integrators
};
```

6.2 Quaternion attitude kinematics

Attitude is propagated as the quaternion $q_{i\to b}$ from inertial to body. The time derivative is the kinematic equation

$$\dot{q} = \tfrac{1}{2} \cdot q \otimes \omega_{b/i}$$

where $\omega_{b/i}$ is the angular velocity of the body with respect to the inertial frame, expressed in the body frame (`vPQRi`). JSBSim's `FGQuaternion::GetQDot()` computes exactly this; `FGPropagate::CalculateQuatdot()` wraps it. Multiple integrators are available:

```
0  eNone               freeze
1  eRectEuler          q(n+1) = q(n) + dt * q-dot(n)
2  eTrapezoidal        q(n+1) = q(n) + 0.5*dt*(q-dot(n)+q-dot(n-1))
3  eAdamsBashforth2    q(n+1) = q(n) + dt*(1.5 q-dot(n) - 0.5 q-dot(n-1))
4  eAdamsBashforth3    23/12, -16/12, +5/12 weights
5  eAdamsBashforth4    55/24, -59/24, 37/24, -9/24
6  eAdamsBashforth5    classical 5-step coefficients
7  eBuss1              q(n+1) = q(n) * exp(0.5*dt*omega)
8  eBuss2              augmented with omega-dot terms
9  eLocalLinearization Barker et al. (NASA TN D-7347)
```

Default integrators are rectangular Euler for attitude and angular rate, Adams-Bashforth-2 for translational velocity, Adams-Bashforth-3 for position. The Buss integrators are interesting because they preserve quaternion norm exactly (no need for the renormalisation that rectangular Euler requires every tick).

6.3 Translational dynamics in the inertial frame

JSBSim solves Newton's second law in the inertial frame and transforms back to body coordinates for output:

$$\dot{v}_{body} = F/m - (p_{body} + 2\cdot T_{i2b}\cdot\Omega_{planet}) \times v_{body} - T_{i2b}\cdot\Omega_{planet} \times (\Omega_{planet} \times r_i)$$

The first term is the conventional $F = ma$. The second is the Coriolis acceleration, including the contribution from Earth's rotation ($\Omega_{planet} \approx 7.292\cdot10$■■ rad/s about the ECI Z axis). The third is the centrifugal acceleration from rotating about the Earth's centre. For terrestrial flight at transport-aircraft scales these are small but visible — they are what makes the simulator agree with the NASA-2015 check-cases.

6.4 Rotational dynamics

$$J \cdot \dot{\omega}_i = M - \omega_i \times (J \cdot \omega_i)$$

The term $\omega \times (J\omega)$ is the Euler gyroscopic torque (non-zero whenever the inertia tensor is not isotropic). `FGAccelerations::CalculatePQRdot()` computes this directly. The inertia tensor J in the body frame is maintained by *FGMassBalance* and is updated every tick to account for fuel burn and pointmass shifts. An optional gravity-gradient torque is enabled via the property `simulation/gravitational-torque`; useful for orbital mechanics.

6.5 Total force and moment assembly

`FGAircraft` sums forces and moments from *aero, propulsion, ground, external, buoyant* models and exposes them as a single body-frame F and M to the Accelerations model. Gravity is treated separately by `FGInertial` (either spherical or WGS-84 with the J2 term) and added directly to the acceleration vector — keeping it out of M means the moment of gravity about the CG is automatically zero (gravity acts at the CG by definition).

6.6 Ground friction as a constrained problem

Standard explicit integration of stiff spring-damper landing-gear models is unstable. JSBSim instead poses the friction problem as a *linear-complementarity problem (LCP)*: each contact point imposes a non-penetration constraint normal to the runway and a Coulomb-friction constraint tangent to it. The unknowns are the Lagrange multipliers λ. The system $A \cdot \lambda = b$ with $A = J \cdot M^{-1} \cdot J$■ is solved iteratively (Projected Gauss-Seidel, Catto 2005) up to 50 iterations per tick — see `FGAccelerations::CalculateFrictionForces()`. The contact friction forces and the moment $r \times F$ are then added to the F and M vectors.

6.7 Trim

Trim is implemented in `FGTrim` by a bracketed root-finder applied independently to each axis-control pair, iterated in an outer loop until all axes converge. It is **not** Newton-Raphson — JSBSim uses a linear-interpolation secant variant with a 0.9 relaxation factor to suppress oscillation. The modes are:

Mode	Constraints	Controls
tLongitudinal	w̄■=0, ū■=0, q̄■=0	α, throttle, elevator
tFull	longitudinal + v̇=0, ■=0, ■=0 + ψ track	+ ϕ, aileron, rudder, β
tGround	w̄■=0, q̄■=0, ■=0	altitude, θ, ϕ
tPullup	longitudinal at target load factor	α, throttle, elev
tTurn	coordinated turn at target bank angle	throttle, elevator, rudder
tCustom	user-defined (state, control) pairs	any

6.8 Atmospheric model

`FGStandardAtmosphere` implements the 1976 U.S. Standard Atmosphere by piecewise linear lapse rates from 0 to 86 km. Properties are exposed under `atmosphere/`: `T-R`, `P-psf`, `rho-slugs_ft3`, `a-fps`. A custom atmosphere can be subclassed; the C-172 model offsets temperature to model density altitude effects.

6.9 Winds and turbulence

`FGWinds` supports:

- Steady wind in NED (set `atmosphere/wind-north-fps`, `wind-east-fps`, `wind-down-fps`).
- Linear wind shear with altitude.
- Gusts (1-cosine "discrete gust" model used in MIL-F-8785C).
- Dryden or von Kármán continuous turbulence with configurable intensity and scale lengths.
- Microburst model with three components and a vortex ring.

`FGAuxiliary` consumes the wind vector to compute the airmass-relative velocity used by aerodynamics.

6.10 Property tree

`FGPropertyManager` wraps SimGear's property tree. Every state variable, control input, FCS signal, aero coefficient and atmosphere quantity is exposed at a string path; values flow between models entirely through it. Aircraft XML `<property>` elements can *declare* a new property (creating it if needed); FCS components, aero functions and external interfaces all bind themselves to the tree.

```
# Inspect a property in the JSBSim Python module
import jsbsim
fdm = jsbsim.FGFDMExec(None)
fdm.load_model("c172p")
fdm.run_ic()
print(fdm["velocities/vt-fps"], fdm["aero/alpha-deg"])
```

Chapter 7: Building Your Own Aircraft, Step-by-Step

This is the practical playbook. We assume you have access to **geometry** (CAD or at least 3-view), **mass properties** (weight, CG, ideally an inertia estimate), **engine data** (power or thrust curves), and **aerodynamic data** (CFD or wind tunnel). The workflow below brings them together into a working JSBSim model in a couple of weeks of part-time effort.

7.1 Step 1: Lay out the directory

```
$JSBSIM_ROOT/aircraft/MyAcft/
    MyAcft.xml          # the main XML (this is what JSBSim loads)
    reset00.xml         # initial conditions for default launch
    Systems/            # (optional) external <system> files
    Engines/            # (optional) aircraft-specific engine files
$JSBSIM_ROOT/scripts/
    MyAcft_takeoff.xml  # a script that loads MyAcft + reset00
```

Engines and propellers can live in the project-wide `$JSBSIM_ROOT/engine/` directory if you want to reuse them across multiple airframes.

7.2 Step 2: Skeleton XML

```
<?xml version="1.0"?>
<fdm_config name="MyAcft" version="2.0" release="ALPHA"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:noNamespaceSchemaLocation="http://jsbsim.sourceforge.net/JSBSim.xsd">

    <fileheader>
        <author>You</author>
        <filecreationdate>2026-05-24</filecreationdate>
        <description>My new aircraft.</description>
    </fileheader>

    <metrics>            ... </metrics>
    <mass_balance>       ... </mass_balance>
    <ground_reactions>   ... </ground_reactions>
    <propulsion>         ... </propulsion>
    <flight_control name="FCS: MyAcft">
        <channel name="Pitch"> ... </channel>
        <channel name="Roll">  ... </channel>
        <channel name="Yaw">   ... </channel>
    </flight_control>
    <aerodynamics>       ... </aerodynamics>
    <output type="CSV" rate="60" name="MyAcft.csv">
        <rates>ON</rates>
        <velocities>ON</velocities>
        <position>ON</position>
    </output>
</fdm_config>
```

7.3 Step 3: Geometry, mass, and gear

Geometry: from your CAD, pull out the wing reference area *S*, wing span *b*, mean aerodynamic chord $\bar{c}$, tail areas and arms. Pick the AERORP at 25% MAC and locate it in your structural frame. Locate the CG (any acceptable balance envelope point — you can shift it later with pointmasses).

Mass: empty weight + crew/passengers/cargo as pointmasses. For inertia, if you have a CAD model the easiest path is to export it to a kinematics tool (SolidWorks, Onshape, Fusion 360) that gives you $I_{xx,yy,zz,xy,xz,yz}$ about the CG directly. Otherwise estimate from radii of gyration (Roskam vol. V tables 9.1-9.4).

Gear: locate each contact point in the structural frame. Spring/damper choice is part craft, part calculation: aim for static deflection of 2-4 inches under weight on wheels, and a damping ratio of about 0.3-0.5. A quick estimator:

$$k = W_{gear} / \Delta_{static} \qquad c = 2\,\zeta\,\sqrt{(k \cdot W_{gear} / g)}$$

7.4 Step 4: Propulsion

Pick an engine file that matches your type. The standard JSBSim engine library has good starting points: `eng_io320` (160 hp 4-cyl), `CFM56` (turbofan), `F100-PW-229` (afterburning turbofan), `Estes_E9` (model rocket), `DJI_E305` (drone electric). For a new engine, copy an existing file and edit displacement / max power / BSFC / RPM range. Pair the engine with a thruster — `direct` for jets/rockets, `prop_*` for propellers.

```
<propulsion>
    <engine file="eng_io360">              <!-- 180 hp -->
        <feed>0</feed>
        <feed>1</feed>
        <thruster file="prop_77in">
            <location unit="IN"> <x>-30</x><y>0</y><z>2</z> </location>
            <orient unit="DEG"><pitch>2</pitch></orient>  <!-- thrust line up 2° -->
        </thruster>
    </engine>
    <tank type="FUEL" number="0">
        <location unit="IN"><x>0</x><y>-80</y><z>30</z></location>
        <capacity unit="LBS">300</capacity>
        <contents unit="LBS">280</contents>
    </tank>
    <tank type="FUEL" number="1">
        <location unit="IN"><x>0</x><y>80</y><z>30</z></location>
        <capacity unit="LBS">300</capacity>
        <contents unit="LBS">280</contents>
    </tank>
</propulsion>
```

7.5 Step 5: Flight control system

The smallest possible FCS just maps `fcs/elevator-cmd-norm` (–1..+1) to `fcs/elevator-pos-rad` via an `aerosurface_scale`. Real aircraft add trim sums, rate limits, deadbands, gust filters, autopilots — but start minimal, get the aircraft flying open-loop, then add complexity.

7.6 Step 6: Aerodynamics from CFD

This is the most labour-intensive step. The general procedure:

- **1. Define your sweep grid.** A reasonable subsonic grid is $\alpha \in \{-6, -4, -2, 0, 2, 4, 6, 8, 10, 12, 14, 16, 18\}$ deg, $\beta \in \{-15, -10, -5, 0, 5, 10, 15\}$ deg, Mach $\in \{0.15, 0.30, 0.50, 0.70\}$, control deflections at 5-7 points each. That is ~3000 CFD cases for a full lookup matrix; in practice you decouple.
- **2. Run steady RANS cases** in your CFD tool of choice (OpenFOAM, Fluent, STAR-CCM+, SU2). For each case extract F_x, F_y, F_z, M_x, M_y, M_z in the body frame about the AERORP and convert to coefficients by dividing by $\bar{q}\cdot S$, $\bar{q}\cdot S\cdot b$, $\bar{q}\cdot S\cdot \bar{c}$.
- **3. Decouple where possible.** Generally take C_L, C_D, C_m from the α-sweep at β=0; take C_Y, C_l, C_n from β-sweeps at α=0; take control derivatives as the slope of moment vs. deflection at the trim condition. For high-α or transonic models the coupling is real and you do need a full 2-D or 3-D table.
- **4. Run damping derivative cases** — either forced oscillation or quasi-steady rolling/yawing/pitching with constant rate. Extract C_{lp}, C_{mq}, C_{nr}, C_{lr}, C_{np}, $C_{m\_\dot{\alpha}}$.
- **5. Tabulate and validate.** Plot every coefficient as a function of every independent variable. Look for non-monotonic behaviour you didn't expect — usually a sign of mesh or convergence problems.
- **6. Convert to JSBSim XML.** The format is a simple matrix of numbers in `<tableData>`. The Python script in the next chapter automates this.

7.7 Step 7: Initial conditions and a smoke-test script

```
<!-- reset00.xml -->
<?xml version="1.0"?>
<initialize name="reset00">
    <ubody unit="FT/SEC">  150 </ubody>     <!-- ~100 KT -->
    <vbody unit="FT/SEC">   0  </vbody>
    <wbody unit="FT/SEC">   0  </wbody>
    <latitude unit="DEG">  37.6 </latitude>
    <longitude unit="DEG">-122.4</longitude>
    <altitude  unit="FT">  5000 </altitude>
    <phi unit="DEG">  0 </phi>
    <theta unit="DEG">2 </theta>
    <psi unit="DEG"> 90 </psi>               <!-- heading east -->
</initialize>
```

```
<!-- scripts/MyAcft_smoke.xml -->
<?xml version="1.0"?>
<runscript name="MyAcft smoke">
    <use aircraft="MyAcft" initialize="reset00"/>
    <run start="0" end="120" dt="0.00833333">
        <event name="Trim">
            <condition>simulation/sim-time-sec ge 0</condition>
            <set name="simulation/do_simple_trim" value="1"/>
            <notify/>
        </event>
        <event name="Pitch doublet">
            <condition>simulation/sim-time-sec ge 30</condition>
            <set name="fcs/elevator-cmd-norm" value="0.2" type="FG_VALUE"/>
            <notify/>
        </event>
        <event name="Pitch return">
            <condition>simulation/sim-time-sec ge 32</condition>
            <set name="fcs/elevator-cmd-norm" value="0"/>
            <notify/>
        </event>
    </run>
</runscript>
```

Run it with:

```
JSBSim --script=scripts/MyAcft_smoke.xml --logdirectivefile=...
```

7.8 Step 8: Iteration loop

- **Does it trim?** If `do_simple_trim` fails, check sign conventions (elevator producing the right direction of pitching moment), check that $Cm_{\alpha} < 0$ (statically stable in pitch), and check the α range of your tables covers the trim α.
- **Does it fly straight?** Static lateral-directional stability needs $Cn_{\beta} > 0$ and $Cl_{\beta} < 0$. If the aircraft rolls off, check the dihedral effect.
- **Does it respond like the real aircraft?** Compare phugoid, short-period, Dutch-roll, roll-mode and spiral-mode eigenvalues against published data or flight test.
- **Does it land?** Tune gear spring/damping until the bounce looks right. If the aircraft sinks through the runway, your spring is too soft or the contact point is too high (Z too positive in structural frame).

Chapter 8: CFD-to-JSBSim Workflow

This chapter describes the recommended workflow for generating JSBSim aerodynamic tables from CFD data, including the specific cases to run, the formulas to apply, and a reference Python script to convert results into JSBSim XML.

8.1 Choosing a CFD solver

- **OpenFOAM** (open source) — *simpleFoam* for steady RANS; *pimpleFoam* with overset for moving meshes. Free but mesh quality and turbulence model selection are on you.
- **SU2** (open source) — modern, adjoint capable, good for shape optimisation as well as analysis.
- **ANSYS Fluent / STAR-CCM+** — commercial, very mature. Worth the licence if you have it.
- **NASA OVERFLOW / FUN3D** — research-grade structured/unstructured RANS, available to U.S. universities.
- **XFLR5 / AVL** (vortex-lattice) — fast, surprisingly accurate for early-design subsonic estimates. Use it to bootstrap before spending CFD budget.

8.2 Recommended case matrix

For a conventional subsonic aircraft, a complete model needs the cases below. Each is a separate CFD run.

Group	Sweep	α (deg)	β (deg)	Output
Longitudinal static	α at clean config, β=0	–6 to +18 step 2	0	$C_L(\alpha)$, $C_D(\alpha)$, $C_m(\alpha)$
Longitudinal w/ flap	α × flap	–4 to +14 step 2	0	ΔC_L, ΔC_D, ΔC_m vs flap
Longitudinal w/ elevator	$\alpha \times \delta_e$	–4 to +14 step 2	0	$C_{m\_\delta e}(\alpha, \delta_e)$
Lateral static	β at α_{cruise}	α_{cr}	–15 to +15 step 3	$C_Y(\beta)$, $C_l(\beta)$, $C_n(\beta)$
Aileron	$\beta \times \delta_a$	α_{cr}	–10 to +10	$C_{l\_\delta a}$, $C_{n\_\delta a}$
Rudder	$\beta \times \delta_r$	α_{cr}	–10 to +10	$C_{Y\_\delta r}$, $C_{n\_\delta r}$
Compressibility	Mach	0	0	$\Delta C_D(M)$, $\Delta C_m(M)$
Ground effect	h/c̄	α_{cr}	0	$k_{CL,ge}(h/\bar{c})$, $k_{CD,ge}(h/\bar{c})$
Pitch damping	forced osc. about Y	α_{cr}	0	C_{mq}, $C_{m\_\dot{\alpha}}$
Roll damping	forced osc. about X	α_{cr}	0	C_{lp}, C_{Yp}, C_{np}
Yaw damping	forced osc. about Z	α_{cr}	0	C_{nr}, C_{Yr}, C_{lr}

If you have transonic or supersonic operations, add Mach as a third axis to all of the longitudinal cases. For a fighter aircraft or a UCAV operating to high α, double the α range and step size.

8.3 Forced-oscillation method for damping derivatives

The cleanest way to get C_{mq} et al. from CFD is to drive the geometry sinusoidally about the relevant axis with small amplitude and a known reduced frequency, then fit the in-phase and out-of-phase moment components. For pitch damping:

$$\theta(t) = \theta_0 + \Delta\theta \cdot \sin(\omega t)$$

$$q(t) = \Delta\theta \cdot \omega \cdot \cos(\omega t)$$

$$C_m(t) = C_{m,0} + C_{m\_\alpha} \cdot \Delta\alpha \cdot \sin(\omega t) + [C_{mq} + C_{m\_\dot{\alpha}}] \cdot (\bar{c}/2V) \cdot \Delta\theta \cdot \omega \cdot \cos(\omega t)$$

Fit by least squares: the cos-coefficient is $(C_{mq} + C_{m\_\dot{\alpha}})\cdot(\bar{c}/2V)\cdot\Delta\theta\cdot\omega$, the sin-coefficient is $C_{m\_\alpha}\cdot\Delta\alpha$. Separating C_{mq} from $C_{m\_\dot{\alpha}}$ requires an additional *plunge* oscillation (α changes without q changing). In practice many handbooks just publish the sum $(C_{mq} + C_{m\_\dot{\alpha}})$ and split it roughly 70/30.

Recommended amplitudes and reduced frequencies for a transport: $\Delta\theta = 1°$, $k = \omega\bar{c}/(2V) \in [0.01, 0.1]$. Five or ten cycles in CFD before extracting the fit.

8.4 Quasi-steady method (faster)

Run several steady CFD cases with a *constant* body-fixed angular rate applied as a rotating reference frame. Plot the resulting moment against pb/(2V) (or qc̄/(2V), or rb/(2V)); the slope is the damping derivative. This is much cheaper than URANS but only valid for small rates and incompressible flow. Adequate for a first-cut model.

8.5 Extracting and tabulating

Each CFD case gives you body-frame F and M about a known point. Convert to body-frame coefficients about the AERORP, then rotate force coefficients into wind-axis if your JSBSim model uses LIFT/DRAG/SIDE:

$$C_L = C_Z \cos\alpha - C_X \sin\alpha$$

$$C_D = -C_X \cos\alpha - C_Z \sin\alpha$$

$$C_Y \text{ (unchanged from body)}$$

8.6 Python helper: CFD CSV → JSBSim XML

A minimal generator that turns a CSV with columns `alpha_deg, flap_deg, CL, CD, Cm` into a JSBSim LIFT/DRAG/PITCH function block:

```
import pandas as pd

df = pd.read_csv('cfd_results.csv')
alphas = sorted(df.alpha_deg.unique())
flaps  = sorted(df.flap_deg.unique())

def emit_table(name, indvar_row, indvar_col, rows, cols, values):
    out = ['<table>',
           f'  <independentVar lookup="row">{indvar_row}</independentVar>',
           f'  <independentVar lookup="column">{indvar_col}</independentVar>',
           '  <tableData>']
    header = '          ' + '  '.join(f'{c:8.3f}' for c in cols)
    out.append(header)
    for r in rows:
        row = [f'{r:8.4f}'] + [f'{values[(r,c)]:8.4f}' for c in cols]
        out.append('  ' + '  '.join(row))
    out += ['  </tableData>', '</table>']
    return '\n'.join(out)

CL = {(r,c): df[(df.alpha_deg==r)&(df.flap_deg==c)].CL.iloc[0]
      for r in alphas for c in flaps}

print('<function name="aero/coefficient/CL">')
print('  <product>')
print('    <property>aero/qbar-psf</property>')
print('    <property>metrics/Sw-sqft</property>')
alpha_rad = sorted([a*3.14159/180 for a in alphas])
print(emit_table('CL', 'aero/alpha-rad', 'fcs/flap-pos-deg',
                 alpha_rad, flaps, CL))
print('  </product>')
print('</function>')
```

8.7 Validation: from CFD numbers to flight feel

Three classes of validation, in order of increasing difficulty:

- **Coefficient plots** — overlay JSBSim coefficients against the CFD source data. Should match by construction; if not, you have a units bug.
- **Eigenvalue analysis** — linearise the JSBSim model at cruise with the bundled `python/JSBSim/utils/linearize` tool. Compare phugoid, short-period, Dutch-roll, roll, spiral roots against textbook estimates (Stevens-Lewis ch. 5) or published data for similar aircraft.
- **Manoeuvre matching** — run pitch doublets, aileron rolls, rudder kicks. Plot p, q, r, α, β. Compare against flight test or documented handling qualities (Cooper-Harper, MIL-F-8785C).

Chapter 9: Reference Appendices

9.1 Property tree cheat sheet

```
# Pilot inputs (writable from any external program / script)
fcs/elevator-cmd-norm        -1..+1, positive = nose-up command
fcs/aileron-cmd-norm         -1..+1, positive = roll right
fcs/rudder-cmd-norm          -1..+1, positive = yaw right
fcs/throttle-cmd-norm[n]     0..1 per engine
fcs/mixture-cmd-norm[n]      0..1 per piston engine
gear/gear-cmd-norm           0 or 1
fcs/flap-cmd-norm            0..1
fcs/speedbrake-cmd-norm      0..1

# Aero state
aero/alpha-rad / alpha-deg
aero/beta-rad  / beta-deg
aero/qbar-psf
velocities/vt-fps           true airspeed
velocities/vc-kts           calibrated airspeed
velocities/mach
velocities/p-rad_sec        body-frame angular rates
velocities/p-aero-rad_sec   aero-frame angular rates

# Position and attitude
position/lat-geod-deg, position/long-gc-deg
position/h-sl-ft            altitude above MSL
position/h-agl-ft           altitude above ground
attitude/phi-rad, theta-rad, psi-rad

# Forces and moments (body frame, lbf, ft-lbf)
forces/fbx-aero, fby-aero, fbz-aero
forces/fbx-prop, fby-prop, fbz-prop
moments/l-aero, m-aero, n-aero
moments/l-total, m-total, n-total
```

9.2 Common FCS components

Component	Purpose
summer	Sum inputs with optional bias and clipping.
pure_gain	y = k * x (k can be a property)
scheduled_gain	k = table(independent_var); y = k * x
aerosurface_scale	Map normalised input to physical deflection range.
kinematic	Discrete positions with traverse times — flap, gear.
lag_filter	First-order lag: ■ = (x – y) / τ
lead_lag_filter	Continuous (a■+a■s)/(b■+b■s)
washout_filter	High-pass: τs/(τs+1)
second_order_filter	Notch/low-pass with 4 numerator + 4 denominator coefficients
integrator	y = ∫ x dt, with optional reset trigger
deadband	Output zero within a band around input
switch	Conditional logic with test/default cases
fcs_function	Embed an arbitrary function expression as an FCS block
pid	PID(Kp,Ki,Kd) with trigger, optional clipto
sensor	Adds noise, bias, drift, lag, delay
actuator	Rate limit, lag, hysteresis, deadband, fail modes

9.3 Engine type cross-reference

Engine class	XML root tag	Typical thruster	Key inputs
Piston			throttle, mixture, magnetos
Turbojet/Turbofan		or	throttle, AB on/off
Turboprop			throttle, propeller pitch
Rocket			throttle (often 0/1)
Electric DC			throttle (PWM)
Brushless DC			throttle, current limit
Rotor		(integrated)	collective, cyclic, throttle

9.4 Source-code map

```
src/FGFDMExec.cpp / .h           main executive, model orchestration
src/initialization/FGInitialCondition.cpp
                                 IC parsing and application
src/initialization/FGTrim.cpp    trim algorithm
src/input_output/FGPropertyManager.cpp
                                 property tree bindings
src/input_output/FGInputType*.cpp
                                 telnet/socket/QtJSBSim input drivers
src/input_output/FGOutputType*.cpp
                                 CSV, FlightGear, socket output drivers
src/math/FGFunction.cpp          function-language evaluator
src/math/FGTable.cpp             N-D table interpolation
src/math/FGPropertyValue.cpp     property-bound value node
src/math/FGQuaternion.cpp        quaternion arithmetic
src/math/FGMatrix33.cpp          3x3 matrix
src/math/FGColumnVector3.cpp     3-vector
src/models/FGPropagate.cpp       state integration
src/models/FGAccelerations.cpp   Newton-Euler, ground LCP
src/models/FGAerodynamics.cpp    aero force/moment assembly
src/models/FGAuxiliary.cpp       alpha, beta, qbar, Mach
src/models/FGAtmosphere.cpp      standard atmosphere
src/models/atmosphere/FGWinds.cpp  winds, gusts, turbulence
src/models/FGInertial.cpp        gravity, WGS-84, planet rotation
src/models/FGMassBalance.cpp     mass/CG/inertia composition
src/models/FGPropulsion.cpp      engine container
src/models/propulsion/FGPiston.cpp        piston engine
src/models/propulsion/FGTurbine.cpp       turbojet/turbofan
src/models/propulsion/FGTurboProp.cpp     turboprop
src/models/propulsion/FGRocket.cpp        rocket
src/models/propulsion/FGElectric.cpp      electric
src/models/propulsion/FGBrushLessDCMotor.cpp
src/models/propulsion/FGPropeller.cpp     propeller
src/models/propulsion/FGNozzle.cpp        rocket nozzle
src/models/propulsion/FGRotor.cpp         helicopter rotor
src/models/propulsion/FGTank.cpp          fuel tank
src/models/FGGroundReactions.cpp ground reactions container
src/models/FGLGear.cpp           landing gear / contact point
src/models/flight_control/...    FCS components (summer, pid, ...)
src/models/FGFCS.cpp             flight control system orchestrator
```

9.5 Bibliography and references

• Berndt, J. S. *JSBSim Reference Manual* (online and PDF). https://jsbsim.sourceforge.net/documentation.html

• Stevens B. L., Lewis F. L. *Aircraft Control and Simulation*, 2nd ed., Wiley, 2003. — definitive reference for the equations of motion JSBSim implements.

• Etkin B., Reid L. D. *Dynamics of Flight: Stability and Control*, 3rd ed., Wiley, 1996. — classical stability derivative definitions.

• Roskam, J. *Airplane Design, Vols. I-VIII*, DARcorp. — engineering estimates for inertia, gear, control-surface geometry.

• USAF DATCOM, *USAF Stability and Control DATCOM*, USAF AFFDL-TR-79-3032. — handbook estimates for stability and damping derivatives; useful sanity check on CFD.

• Cooke J., Zyda M., Pratt D., McGhee R. *NPSNET: Flight Simulation Dynamic Modeling Using Quaternions*, 1994.

• Buss S. *Accurate and Efficient Simulation of Rigid Body Rotations*, UCSD, 1999.

• Catto E. *Iterative Dynamics with Temporal Coherence*, Crystal Dynamics, 2005.

• U.S. Standard Atmosphere, 1976, NASA-TM-X-74335.

• NASA-NESC. *2015 Flight Simulation Check Cases*. https://nescacademy.nasa.gov/flightsim/2015

• MIL-F-8785C, *Military Specification, Flying Qualities of Piloted Airplanes*.

• MIL-HDBK-516B, *Department of Defense Handbook: Airworthiness Certification Criteria*.

9.6 Closing thoughts

JSBSim's design rewards engineers who already think in terms of stability derivatives, frames and equations of motion: there is very little hidden magic. Most of what feels like rote configuration in the XML is actually a direct expression of the physics — a function multiplying a coefficient by $\bar{q}\cdot S\cdot\bar{c}\cdot\ldots$ is the textbook formula, not a JSBSim convention. The corollary is that when something goes wrong, the bug is almost always in the data, the units, or the signs — not in the simulator. Trust the framework, instrument it (the property tree makes instrumentation trivial), and iterate.

> *The model is wrong, but the simulator is right. — folklore*

Chapter 10: Visual Diagrams of the FDM Loop

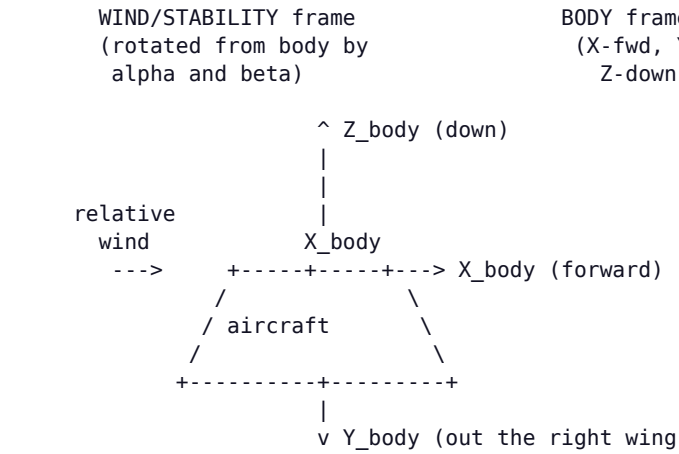
JSBSim has no built-in graphical documentation. The ASCII diagrams in this chapter are reverse-engineered from the source and faithfully reflect data flow as of v2.0.

10.1 Top-level simulation loop

10.2 Per-tick data flow between models

```
Propagate   --> (x, v, q)                                          ---+
    |                                                                 |
    v                                                                 |
Inertial    --> g, Omega_planet                                       |
    |                                                                 |
    v                                                                 |
Atmosphere --> T, p, rho, a                                           |
    |                                                                 |
    v                                                                 |
Winds       --> Vw_local, Tw2b                                        |
    |                                                                 |
    v                                                                 |
Systems(FCS) --> delta_e, delta_a, delta_r, throttle, etc.            |
    |                                                                 |
    v                                                                 |
MassBalance --> m, J, r_cg                                            |
    |                                                                 |
    v                                                                 |
Auxiliary  --> alpha, beta, qbar, Vt, Mach, n_load                    |
    |                                                                 |
    v                                                                 |
Propulsion --> F_thrust, M_thrust                                     |
    |                                                                 |
    v                                                                 |
Aerodynamics --> F_aero, M_aero                                       |
    |                                                                 |
    v                                                                 |
GroundReactions --> F_gear, M_gear (LCP)                              |
    |                                                                 |
    v                                                                 |
ExternalReactions / BuoyantForces --> F_ext, M_ext                    |
    |                                                                 |
    v                                                                 |
Aircraft    --> F_total, M_total                                      |
    |                                                                 |
    v                                                                 |
Accelerations --> v_dot, omega_dot    --(next tick)-------------------+
    |
    v
Output      --> CSV/socket/FlightGear
```

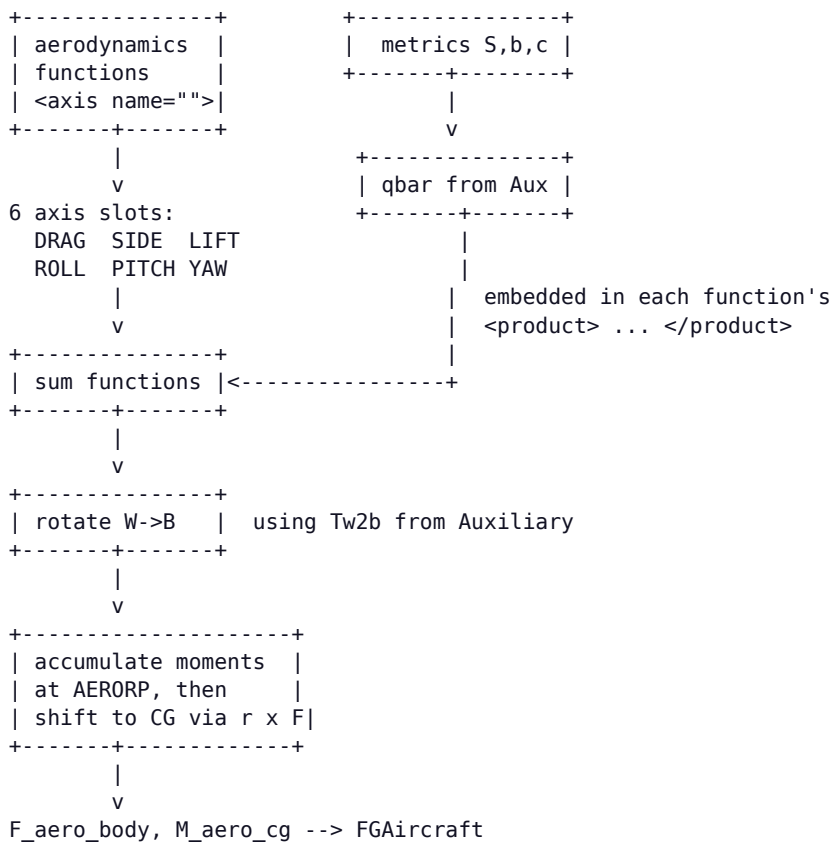
10.3 Body-frame, wind-frame and structural-frame sketch




```
alpha = angle between V_rel and the body X axis in the X-Z plane
beta  = angle between V_rel and the body X-Z plane

STRUCTURAL frame (used in <location> elements of the XML):
    X_struct = aft (typically),
    Y_struct = right,
    Z_struct = up.
JSBSim flips X and Z internally when converting to body frame.
```

10.4 Aerodynamic axis-to-force flow



Chapter 11: Propulsion — Per-Engine-Type Reference

11.1 Piston engines

`FGPiston` models a four-stroke piston engine with manifold-absolute-pressure dynamics, mixture and throttle controls, magneto failures and altitude derating. Configuration parameters:

```
<piston_engine name="NewEngine">
    <minmp     unit="INHG">  10.0 </minmp>      <!-- idle MAP -->
    <maxmp     unit="INHG">  28.5 </maxmp>      <!-- max MAP (SL) -->
    <displacement unit="IN3"> 320 </displacement>
    <maxhp>                   160 </maxhp>       <!-- SL max power -->
    <bsfc>                    0.45 </bsfc>       <!-- brake-spec fuel csmp -->
    <cycles>                  4    </cycles>     <!-- 4 or 2-stroke -->
    <idlerpm>                 600  </idlerpm>
    <maxrpm>                 2700  </maxrpm>
    <maxthrottle>             1.0  </maxthrottle>
    <minthrottle>             0.05 </minthrottle>
    <sparkfaildrop>           0.1  </sparkfaildrop>
    <numboostspeeds>          1    </numboostspeeds>
    <boostoverride>           0    </boostoverride>
    <ratedpower>            160    </ratedpower>
    <ratedrpm>             2700    </ratedrpm>
    <ratedaltitude unit="FT"> 0  </ratedaltitude>
</piston_engine>
```

Power output is modelled as a parabolic relation in MAP and a linear-with-throttle/RPM scaling. Mixture affects density altitude — lean of peak penalises power, rich of peak penalises fuel flow. The model accounts for ram-air recovery at high speed.

11.2 Turbine (turbojet/turbofan)

`FGTurbine` implements a two-spool engine with idle/military/afterburner regions and lookup-table thrust:

```
<turbine_engine name="F100">
    <milthrust unit="LBS">    17800 </milthrust>
    <maxthrust unit="LBS">    29100 </maxthrust>  <!-- AB rating -->
    <bypassratio>             0.35  </bypassratio>
    <tsfc>                    0.74  </tsfc>        <!-- cruise SFC -->
    <atsfc>                   2.05  </atsfc>       <!-- AB SFC -->
    <bleed>                   0.03  </bleed>
    <idlen1>                  30.0  </idlen1>
    <idlen2>                  60.0  </idlen2>
    <maxn1>                  100.0  </maxn1>
    <maxn2>                  100.0  </maxn2>
    <augmented>               1     </augmented>   <!-- has AB -->
    <augmethod>               1     </augmethod>
    <injected>                0     </injected>

    <function name="IdleThrust">
        <table>
            <independentVar lookup="row">velocities/mach</independentVar>
            <independentVar lookup="column">atmosphere/density-altitude</independentVar>
            <tableData>
                        0        20000     40000     60000
                0.0     0.0430   0.0488    0.0533    0.0593
                0.4     0.0356   0.0418    0.0466    0.0524
                0.8     0.0235   0.0312    0.0376    0.0445
                1.2     0.0070   0.0185    0.0272    0.0364
                1.6    -0.0094   0.0040    0.0156    0.0276
            </tableData>
        </table>
    </function>

    <function name="MilThrust">
```

```
        <table>...</table>      <!-- normalised thrust 0..1 -->
    </function>

    <function name="AugThrust">
        <table>...</table>      <!-- AB normalised thrust -->
    </function>
</turbine_engine>
```

Each thrust table is normalised to 1.0 at the SL static condition; the model multiplies by the relevant rating (milthrust or maxthrust). Spool-up is modelled with first-order lags; the property `propulsion/engine[n]/n2-norm` can be fed to FCS to gate flap deployment or thrust-reverser logic.

11.3 Turboprop

`FGTurboProp` couples a turbine core to a propeller. The XML adds a `maxpower` in shaft horsepower and a `betarangeend`:

```
<turboprop_engine name="PW100">
    <milthrust unit="LBS"> 0 </milthrust>
    <maxpower unit="HP">  1200 </maxpower>
    <idlen1>  50 </idlen1>
    <maxn1>  100 </maxn1>
    <betarangeend> 0.20 </betarangeend>  <!-- pitch override below this -->
    <reversemaxpower> 0.50 </reversemaxpower>
    <function name="EnginePowerVC">
        <table>...</table>
    </function>
    <function name="EnginePowerAltitude">
        <table>...</table>
    </function>
</turboprop_engine>
```

The thruster must be a `<propeller>` (separate XML file) and the propeller's pitch is controlled either by a governor (constant speed) or directly by FCS commands.

11.4 Rocket engines

`FGRocket` models a fixed-Isp rocket. Variable thrust is supported via throttle and a thrust-vs-time table; thrust vectoring is supported by a `<nozzle>` with steerable PYR angles. Suitable for boosters, sounding rockets, and missile models.

```
<rocket_engine name="SolidBooster">
    <isp>      265.0 </isp>             <!-- specific impulse, sec -->
    <builduptime> 0.2 </builduptime>     <!-- ignition transient -->
    <thrust_table>
        <table>                          <!-- F vs t in sec -->
            <independentVar>propulsion/engine[0]/thrust-time</independentVar>
            <tableData>
                0.0   1500
                0.5   3200
                3.0   3200
                4.0   1000
                4.5      0
            </tableData>
        </table>
    </thrust_table>
</rocket_engine>
```

11.5 Electric and brushless DC motors

```
<electric_engine name="E305">
    <power unit="WATTS"> 1000 </power>   <!-- max shaft power -->
</electric_engine>

<brushless_dc_motor name="OutrunnerBLDC">
    <maxvolts>     22.2  </maxvolts>     <!-- 6S LiPo -->
    <velocityconstant> 920 </velocityconstant>   <!-- Kv, rpm/V -->
    <coilresistance>   0.02 </coilresistance>    <!-- ohms -->
    <noloadcurrent>   1.0 </noloadcurrent>       <!-- A -->
</brushless_dc_motor>
```

Both models compute shaft torque from throttle (PWM) and a back-EMF curve. `FGBrushLessDCMotor` explicitly tracks current draw, which is useful for battery-life modelling — multiply current by terminal voltage for instantaneous power, then integrate against tank capacity to model battery state of charge.

11.6 Helicopter rotor

`FGRotor` models a helicopter main or tail rotor using an analytical blade-element approach with inflow modelling, flapping dynamics and ground-effect. It is complex — for a first pass start from the J3Cub piston piston or the X15 examples in `aircraft/` rather than from scratch.

11.7 Propellers

Propellers live in `engine/prop_*.xml` and reference C_T and C_P tables vs advance ratio J:

```
<propeller name="prop_75in2f">
    <ixx>         1.67 </ixx>
    <diameter unit="IN"> 75.0 </diameter>
    <numblades>      2 </numblades>
    <gearratio>    1.0 </gearratio>
    <cp_factor>    1.0 </cp_factor>
    <ct_factor>    1.0 </ct_factor>
    <minpitch>    10.0 </minpitch>   <!-- variable-pitch range -->
    <maxpitch>    35.0 </maxpitch>
    <minrpm>      640  </minrpm>     <!-- governor range -->
    <maxrpm>     2700  </maxrpm>
    <constspeed>   1   </constspeed> <!-- 1 = constant-speed prop -->
    <table name="C_THRUST" type="internal">
        <tableData>
             0.00  0.0863
             0.10  0.0837
             0.20  0.0792
             0.30  0.0727
             0.40  0.0643
             0.50  0.0540
        </tableData>
    </table>
    <table name="C_POWER" type="internal">
        <tableData> ... </tableData>
    </table>
</propeller>
```

Use `p_factor` on the thruster to model the slight nose-left yawing moment from prop disc asymmetry at high α. `sense` is +1 for clockwise (viewed from cockpit) and −1 for counter-clockwise; this matters for torque reaction direction and twin-engine "critical-engine" effects.

11.8 Fuel tanks

```
<tank type="FUEL" number="0" name="LeftWingTank">
    <location unit="IN">  <x>56</x><y>-112</y><z>59.4</z> </location>
    <drain_location unit="IN">
        <x>56</x><y>-112</y><z>50</z>           <!-- pickup point -->
    </drain_location>
    <type>AVGAS</type>                          <!-- or JET-A, RP-1, ... -->
    <capacity unit="LBS"> 185 </capacity>
    <contents unit="LBS"> 100 </contents>      <!-- initial fuel -->
    <density unit="LBS/GAL"> 6.0 </density>
    <temperature> 50 </temperature>             <!-- degF -->
    <standpipe unit="LBS"> 5 </standpipe>      <!-- unusable bottom -->
    <unusable-volume unit="GAL"> 0.5 </unusable-volume>
    <priority> 1 </priority>                    <!-- 1 = consume first -->
</tank>
```

Tanks contribute mass and inertia (parallel-axis-shifted from tank location to CG) that decrease as fuel is consumed. Multiple tanks with different priorities let you model the real fuel system schedule (e.g. "transfer from aux to main first").

Chapter 12: Flight Control Components — Reference

Every FCS component has the same shape: a name, one or more `<input>` properties, component-specific configuration, optional `<clipto>`, and an `<output>` property that other components downstream can read. The output is computed in the *order* in which components appear in a channel.

12.1 Summer / Pure gain / Aerosurface scale

```
<summer name="trim_sum">                <!-- y = sum(inputs) + bias -->
    <input>fcs/elevator-cmd-norm</input>
    <input>fcs/pitch-trim-cmd-norm</input>
    <bias>0.0</bias>
    <clipto> <min>-1</min><max>1</max> </clipto>
</summer>

<pure_gain name="elev_gain">             <!-- y = k * x -->
    <input>fcs/trim_sum</input>
    <gain>2.5</gain>                      <!-- can be a property -->
</pure_gain>

<scheduled_gain name="yaw_damper_gain">  <!-- k = table(prop) -->
    <input>velocities/r-aero-rad_sec</input>
    <table>
        <independentVar>velocities/mach</independentVar>
        <tableData>
            0.0   0.0
            0.3   1.0
            0.9   1.2
        </tableData>
    </table>
    <output>fcs/yaw-damper-cmd</output>
</scheduled_gain>

<aerosurface_scale name="elev_to_rad">
    <input>fcs/trim_sum</input>
    <gain>0.01745</gain>          <!-- multiply input first -->
    <domain> <min>-1</min><max>1</max> </domain>      <!-- input range -->
    <range>  <min>-28</min><max>23</max> </range>     <!-- output range -->
    <output>fcs/elevator-pos-rad</output>
</aerosurface_scale>
```

12.2 Filters

Filters use Tustin (bilinear) discretisation. Each has a characteristic constant `c1..c5` that maps to the textbook coefficients.

```
<lag_filter name="y">                    <!-- H = c1/(s+c1) -->
    <input>fcs/x</input>
    <c1>500</c1>                          <!-- time constant 1/500 s -->
</lag_filter>

<lead_lag_filter name="y">               <!-- H = (c1 s + c2)/(c3 s + c4) -->
    <input>fcs/x</input>
    <c1>0.5</c1> <c2>1</c2> <c3>0.05</c3> <c4>1</c4>
</lead_lag_filter>

<washout_filter name="y">                <!-- H = s/(s+c1) -->
    <input>velocities/q-rad_sec</input>
    <c1>1.0</c1>
</washout_filter>

<second_order_filter name="y">           <!-- H = (c1 s2 + c2 s + c3)/(c4 s2 + c5 s + c6) -->
    <input>fcs/x</input>
    <c1>0</c1><c2>0</c2><c3>4</c3>
```

```
    <c4>1</c4><c5>2.0</c5><c6>4</c6>      <!-- 2 Hz, zeta=0.5 -->
</second_order_filter>
```

12.3 Integrator

```
<integrator name="alpha_int">
    <input>aero/alpha-rad</input>
    <c1>1.0</c1>                              <!-- integration gain -->
    <trigger>fcs/integrator-trigger</trigger> <!-- 0=run, 1=hold, -1=reset -->
</integrator>
```

The trigger property lets you anti-windup with conventional logic: hold while saturated, reset on disengage.

12.4 PID controller

```
<pid name="AltitudeHold">
    <input>fcs/altitude-error-ft</input>
    <kp>0.05</kp>
    <ki type="ab3">0.001</ki>           <!-- ab2/ab3 = Adams-Bashforth -->
    <kd>0.10</kd>
    <trigger>ap/altitude-hold-engaged</trigger>
    <clipto> <min>-0.5</min><max>0.5</max> </clipto>
</pid>
```

12.5 Kinematic, deadband, switch

```
<kinematic name="flaps">                  <!-- discrete positions w/ traverse times -->
    <input>fcs/flap-cmd-norm</input>
    <traverse>
        <setting><position> 0</position><time>0</time></setting>
        <setting><position>10</position><time>2</time></setting>
        <setting><position>30</position><time>4</time></setting>
    </traverse>
    <output>fcs/flap-pos-deg</output>
</kinematic>

<deadband name="stick_db">                <!-- zero out small inputs -->
    <input>fcs/elevator-cmd-norm</input>
    <width>0.02</width>
</deadband>

<switch name="gear_switch">               <!-- conditional output -->
    <default value="0"/>
    <test logic="AND" value="1">
        gear/gear-cmd-norm ge 0.5
        velocities/vc-kts le 250
    </test>
    <output>fcs/gear-extend</output>
</switch>
```

12.6 Sensor and actuator wrapping

Use `<sensor>` to simulate imperfect measurements (lag, bias, noise) and `<actuator>` to simulate imperfect surface actuation (rate limit, hysteresis, deadband, fail modes). Both are useful for fault-injection studies.

```
<sensor name="ias_sensor">
    <input>velocities/vc-kts</input>
    <lag>0.5</lag>                <!-- first-order lag tau -->
    <bias>2</bias>                <!-- additive offset -->
    <drift_rate>0.01</drift_rate> <!-- units/sec -->
    <gain>1.005</gain>
    <noise variation="PERCENT">0.5</noise>
    <quantization name="adc12bit">
        <bits>12</bits><min>0</min><max>500</max>
    </quantization>
    <delay>0.1</delay>            <!-- pure time delay -->
    <output>instr/ias-indicated</output>
```

```
</sensor>

<actuator name="elev_act">
    <input>fcs/elevator-cmd-rad</input>
    <lag>0.05</lag>
    <rate_limit sense="incr">0.6</rate_limit>
    <rate_limit sense="decr">0.6</rate_limit>
    <bias>0</bias>
    <deadband_width>0.0017</deadband_width>
    <hysteresis_width>0.0035</hysteresis_width>
    <fail_zero> fcs/elev-act-fail-zero </fail_zero>
    <fail_hardover> fcs/elev-act-fail-hard </fail_hardover>
    <fail_stuck> fcs/elev-act-fail-stuck </fail_stuck>
    <output>fcs/elevator-pos-rad</output>
</actuator>
```

Chapter 13: Atmosphere, Winds, and Earth

13.1 ISA 1976 standard atmosphere

`FGStandardAtmosphere` evaluates the 1976 US Standard Atmosphere — eight piecewise lapse-rate segments from sea level to 86 km geometric altitude. The model exposes:

```
atmosphere/T-R              static temperature, Rankine
atmosphere/T-sl-R           sea-level reference temperature
atmosphere/T-dev-R          local deviation from standard
atmosphere/P-psf            static pressure, lb/ft^2
atmosphere/P-sl-psf
atmosphere/rho-slugs_ft3    density
atmosphere/a-fps            speed of sound
atmosphere/density-altitude
atmosphere/pressure-altitude
```

Custom atmosphere. Inherit from `FGAtmosphere` in C++ and override `Calculate(double altitudeASL)`. For most needs the standard model plus a temperature offset (`atmosphere/delta-T`) is enough — shifting T while keeping P constant gives a density-altitude shift, which is how density-altitude effects (hot-and-high) are usually modelled.

13.2 Winds and shear

```
atmosphere/wind-north-fps          steady wind from FGWinds
atmosphere/wind-east-fps           (NED frame, points TO direction wind blows toward)
atmosphere/wind-down-fps
atmosphere/wind-mag-fps            magnitude
atmosphere/psiw-rad                wind heading
atmosphere/total-wind-north-fps    steady + turbulence + gust + shear
atmosphere/turb-rate-rad_sec       Dryden/Karman turbulence angular component
```

Turbulence models: select via `atmosphere/turb-type`:

```
0 ttNone            no turbulence
1 ttStandard        legacy white-noise model
2 ttCulp            John Culp's model (used in FlightGear)
3 ttMilspec         MIL-F-8785C Dryden
4 ttTustin          MIL-F-8785C Tustin discrete-Dryden
```

For the MIL-spec models you set `atmosphere/turbulence/milspec/severity` on a 0-7 scale; the model auto-selects scale lengths and intensities according to the spec.

13.3 Gusts (discrete)

A discrete "1-cosine" gust can be triggered by setting:

```
atmosphere/cosine-gust-start  -> set to 1 to start
atmosphere/cosine-gust-frame  -> 1 BODY, 2 WIND, 3 LOCAL
atmosphere/cosine-gust-duration
atmosphere/cosine-gust-magnitude-ft_sec
atmosphere/cosine-gust-startup-duration
atmosphere/cosine-gust-steady-duration
atmosphere/cosine-gust-end-duration
atmosphere/cosine-gust-X-velocity   (and Y, Z in chosen frame)
```

13.4 Microburst

A three-component microburst model (Vicroy + Mulgund) is available via `atmosphere/turbulence-cosine-set`. Used for go-around and wind-shear-recovery training scenarios.

13.5 Inertial: gravity and Earth rotation

`FGInertial` publishes the gravity vector and the planet rotation rate. Two gravity models are available, selected via `simulation/gravity-model`:

```
0  Spherical inverse-square gravity (uniform mass distribution)
1  WGS-84 ellipsoid with J2 term (default)

simulation/gravitational-torque  -> 0/1 (default 0)
    Adds tidal torque on extended bodies; relevant for spacecraft.

Planet rotation: 7.2921151467e-5 rad/s about ECI Z (sidereal day).

WGS-84 ellipsoid:
    semi-major axis a   = 6378137.0 m
    semi-minor axis b   = 6356752.3142 m
    flattening f        = 1/298.257223563
    GM                  = 3.986004418e14 m^3/s^2
    J2                  = 1.08263e-3
```

Why this matters even for low-altitude flight. The Coriolis acceleration at Mach 0.8 cruise is on the order of 0.003 m/s² — small but non-zero. JSBSim correctly accounts for it, which is part of why it passes the NASA 2015 check cases. If you compare against simulators that use a flat-Earth approximation, expect small steady-state heading drifts when flying long distances east-west.

Chapter 14: Quaternion Attitude — A Primer for the Code

Reading `FGPropagate` and `FGQuaternion` is much easier if you already speak quaternions. This chapter is a quick refresher with JSBSim-specific notation.

14.1 Definition and conventions

A unit quaternion is a 4-tuple $q = (q_0, q_1, q_2, q_3)$ with norm 1. JSBSim follows the Hamilton ($i^2 = j^2 = k^2 = ijk = -1$) convention; `FGQuaternion` stores them in the order (w, x, y, z) — i.e. the scalar component first.

$$q = \cos(\theta/2) + \hat{n} \cdot \sin(\theta/2)$$

where $\hat{n}$ is the rotation axis and θ is the rotation angle. The quaternion that rotates a vector in the inertial frame into the body frame is $q_{i \to b}$ and the equivalent passive rotation is

$$v_{body} = q^* \cdot v_{inertial} \cdot q$$

(with vectors lifted to pure quaternions, and $\cdot$ denoting Hamilton product). This is exactly what `FGQuaternion::GetTransformationMatrix()` precomputes as a 3×3 rotation matrix to use for fast frame transforms.

14.2 Kinematic equation

$$\dot{q} = \tfrac{1}{2} \cdot q \cdot \omega_{b/i}$$

Here $\omega_{b/i}$ is the inertial angular rate expressed as a pure quaternion $(0, p_i, q_i, r_i)$. `FGQuaternion::GetQDot()` implements this in 12 floating-point operations.

```
FGQuaternion FGQuaternion::GetQDot(const FGColumnVector3& PQR) const {
    return FGQuaternion(
        -0.5*( data[1]*PQR(eP) + data[2]*PQR(eQ) + data[3]*PQR(eR)),
         0.5*( data[0]*PQR(eP) - data[3]*PQR(eQ) + data[2]*PQR(eR)),
         0.5*( data[3]*PQR(eP) + data[0]*PQR(eQ) - data[1]*PQR(eR)),
         0.5*(-data[2]*PQR(eP) + data[1]*PQR(eQ) + data[0]*PQR(eR))
    );
}
```

14.3 Renormalisation

Numerical integration of $\dot{q}$ with Euler or Adams-Bashforth schemes does not preserve the unit norm — over time the quaternion drifts off the unit 3-sphere, equivalent to introducing a spurious dilation. JSBSim renormalises every tick by dividing by the magnitude (`FGQuaternion::Normalize()`). The Buss integrators avoid this by integrating with the exponential map directly:

$$q(t+\Delta t) = q(t) \cdot \exp(\tfrac{1}{2} \cdot \Delta t \cdot \omega)$$

which is exact for constant ω and stays unit-norm to floating-point precision.

14.4 Euler angle extraction

`FGQuaternion::GetEulerDeg()` returns the conventional aircraft Euler angles (ϕ, θ, ψ) in degrees using a 3-2-1 (Z-Y-X intrinsic) sequence. The singularity at $\theta = \pm 90°$ is unavoidable — quaternions exist precisely to *avoid* the singularity in the integration, but they still have to project through it when you read out Euler angles. For highly aerobatic aircraft, do all your work in quaternion space or in DCM space and only extract Euler at the end.

Chapter 15: Complete Worked Example: A Simple Aircraft

This chapter walks every section of a minimal but flyable aircraft — an Acme A-1 light single — pulling values from first-principles geometry, a piston engine reused from the library, and CFD-derived aerodynamic coefficients. The complete XML is reproduced in pieces with commentary.

15.1 Header and reference data

```
<?xml version="1.0"?>
<fdm_config name="AcmeA1" version="2.0" release="ALPHA">

<fileheader>
    <author>Engineering Team</author>
    <filecreationdate>2026-05-24</filecreationdate>
    <description>Acme A-1, single-engine 2-seat sport aircraft.</description>
</fileheader>

<metrics>
    <wingarea unit="FT2">  140 </wingarea>      <!-- S -->
    <wingspan unit="FT">    32 </wingspan>      <!-- b -->
    <chord    unit="FT">   4.4 </chord>         <!-- cbar -->
    <htailarea unit="FT2">  18 </htailarea>
    <htailarm  unit="FT">   14 </htailarm>
    <vtailarea unit="FT2">  15 </vtailarea>
    <vtailarm  unit="FT">   14 </vtailarm>
    <location name="AERORP" unit="IN">
        <x>40</x><y>0</y><z>30</z>             <!-- 25% MAC -->
    </location>
    <location name="EYEPOINT" unit="IN">
        <x>30</x><y>-10</y><z>50</z>
    </location>
    <location name="VRP" unit="IN">
        <x>0</x><y>0</y><z>0</z>
    </location>
</metrics>
```

15.2 Mass, balance and inertia

```
<mass_balance>
    <!-- Inertias from CAD about the empty-weight CG, structural frame -->
    <ixx unit="SLUG*FT2">  800 </ixx>
    <iyy unit="SLUG*FT2"> 1100 </iyy>
    <izz unit="SLUG*FT2"> 1700 </izz>
    <ixz unit="SLUG*FT2">   0  </ixz>
    <emptywt unit="LBS"> 1250 </emptywt>
    <location name="CG" unit="IN">
        <x>39</x><y>0</y><z>30</z>             <!-- ~24% MAC -->
    </location>
    <pointmass name="Pilot">
        <weight unit="LBS">175</weight>
        <location unit="IN"><x>34</x><y>-12</y><z>32</z></location>
    </pointmass>
    <pointmass name="Passenger">
        <weight unit="LBS">175</weight>
        <location unit="IN"><x>34</x><y> 12</y><z>32</z></location>
    </pointmass>
    <pointmass name="Baggage">
        <weight unit="LBS"> 60</weight>
        <location unit="IN"><x>70</x><y>0</y><z>30</z></location>
    </pointmass>
</mass_balance>
```

15.3 Landing gear

```
<ground_reactions>
    <contact type="BOGEY" name="NOSE">
        <location unit="IN"><x>-10</x><y>0</y><z>-20</z></location>
        <static_friction>0.8</static_friction>
        <dynamic_friction>0.5</dynamic_friction>
        <rolling_friction>0.02</rolling_friction>
        <spring_coeff  unit="LBS/FT">    1500 </spring_coeff>
        <damping_coeff unit="LBS/FT/SEC"> 500 </damping_coeff>
        <max_steer unit="DEG">15</max_steer>
        <brake_group>NONE</brake_group>
    </contact>
    <contact type="BOGEY" name="LEFT_MAIN">
        <location unit="IN"><x>55</x><y>-40</y><z>-18</z></location>
        <static_friction>0.8</static_friction>
        <dynamic_friction>0.5</dynamic_friction>
        <rolling_friction>0.02</rolling_friction>
        <spring_coeff  unit="LBS/FT">    4500 </spring_coeff>
        <damping_coeff unit="LBS/FT/SEC">1300 </damping_coeff>
        <brake_group>LEFT</brake_group>
    </contact>
    <contact type="BOGEY" name="RIGHT_MAIN">
        <location unit="IN"><x>55</x><y> 40</y><z>-18</z></location>
        <static_friction>0.8</static_friction>
        <dynamic_friction>0.5</dynamic_friction>
        <rolling_friction>0.02</rolling_friction>
        <spring_coeff  unit="LBS/FT">    4500 </spring_coeff>
        <damping_coeff unit="LBS/FT/SEC">1300 </damping_coeff>
        <brake_group>RIGHT</brake_group>
    </contact>
    <contact type="STRUCTURE" name="LEFT_TIP">
        <location unit="IN"><x>40</x><y>-192</y><z>30</z></location>
        <static_friction>0.2</static_friction>
        <dynamic_friction>0.2</dynamic_friction>
        <spring_coeff  unit="LBS/FT">    20000 </spring_coeff>
        <damping_coeff unit="LBS/FT/SEC"> 2000 </damping_coeff>
    </contact>
    <contact type="STRUCTURE" name="RIGHT_TIP">
        <location unit="IN"><x>40</x><y> 192</y><z>30</z></location>
        <static_friction>0.2</static_friction>
        <dynamic_friction>0.2</dynamic_friction>
        <spring_coeff  unit="LBS/FT">    20000 </spring_coeff>
        <damping_coeff unit="LBS/FT/SEC"> 2000 </damping_coeff>
    </contact>
</ground_reactions>
```

15.4 Propulsion

```
<propulsion>
    <engine file="eng_io360">          <!-- 180 hp 4-cyl -->
        <feed>0</feed>
        <feed>1</feed>
        <thruster file="prop_77in">
            <location unit="IN"><x>-30</x><y>0</y><z>30</z></location>
            <orient unit="DEG"><pitch>2</pitch></orient>
            <sense>1</sense>
            <p_factor>5</p_factor>
        </thruster>
    </engine>
    <tank type="FUEL" number="0">
        <location unit="IN"><x>40</x><y>-80</y><z>32</z></location>
        <capacity unit="LBS">160</capacity>
        <contents unit="LBS">120</contents>
        <type>AVGAS</type>
    </tank>
    <tank type="FUEL" number="1">
        <location unit="IN"><x>40</x><y> 80</y><z>32</z></location>
        <capacity unit="LBS">160</capacity>
        <contents unit="LBS">120</contents>
        <type>AVGAS</type>
    </tank>
</propulsion>
```

15.5 Flight control system

```
<flight_control name="FCS: AcmeA1">
    <channel name="Pitch">
        <summer name="pitch_sum">
            <input>fcs/elevator-cmd-norm</input>
            <input>fcs/pitch-trim-cmd-norm</input>
            <clipto><min>-1</min><max>1</max></clipto>
        </summer>
        <aerosurface_scale name="elevator">
            <input>fcs/pitch_sum</input>
            <range><min>-25</min><max>20</max></range>
            <gain>0.01745</gain>
            <output>fcs/elevator-pos-rad</output>
        </aerosurface_scale>
    </channel>

    <channel name="Roll">
        <summer name="roll_sum">
            <input>fcs/aileron-cmd-norm</input>
            <input>fcs/roll-trim-cmd-norm</input>
            <clipto><min>-1</min><max>1</max></clipto>
        </summer>
        <aerosurface_scale name="left_aileron">
            <input>fcs/roll_sum</input>
            <range><min>-20</min><max>15</max></range>
            <gain>0.01745</gain>
            <output>fcs/left-aileron-pos-rad</output>
        </aerosurface_scale>
        <aerosurface_scale name="right_aileron">
            <input>fcs/roll_sum</input>
            <range><min>-15</min><max>20</max></range>
            <gain>-0.01745</gain>
            <output>fcs/right-aileron-pos-rad</output>
        </aerosurface_scale>
    </channel>

    <channel name="Yaw">
        <summer name="yaw_sum">
            <input>fcs/rudder-cmd-norm</input>
            <input>fcs/yaw-trim-cmd-norm</input>
            <clipto><min>-1</min><max>1</max></clipto>
        </summer>
        <aerosurface_scale name="rudder">
            <input>fcs/yaw_sum</input>
            <range><min>-20</min><max>20</max></range>
            <gain>0.01745</gain>
            <output>fcs/rudder-pos-rad</output>
        </aerosurface_scale>
    </channel>

    <channel name="Flaps">
        <kinematic name="flaps">
            <input>fcs/flap-cmd-norm</input>
            <traverse>
                <setting><position> 0</position><time>0</time></setting>
                <setting><position>10</position><time>3</time></setting>
                <setting><position>20</position><time>2</time></setting>
                <setting><position>30</position><time>2</time></setting>
            </traverse>
            <output>fcs/flap-pos-deg</output>
        </kinematic>
    </channel>
</flight_control>
```

15.6 Aerodynamics (representative values from CFD)

```
<aerodynamics>
    <alphalimits unit="RAD">
        <min>-0.087</min><max>0.30</max>
    </alphalimits>
    <hysteresis_limits unit="RAD">
        <min>0.20</min><max>0.30</max>
    </hysteresis_limits>

    <!-- ============================ LIFT ============================ -->
    <axis name="LIFT">
        <function name="aero/coefficient/CLwbh">
            <description>Wing-body-tail lift vs alpha (clean)</description>
            <product>
                <property>aero/qbar-psf</property>
                <property>metrics/Sw-sqft</property>
                <table>
                    <independentVar>aero/alpha-rad</independentVar>
                    <tableData>
                       -0.087  -0.250
                       -0.044   0.100
                        0.000   0.300
                        0.087   0.760
                        0.175   1.230
                        0.262   1.500
                        0.300   1.200    <!-- post-stall -->
                    </tableData>
                </table>
            </product>
        </function>

        <function name="aero/coefficient/CLDf">
            <description>Lift increment due to flap</description>
            <product>
                <property>aero/qbar-psf</property>
                <property>metrics/Sw-sqft</property>
                <table>
                    <independentVar>fcs/flap-pos-deg</independentVar>
                    <tableData>
                         0   0.00
                        10   0.20
                        20   0.35
                        30   0.45
                    </tableData>
                </table>
            </product>
        </function>

        <function name="aero/coefficient/CLde">
            <description>Lift due to elevator</description>
            <product>
                <property>aero/qbar-psf</property>
                <property>metrics/Sw-sqft</property>
                <property>fcs/elevator-pos-rad</property>
                <value>0.43</value>
            </product>
        </function>
    </axis>

    <!-- ============================ DRAG ============================ -->
    <axis name="DRAG">
        <function name="aero/coefficient/CD0">
            <product>
                <property>aero/qbar-psf</property>
                <property>metrics/Sw-sqft</property>
                <value>0.025</value>
            </product>
        </function>
        <function name="aero/coefficient/CDi">
```

```
        <description>Induced drag, CL^2 / (pi * AR * e)</description>
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <quotient>
                <property>aero/cl-squared</property>
                <value>20.4</value>           <!-- pi * AR * e ~ 20 -->
            </quotient>
        </product>
    </function>
</axis>

<!-- ========================== PITCH ========================== -->
<axis name="PITCH">
    <function name="aero/coefficient/Cm0">
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/cbarw-ft</property>
            <value>-0.04</value>
        </product>
    </function>
    <function name="aero/coefficient/Cma">
        <description>Pitch moment slope (must be negative)</description>
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/cbarw-ft</property>
            <property>aero/alpha-rad</property>
            <value>-0.5</value>
        </product>
    </function>
    <function name="aero/coefficient/Cmde">
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/cbarw-ft</property>
            <property>fcs/elevator-pos-rad</property>
            <value>-1.10</value>
        </product>
    </function>
    <function name="aero/coefficient/Cmq">
        <description>Pitch damping</description>
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/cbarw-ft</property>
            <property>aero/ci2vel</property>
            <property>velocities/q-aero-rad_sec</property>
            <value>-12.0</value>
        </product>
    </function>
</axis>

<!-- ========================== SIDE ========================== -->
<axis name="SIDE">
    <function name="aero/coefficient/CYb">
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>aero/beta-rad</property>
            <value>-0.7</value>
        </product>
    </function>
</axis>

<!-- ========================== ROLL ========================== -->
<axis name="ROLL">
    <function name="aero/coefficient/Clb">
        <description>Dihedral effect (must be negative)</description>
        <product>
```

```
                <property>aero/qbar-psf</property>
                <property>metrics/Sw-sqft</property>
                <property>metrics/bw-ft</property>
                <property>aero/beta-rad</property>
                <value>-0.08</value>
            </product>
        </function>
        <function name="aero/coefficient/Clp">
            <description>Roll damping</description>
            <product>
                <property>aero/qbar-psf</property>
                <property>metrics/Sw-sqft</property>
                <property>metrics/bw-ft</property>
                <property>aero/bi2vel</property>
                <property>velocities/p-aero-rad_sec</property>
                <value>-0.46</value>
            </product>
        </function>
        <function name="aero/coefficient/Clda">
            <product>
                <property>aero/qbar-psf</property>
                <property>metrics/Sw-sqft</property>
                <property>metrics/bw-ft</property>
                <property>fcs/left-aileron-pos-rad</property>
                <value>0.22</value>
            </product>
        </function>
    </axis>

    <!-- ============================ YAW ============================== -->
    <axis name="YAW">
        <function name="aero/coefficient/Cnb">
            <description>Weathercock stability (must be positive)</description>
            <product>
                <property>aero/qbar-psf</property>
                <property>metrics/Sw-sqft</property>
                <property>metrics/bw-ft</property>
                <property>aero/beta-rad</property>
                <value>0.06</value>
            </product>
        </function>
        <function name="aero/coefficient/Cnr">
            <description>Yaw damping</description>
            <product>
                <property>aero/qbar-psf</property>
                <property>metrics/Sw-sqft</property>
                <property>metrics/bw-ft</property>
                <property>aero/bi2vel</property>
                <property>velocities/r-aero-rad_sec</property>
                <value>-0.10</value>
            </product>
        </function>
        <function name="aero/coefficient/Cndr">
            <product>
                <property>aero/qbar-psf</property>
                <property>metrics/Sw-sqft</property>
                <property>metrics/bw-ft</property>
                <property>fcs/rudder-pos-rad</property>
                <value>-0.05</value>
            </product>
        </function>
    </axis>
</aerodynamics>

</fdm_config>
```

Sanity checks for this model:

- $C_{L\alpha}$ ≈ 5.4/rad (slope of the lift table near α=0). Typical for a Hershey-bar wing at AR ≈ 7.3 is around 4.7/rad — our number is close enough.

• $C_{Lmax} \approx 1.5$. Reasonable for a clean wing.
• $C_{D0} = 0.025$. Around the right number for a fixed-gear single.
• Statically stable: $C_{m\alpha} = -0.5 < 0$, $C_{n\beta} = +0.06 > 0$, $C_{l\beta} = -0.08 < 0$.
• Elevator-derivative sign: $C_{m\delta e} = -1.1$. Trailing-edge down (positive δ_e) produces nose-down pitch — that matches our sign convention.

Chapter 16: Initial Conditions, Scripts, and Resets

16.1 Initial conditions (reset files)

The IC file sets every state variable to a self-consistent starting point for the simulation. The minimal set is position (3), attitude (3), velocity (3) — JSBSim then computes α, β, q̄, Mach internally. You can over-specify and let JSBSim resolve, but explicit is better.

```
<?xml version="1.0"?>
<initialize name="cruise_5000">
    <!-- Position -->
    <latitude  unit="DEG">  37.6 </latitude>
    <longitude unit="DEG">-122.4 </longitude>
    <altitude  unit="FT">   5000 </altitude>          <!-- above MSL -->

    <!-- Attitude (Euler 3-2-1) -->
    <phi   unit="DEG"> 0 </phi>                       <!-- bank -->
    <theta unit="DEG"> 2 </theta>                     <!-- pitch -->
    <psi   unit="DEG"> 90 </psi>                      <!-- heading -->

    <!-- Pick ONE velocity spec: -->
    <ubody unit="FT/SEC"> 150 </ubody>
    <vbody unit="FT/SEC">   0 </vbody>
    <wbody unit="FT/SEC">   5 </wbody>
    <!--  or: <vt unit="KTS">100</vt> with <alpha unit="DEG">2</alpha> -->
    <!--  or: <vc unit="KTS">95</vc> (calibrated airspeed) -->
    <!--  or: <mach>0.55</mach> -->
</initialize>
```

Other useful IC elements:

```
<gamma unit="DEG"> 0 </gamma>          flight path angle
<vnorth>0</vnorth><veast>0</veast><vdown>0</vdown>   NED wind-rel velocity
<roc   unit="FT/MIN"> 0 </roc>          rate of climb
<altitudeAGL unit="FT"> 0 </altitudeAGL> above ground level
<targetNlf> 1 </targetNlf>               target load factor for trim-pullup
```

16.2 Scripts (event-based scenarios)

A script is the closed-form description of a flight test scenario: a sequence of events with conditions and property actions. The Run loop ticks the model at the specified `dt` until it reaches `end`.

```
<?xml version="1.0"?>
<runscript name="Cruise climb">
    <description>5000 ft cruise -> trim -> climb to 8000</description>
    <use aircraft="AcmeA1" initialize="cruise_5000"/>

    <run start="0" end="300" dt="0.00833333">
        <event name="Trim">
            <condition>simulation/sim-time-sec ge 0</condition>
            <set name="simulation/do_simple_trim" value="1"/>
            <notify/>
        </event>

        <event name="Throttle up">
            <condition>simulation/sim-time-sec ge 30</condition>
            <set name="fcs/throttle-cmd-norm"
                 action="FG_EXP" tc="5.0" value="1.0"/>
            <notify/>
        </event>

        <event name="Climb to 8000">
            <condition>simulation/sim-time-sec ge 60</condition>
            <set name="fcs/elevator-cmd-norm"
                 action="FG_RAMP" tc="3.0" value="-0.1"/>
```

```
            <notify/>
        </event>

        <event name="Level off">
            <condition>position/h-sl-ft ge 8000</condition>
            <set name="fcs/elevator-cmd-norm" value="0"/>
            <notify>
                <property>position/h-sl-ft</property>
                <property>velocities/vc-kts</property>
            </notify>
        </event>
    </run>
</runscript>
```

Action types on `<set>`:

```
FG_VALUE   set property to value immediately (default)
FG_DELTA   add value to current property
FG_RAMP    linear ramp over tc seconds
FG_EXP     first-order exponential approach with tc seconds
```

Conditions: any boolean expression over properties, using `ge le gt lt eq nq and or not`. Conditions can be nested via `<condition logic="AND|OR">`.

Chapter 17: Validation Workflow

A model that compiles and trims is only the first step. The industry-standard validation steps below catch the vast majority of modelling bugs before they reach a customer.

17.1 Step 1: trim diagnostics

Run `simulation/do_simple_trim` = 1 (longitudinal) and check the resulting elevator and throttle positions are inside the actual control envelope. If trim α is near the top of the alpha-limits, the trim is unstalled by luck only; lower the cruise speed or check your $C_L(\alpha)$ curve.

17.2 Step 2: open-loop step responses

Run pitch doublets, aileron pulses and rudder kicks from a trimmed condition. Plot p, q, r, α, β, ϕ, θ, ψ:

- **Phugoid**: oscillatory at ~30-60 s period, lightly damped ($\zeta \approx 0.05$). Visible as long-period altitude and airspeed swings after a pitch perturbation.
- **Short period**: ~2-5 s period, well damped ($\zeta \approx 0.5$). Visible as a rapid pitch-rate oscillation after a doublet.
- **Roll mode**: first-order, time constant ~0.5-1.5 s. Visible as exponential decay of p after an aileron kick.
- **Dutch roll**: ~3-6 s period, $\zeta \approx 0.05$-0.3. Coupled yaw-roll after a rudder kick.
- **Spiral mode**: very slow (~30-300 s time constant), often slightly unstable for conventional aircraft.

17.3 Step 3: linear model extraction

JSBSim ships a Python utility (`python/JSBSim/utils/linearize.py`) that builds an A, B, C, D linear model at a trim point. Compute the eigenvalues of A and compare poles against the closed-form estimates of Etkin or Stevens-Lewis. The order of magnitude should agree; if a pole is in the wrong half-plane, you have a sign error in your aero coefficients.

17.4 Step 4: performance envelope check

Quantity	How to obtain in JSBSim	Compare against
V_{S0} stall speed (clean)	Trim at min α holding altitude, decrement throttle	POH or design target
V_{S1} stall speed (flaps)	Same but with flap pos = max	POH
V_{max}	Full throttle level flight, record speed	POH
Best rate of climb (V_y)	Climb at various speeds, find max ROC	POH chart
Service ceiling	Step-altitude trims, find h at ROC = 100 fpm	POH
Range	Cruise at best-economy, integrate fuel burn	POH

17.5 Step 5: handling-qualities check

If you care about pilot feel, evaluate against MIL-F-8785C / MIL-HDBK-1797 handling-qualities specifications. The most valuable single number is the *C* parameter* (pitch-rate weighted load factor) plotted on a Cooper-Harper boundary chart at the cruise condition. JSBSim outputs all the necessary properties to compute it offline.

17.6 Step 6: flight-test comparison

Where flight-test data exists, run the same manoeuvre in JSBSim with the same inputs and overlay. The disagreements you see are a recipe for the next iteration of CFD or coefficient tuning. Beware of "chasing the flight": tuning JSBSim to match one manoeuvre often degrades others. Always validate against a held-out manoeuvre set.

Chapter 18: Common Bugs and How to Diagnose Them

These are the failure modes most newcomers encounter, in rough order of frequency. The solution column points at the first thing to inspect.

Symptom	Likely cause	First check
Aircraft "falls through" the runway at IC	Gear Z coordinate has wrong sign or wrong unit	`<contact>` Z negative if gear is below the structural-frame origin
Aircraft is upside down or rotates 180° on startup	ψ vs heading conventions or negated_crossproduct_inertia mismatch	Swap inertia sign attribute
Trim fails immediately	α range in CFD tables doesn't span trim α	Set `<output>` for α and run unwise — see what α it asks for
Trim converges but aircraft pitches down nose-first	$C_{m\alpha}$ sign error (model is statically unstable)	Plot Cm vs α; slope must be negative
Aircraft yaws sideways uncontrollably	$C_{n\beta}$ sign error	Slope must be positive
Slow but steady roll-off	Asymmetric pointmass or aileron not centred	Check pointmass Y locations sum to zero
Pitch oscillates at 0.5-2 Hz	Insufficient C_{mq} or wrong sign	Should be negative; magnitude 5-15 for GA
Aileron input has no roll response	$C_{l\delta a}$ wrong sign or aileron pos property not matching aero coefficient property	Trace fcs/left-aileron-pos-rad through to the function
Drag enormous at trim	Forgot to subtract reference drag, or CFD coefficients in body frame fed to LIFT/DRAG axes	Compute $D = C_D \cdot \bar{q} \cdot S$ by hand; compare to $F_{x,aero,body}$
Engine produces no thrust	Tank capacity = 0 or feed indices wrong	Check `propulsion/tank[n]/contents-lbs`
Simulation crashes with NaN	Divide-by-zero in a function (often /Vt at zero airspeed) or quaternion gone non-unit	Add `<clipto>` or guard with `<ifthen>`
Aircraft "sinks" through ground after touch-down	Spring constant too soft for the weight	$k \geq W / 0.5$ ft for normal damping rates
Bounces violently on landing	Damping coefficient too low	Aim for $\zeta \approx 0.4$-0.6 ($c = 2\zeta\sqrt{(km)}$)
FCS output stuck at clipto	Saturated; integrator wound up	Add an anti-windup trigger to the PID
FlightGear shows the aircraft jittering	Output rate mismatch or FG/FDM timestep diverge	Set output rate to 60 Hz, FG to 60 Hz too

Chapter 19: Python API

JSBSim ships a Python module (`jsbsim`) on PyPI and conda-forge that wraps the C++ library. It is the recommended way to drive JSBSim from machine-learning code, batch parameter studies, or unit tests.

19.1 Quick start

```
import jsbsim

fdm = jsbsim.FGFDMExec(root_dir=None)  # auto-detect aircraft path
fdm.set_debug_level(0)
fdm.load_model('c172p')
fdm.load_ic('reset00', useStoredPath=True)
fdm.run_ic()                           # apply initial conditions

# Trim longitudinally
fdm['simulation/do_simple_trim'] = 1

# Step the model 10 seconds at 120 Hz
for _ in range(1200):
    fdm.run()

print("alt:", fdm['position/h-sl-ft'])
print("V :", fdm['velocities/vt-fps'])
```

19.2 Reading and writing properties

```
# All properties accessible by string path
fdm['fcs/elevator-cmd-norm'] = -0.1     # nose-up command
fdm['fcs/throttle-cmd-norm'] = 0.8
alpha = fdm['aero/alpha-deg']
p, q, r = (fdm[f'velocities/{ax}-rad_sec'] for ax in 'pqr')

# Snapshot the full property tree (slow)
snapshot = fdm.query_property_catalog('')
```

19.3 Trim and linearisation

```
from jsbsim.utils.linearize import linearize

fdm.load_model('c172p')
fdm.load_ic('cruise', useStoredPath=True)
fdm.run_ic()
fdm['simulation/do_simple_trim'] = 1

A, B, C, D, x_trim, u_trim = linearize(fdm)
import numpy as np
eigvals = np.linalg.eigvals(A)
print(sorted(eigvals, key=lambda e: e.real))
```

19.4 Batch parameter studies

```
def sweep_alpha(cg_x_in):
    fdm = jsbsim.FGFDMExec(None)
    fdm.load_model('AcmeA1')
    fdm.load_ic('cruise_5000', useStoredPath=True)
    fdm['inertia/pointmass-location-X-inches[0]'] = cg_x_in
    fdm.run_ic()
    fdm['simulation/do_simple_trim'] = 1
    return fdm['aero/alpha-deg'], fdm['fcs/elevator-pos-deg']

for cg in range(30, 50, 2):
    a, e = sweep_alpha(cg)
    print(f'CG={cg} in   alpha={a:.2f}  elev={e:.2f}')
```

Chapter 20: Extended Property Reference

The complete property tree is published by JSBSim itself with `fdm.query_property_catalog('')`. The most useful properties for instrumentation and machine learning are grouped here.

20.1 Time

```
simulation/sim-time-sec                 simulation time since reset
simulation/frame                        tick counter
simulation/dt                           timestep size (s)
simulation/integrator/rate/rotational
simulation/integrator/rate/translational
simulation/integrator/position/rotational
simulation/integrator/position/translational
```

20.2 Atmosphere and winds

```
atmosphere/T-R, T-sl-R, delta-T
atmosphere/P-psf, P-sl-psf
atmosphere/rho-slugs_ft3
atmosphere/a-fps                        speed of sound
atmosphere/density-altitude
atmosphere/pressure-altitude
atmosphere/wind-{north,east,down}-fps   steady wind in NED
atmosphere/total-wind-{north,east,down}-fps wind+gust+turb
atmosphere/turb-type                    turbulence model (0..4)
atmosphere/turb-rate-rad_sec            current angular turbulence
```

20.3 Position

```
position/lat-geod-deg, lat-gc-rad       geodetic and geocentric lat
position/long-gc-deg
position/h-sl-ft, h-sl-meters           altitude above MSL
position/h-agl-ft                       altitude above ground
position/geod-alt-ft
position/distance-from-start-mag-mt     great-circle distance
position/terrain-elevation-asl-ft
```

20.4 Attitude

```
attitude/phi-rad, theta-rad, psi-rad    Euler
attitude/phi-deg, theta-deg, psi-deg    degrees
attitude/heading-true-rad
attitude/pitch-rad, roll-rad
/sim/attitude/q[0..3]                   quaternion components (if exposed)
```

20.5 Velocities

```
velocities/vt-fps                       true airspeed
velocities/vc-kts                       calibrated airspeed
velocities/ve-kts                       equivalent airspeed
velocities/vg-fps                       ground speed
velocities/mach
velocities/u-fps, v-fps, w-fps          body-frame velocity
velocities/u-aero-fps                   body-frame airmass-relative
velocities/p-rad_sec, q-rad_sec, r-rad_sec   body angular rates
velocities/p-aero-rad_sec               aero-frame angular rates
velocities/vdown-fps                    NED down velocity (=climb rate)
```

20.6 Aerodynamic state

```
aero/alpha-rad, alpha-deg
aero/beta-rad, beta-deg
aero/alphadot-rad_sec, betadot-rad_sec
aero/mag-alpha-rad, mag-beta-rad       |alpha|, |beta|
aero/qbar-psf, qbarUW-psf, qbarUV-psf
aero/h_b-mac-ft                        h/c for ground effect
aero/bi2vel, ci2vel
aero/cl-squared                        CL^2 from previous tick
aero/stall-hyst-norm                   stall hysteresis switch (0 or 1)
```

20.7 Forces and moments (body frame)

```
forces/fbx-aero, fby-aero, fbz-aero    aerodynamic
forces/fbx-prop, fby-prop, fbz-prop    propulsion
forces/fbx-gear, fby-gear, fbz-gear    ground reactions
forces/fbx-external, fby-external, fbz-external
forces/fbx-buoyant, fby-buoyant, fbz-buoyant
forces/fbx-total, fby-total, fbz-total sum (used by Accelerations)
moments/l-aero, m-aero, n-aero
moments/l-prop, m-prop, n-prop
moments/l-gear, m-gear, n-gear
moments/l-total, m-total, n-total
```

20.8 Accelerations

```
accelerations/udot-ft_sec2, vdot-ft_sec2, wdot-ft_sec2
accelerations/pdot-rad_sec2, qdot-rad_sec2, rdot-rad_sec2
accelerations/a-pilot-x-ft_sec2        at the EYEPOINT
accelerations/n-pilot-x-norm           load factor at EYEPOINT, g
accelerations/Nz                       normal load factor
```

20.9 FCS commands and positions

```
fcs/elevator-cmd-norm                  -1..+1 from pilot/script
fcs/aileron-cmd-norm
fcs/rudder-cmd-norm
fcs/pitch-trim-cmd-norm
fcs/roll-trim-cmd-norm
fcs/yaw-trim-cmd-norm
fcs/flap-cmd-norm
fcs/throttle-cmd-norm[n]
fcs/mixture-cmd-norm[n]
fcs/elevator-pos-rad, fcs/elevator-pos-deg, fcs/elevator-pos-norm
fcs/left-aileron-pos-rad, ...
fcs/rudder-pos-rad, ...
fcs/flap-pos-deg, fcs/flap-pos-norm
fcs/speedbrake-pos-rad
fcs/left-brake-cmd-norm, right-brake, center-brake
fcs/steer-cmd-norm
```

20.10 Propulsion

```
propulsion/engine[n]/thrust-lbs
propulsion/engine[n]/fuel-flow-rate-pps        per-second
propulsion/engine[n]/fuel-flow-rate-gph
propulsion/engine[n]/n1, n2                    turbine spool speeds
propulsion/engine[n]/rpm                       prop or piston rpm
propulsion/engine[n]/torque-lbsft
propulsion/engine[n]/power-hp                  piston
propulsion/engine[n]/map-inhg                  manifold pressure
propulsion/engine[n]/egt-degf                  exhaust temp
propulsion/engine[n]/cht-degf                  cylinder head
propulsion/engine[n]/oil-pressure-psi
propulsion/engine[n]/oil-temp-degf
propulsion/tank[n]/contents-lbs, capacity-lbs
propulsion/total-fuel-lbs
```

20.11 Simulation control

```
simulation/do_simple_trim     1 = longitudinal trim, 2 = ground, ...
simulation/do_trim            full trim mode
simulation/reset              reset to initial conditions
simulation/terminate          stop the script
simulation/pause              freeze the model
simulation/gravity-model      0 spherical, 1 WGS-84
simulation/gravitational-torque   spacecraft tidal torque on/off
simulation/notify-time-trigger
simulation/randomseed
```

Part II
Extended Theory and Background

The remainder of this manual moves from the practical "how-to" of Part I into a deeper, citation-driven treatment of the mathematics, physics, aerodynamics, geodesy, propulsion and verification machinery JSBSim implements. Read it linearly to build a complete mental model, or use it as a reference once you hit a topic in Part I that you want to understand in depth.

Chapter 21: Mathematics for Flight Dynamics

Every line of JSBSim is at root a manipulation of three-vectors, rotation matrices, quaternions, and the tensor that ties them all together — the inertia tensor. This chapter is the compact reference for those objects. We assume calculus and basic linear algebra; everything beyond is built up from scratch.

21.1 Vectors in three dimensions

A 3-vector is an ordered triple $\mathbf{v} = (v_x, v_y, v_z)$. Two operations are central:

$$\text{Dot product:} \quad \mathbf{a}\cdot\mathbf{b} = a_x b_x + a_y b_y + a_z b_z = |\mathbf{a}||\mathbf{b}|\cos\theta$$

$$\text{Cross product:} \quad (\mathbf{a}\times\mathbf{b})_i = \varepsilon_{ijk}\, a_j\, b_k; \;\; |\mathbf{a}\times\mathbf{b}| = |\mathbf{a}||\mathbf{b}|\sin\theta$$

The dot product is a scalar; the cross product is a vector perpendicular to both inputs. Both are written in JSBSim source as `FGColumnVector3` overloads.

Triple product. The scalar triple product $\mathbf{a}\cdot(\mathbf{b}\times\mathbf{c})$ equals the signed volume of the parallelepiped spanned by the three vectors. The vector triple product satisfies $\mathbf{a}\times(\mathbf{b}\times\mathbf{c}) = (\mathbf{a}\cdot\mathbf{c})\mathbf{b} - (\mathbf{a}\cdot\mathbf{b})\mathbf{c}$ — the 'BAC-CAB' identity used in rigid-body kinematics.

Projection. The component of $\mathbf{a}$ along the unit vector $\hat{n}$ is $\mathbf{a}\cdot\hat{n}$; the vector projection is $(\mathbf{a}\cdot\hat{n})\hat{n}$. This is used to extract drag components along the relative-wind direction, normal forces along the ellipsoid normal, etc.

21.2 Rotation matrices in SO(3)

A rotation matrix $\mathbf{R} \in$ SO(3) is a 3×3 real matrix with $\mathbf{R}^{\mathrm{T}}\mathbf{R} = \mathbf{I}$ (orthogonal) and $\det(\mathbf{R}) = +1$ (proper, no reflection). Two consequences:

- Inversion is trivial: $\mathbf{R}^{-1} = \mathbf{R}^{\mathrm{T}}$. JSBSim never computes a numerical inverse of a rotation matrix.
- Composition is matrix multiplication: a rotation by $\mathbf{R}_1$ followed by $\mathbf{R}_2$ is $\mathbf{R}_2\mathbf{R}_1$. Composition is non-commutative.

The three elementary rotations about the body axes are the building blocks of every aerospace Euler-angle convention:

$$R_x(\varphi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c\varphi & s\varphi \\ 0 & -s\varphi & c\varphi \end{pmatrix} \quad R_y(\theta) = \begin{pmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{pmatrix} \quad R_z(\psi) = \begin{pmatrix} c\psi & -s\psi & 0 \\ s\psi & c\psi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Note the sign convention: these matrices rotate *vectors* in the active sense (or equivalently transform *frames* in the passive sense — JSBSim uses the passive convention).

21.3 The 3-2-1 aerospace Euler sequence

Aircraft attitude is conventionally given by yaw ψ, then pitch θ, then roll ϕ, applied as intrinsic rotations about Z, then Y', then X''. The combined matrix transforming a vector from the local NED frame to the body frame is

$$R^{b}_{n} = R_x(\varphi)\, R_y(\theta)\, R_z(\psi)$$

Multiplied out element by element this gives the classical 9-entry direction cosine matrix found in every aerospace textbook (Stevens & Lewis Eq. 1.4-7). The 3-2-1 sequence has a singularity at $\theta = \pm 90°$ where ψ and ϕ are no longer separately determined — the well-known "gimbal lock."

21.4 The inertia tensor

For a rigid body with continuous mass distribution $\rho(\mathbf{r})$, the inertia tensor about a point is

$$I = \int\!\!\int\!\!\int \rho(r)\,(|r|^2 1 - r\otimes r)\, dV$$

where **1** is the 3×3 identity and ⊗ is the outer product. Element-wise, the diagonal terms (moments of inertia)

$$I_{xx} = \int (y^2+z^2)\, dm; \quad I_{yy} = \int (x^2+z^2)\, dm; \quad I_{zz} = \int (x^2+y^2)\, dm$$

and the off-diagonal terms (products of inertia)

$$I_{xy} = \int xy\, dm; \quad I_{xz} = \int xz\, dm; \quad I_{yz} = \int yz\, dm$$

Parallel axis theorem. If $\mathbf{I}_{cg}$ is the inertia about the centre of mass, the inertia about a point displaced by **d** from the CG is

$$\mathbf{I}_P = \mathbf{I}_{cg} + m\,(|\mathbf{d}|^2\,\mathbf{1} - \mathbf{d}\otimes\mathbf{d})$$

`FGMassBalance` applies this every tick to add each pointmass's inertia contribution to the empty-weight tensor.

Principal axes. Diagonalising **I** (which is real symmetric, hence orthogonally diagonalisable) gives three *principal moments of inertia* along three *principal axes*. For aircraft with x-z plane symmetry, $I_{xy} = I_{yz} = 0$ by construction; I_{xz} is non-zero whenever the upper and lower halves of the fuselage are mass-imbalanced (always).

21.5 Tensor transformation under rotation

Vectors transform as v' = **R**v. A second-order tensor like the inertia matrix transforms as

$$\mathbf{I}' = \mathbf{R}\,\mathbf{I}\,\mathbf{R}^{T}$$

This is how the structural-frame inertia listed in the XML is rotated to the body frame at load time, and how the body-frame inertia is rotated to the stability or wind frames when needed for stability-derivative analysis.

Chapter 22: Quaternions — Theory and Practice

Quaternions are the workhorse rotation representation inside `FGPropagate`. Reading JSBSim's integration loop is far easier if you have a working grip on the algebra.

22.1 Definition: H, the quaternion algebra

Hamilton's quaternions are the four-dimensional real algebra spanned by 1, i, j, k with multiplication rules

$$i^2 = j^2 = k^2 = ijk = -1, \quad ij = k, \quad jk = i, \quad ki = j$$

A quaternion is $q = q_0 + q_1 i + q_2 j + q_3 k$, often written $(q_0, \mathbf{q})$ with q_0 the scalar part and $\mathbf{q} = (q_1, q_2, q_3)$ the vector part.

22.2 The Hamilton product

$$p \cdot q = (p_0 q_0 - p \cdot q, \quad p_0 q + q_0 p + p \times q)$$

This non-commutative product is the heart of quaternion arithmetic. It is implemented in `FGQuaternion::operator*`. **Warning:** aerospace software is split between Hamilton convention (used by JSBSim, ROS, MATLAB Aerospace Toolbox) and JPL convention (used by JPL/NASA Goddard's spacecraft software, negating the sign of the cross-product term). Mixing them swaps left- and right-handed rotations — a frequent bug source.

22.3 Unit quaternions and rotations

A unit quaternion ($|q| = 1$) parameterises a rotation. The axis-angle correspondence is

$$q = (\cos(\theta/2), \hat{n} \sin(\theta/2))$$

for a rotation by angle θ about unit axis $\hat{n}$. The vector $\mathbf{v}$ rotates as the imaginary part of

$$v' = q \, v \, q^*$$

where v is the pure quaternion $(0, \mathbf{v})$ and $q^* = (q_0, -\mathbf{q})$ is the conjugate. Equivalently, the DCM corresponding to q is

$$R(q) = \begin{vmatrix} q_0^2+q_1^2-q_2^2-q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & q_0^2-q_1^2+q_2^2-q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & q_0^2-q_1^2-q_2^2+q_3^2 \end{vmatrix}$$

Internally `FGQuaternion::GetTransformationMatrix()` caches this 9-entry DCM so that subsequent frame transforms reduce to a matrix-vector multiply.

22.4 Kinematic equation: $\dot{q} = \frac{1}{2} q \otimes \omega$

If the body's angular velocity (in body frame) is ω, the kinematic ODE for the rotation from inertial to body is

$$\dot{q} = \tfrac{1}{2} \, q \otimes (0, \omega_{body})$$

Geometrically: at every instant the quaternion advances perpendicular to itself in 4-D, so $|q|$ is invariant under *exact* integration. Numerical schemes lose this invariance and need either periodic renormalisation or a structure-preserving integrator.

The Buss integrators implement structure-preserving discrete maps. For constant ω over $[t, t+\Delta t]$,

$$q(t+\Delta t) = q(t) \cdot \exp(\tfrac{1}{2} \, \Delta t \, \omega) = q(t) \cdot (\cos(|\omega|\Delta t/2), \hat{\omega} \sin(|\omega|\Delta t/2))$$

is exact and preserves unit norm to floating-point precision. Buss-2 augments this with a correction term from $\dot{\omega}$, giving second-order accuracy for non-constant ω.

22.5 Quaternions vs. Euler angles vs. DCMs

Representation	Parameters	Singularity	Cost	When
Euler 3-2-1	3 (ϕ, θ, ψ)	$\theta = \pm 90°$ (gimbal lock)	Cheap to display	Cockpit display, output
Quaternion	4 ($q_0..q_3$)	None	Compact, fast	Integration, storage
DCM	9 (orthogonality wastes 6)	None	Drift; need renorm	Frame transforms
Rotation vector	3 ($\theta\,\hat{n}$)	Singular at $\theta=2\pi$	Minimal	Linearised perturbations

JSBSim's design carries the quaternion as the primary attitude state (no singularity, four DOF), caches a DCM derived from it (cheap repeated frame transforms), and offers Euler outputs only at the property-tree boundary.

Chapter 23: Numerical Integration — Theory

The properties `simulation/integrator/rate/rotational`, `position/rotational`, `rate/translational` and `position/translational` choose four schemes independently. Knowing the underlying numerical theory is the difference between a stable model at *dt* = 1/120 s and a model that blows up when you change *dt*.

23.1 Single-step methods

For an initial-value problem ■ = f(t, y), $y(t_0) = y_0$:

$$\text{Forward Euler:}\quad y_{n+1} = y_n + h\, f(t_n, y_n)$$

Local truncation error O(h²), global error O(h). Conditionally stable: for the test equation ■ = λy, |1 + hλ| < 1 is required, which for real negative λ means h < 2/|λ|.

$$\text{Backward Euler:}\quad y_{n+1} = y_n + h\, f(t_{n+1}, y_{n+1})$$

Implicit, unconditionally A-stable. Requires a nonlinear solve each step. Rarely used in real-time aircraft sim because of cost.

$$\text{Trapezoidal (Heun, AM2):}\ y_{n+1} = y_n + (h/2)[f(t_n, y_n) + f(t_{n+1}, y_{n+1})]$$

Implicit second-order; predictor-corrector form is the usual explicit version. JSBSim's `eTrapezoidal` uses the predictor-corrector with the prior derivative as predictor — effectively trapezoidal on the previous derivative.

$$\text{RK4:}\quad k_1 = f(t_n, y_n),\quad k_2 = f(t_n+h/2, y_n+h\,k_1/2),\ \ldots\ \ y_{n+1} = y_n + h(k_1+2k_2+2k_3+k_4)/6$$

Classical Runge-Kutta 4 is the gold standard for simulator work where 1-step methods are preferred. Fourth-order accurate but four function evaluations per step — not what JSBSim uses by default.

23.2 Multi-step methods: Adams-Bashforth

JSBSim implements explicit Adams-Bashforth up to order 5. They use a single function evaluation per step but require startup values (filled in by lower-order methods or repeated Eulers).

```
AB2:  y_{n+1} = y_n + h*(  3/2 f_n  -  1/2 f_{n-1} )
AB3:  y_{n+1} = y_n + h*( 23/12 f_n - 16/12 f_{n-1} + 5/12 f_{n-2} )
AB4:  y_{n+1} = y_n + h*( 55/24 f_n - 59/24 f_{n-1} + 37/24 f_{n-2}
                        - 9/24 f_{n-3} )
AB5:  y_{n+1} = y_n + h*( 1901/720 f_n - 2774/720 f_{n-1}
                        + 2616/720 f_{n-2} - 1274/720 f_{n-3}
                        + 251/720 f_{n-4} )
```

JSBSim's defaults are AB2 for translational rate, AB3 for translational position, and rectangular Euler for the quaternion (later renormalised). For a smooth aircraft trajectory this gives energy and angular-momentum conservation to within ~10■■ over thousand-second runs at *dt* = 1/120 s.

23.3 Stability regions and step-size selection

Each integrator has a *region of absolute stability* in the complex λh plane. For aircraft modes with eigenvalues λ on the order of –2 to +0.05 rad/s (short period, phugoid, Dutch roll), $h < 2/|\lambda_{max}| \approx 0.6$ s suffices for stability. But aerodynamic forcing introduces fast modes (the bandwidth of FCS lag filters, gear-strut natural frequencies of 20-50 Hz), pushing the requirement to h ■ 1/120 s in practice.

Stiffness. When the system has eigenvalues spanning many decades (slow flight modes plus fast gear/actuator modes), explicit methods become inefficient — they must use the smallest time constant. Implicit methods would help; JSBSim instead decouples the stiff parts (LCP for gear, prefilters for actuators) so the explicit integration of the slow flight modes stays stable.

Chapter 24: Newton-Euler Rigid Body Dynamics

Aircraft EOMs are a special case of rigid-body mechanics. This chapter derives them from scratch and shows how JSBSim implements each term.

24.1 Inertial form (Newton's laws)

$$m\,a_i = F, \qquad dH/dt = M$$

Linear momentum $p = m\mathbf{v}_i$ obeys ■ = **F**. Angular momentum about the centre of mass is **H** = **I**ω; its inertial-frame time derivative equals the applied moment.

24.2 Body-frame form via the transport theorem

If a vector **q** is expressed in a frame rotating with angular velocity ω, the relation between its inertial and frame-relative derivatives is

$$(dq/dt)_{inertial} = (dq/dt)_{body} + \omega \times q$$

Apply to v_{body} and to **H**:

$$m(\dot{v}_{body} + \omega \times v_{body}) = F_{body}$$

$$I\dot{\omega} + \omega \times (I\omega) = M_{body}$$

These are `FGAccelerations::CalculateUVWdot()` and `CalculatePQRdot()` in 12 lines of arithmetic each. The cross-coupling term ω × (**I**ω) is the source of gyroscopic effects, including the celebrated tennis-racket theorem.

24.3 Including planet rotation

The body frame on a rotating Earth is not strictly inertial. Writing the inertial acceleration in terms of body and ECEF-tangent contributions and applying the transport theorem twice yields

$$\dot{v}_{body} + (\omega_{b/i}) \times v_{body} = F/m - T_{i\to b}(\Omega_p \times r_i)\cdot\ldots$$

The full inertial form including Coriolis and centrifugal terms is integrated in the inertial frame; `FGPropagate` stores `vInertialPosition` and `vInertialVelocity` and transforms to body for output.

24.4 Aircraft-specific simplifications

- x-z plane symmetry: $I_{xy} = I_{yz} = 0$.
- Body-fixed coordinate axes coincide with principal axes only approximately; $I_{xz} \neq 0$ because the upper and lower fuselage halves are mass-asymmetric.
- Steady flight: $\dot{v}_{body} = 0$, $\dot{\omega} = 0$; F and M reduce to zero (the trim condition).
- Linearisation around a trim point yields the longitudinal (u, w, q, θ) and lateral-directional (v, p, r, ϕ, ψ) decoupled state-space systems used for stability analysis.

Chapter 25: Fluid Mechanics Foundations

All aerodynamic coefficients that JSBSim consumes come from physics ultimately rooted in the Navier-Stokes equations. This chapter is the bridge from the continuum equations to the engineering numbers.

25.1 Conservation laws

$$\text{Continuity:}\quad \partial\rho/\partial t + \nabla\cdot(\rho V) = 0$$

$$\text{Momentum:}\quad \rho(\partial V/\partial t + V\cdot\nabla V) = -\nabla p + \nabla\cdot\tau + \rho g$$

$$\text{Energy:}\quad \rho(\partial e/\partial t + V\cdot\nabla e) = -p\,\nabla\cdot V + \Phi + \nabla\cdot(k\nabla T) + Q$$

with the Newtonian viscous stress $\tau_{ij} = \mu(\partial u_i/\partial x_j + \partial u_j/\partial x_i) - (2/3)\mu\delta_{ij}\nabla\cdot\mathbf{V}$. Closed by an equation of state $p = \rho RT$ (calorically perfect gas).

25.2 Dimensionless numbers

Number	Definition	Physical meaning
Reynolds Re	$\rho VL/\mu = VL/\nu$	Inertial / viscous; controls boundary layer
Mach M	V/a, $a = \sqrt{(\gamma RT)}$	Compressibility; M<0.3 incompressible
Knudsen Kn	λ/L	Continuum validity; Kn<0.01 OK
Prandtl Pr	$\mu c_p/k$	Momentum vs thermal boundary thickness
Strouhal St	fL/V	Unsteady-to-convective time-scale ratio
Froude Fr	$V/\sqrt{(gL)}$	Inertial / gravitational (free-surface)

Typical aerospace ranges: Re for a UAV at low altitude is 10■-10■, a GA aircraft 10■-10■, a transport 10■-10■. Mach ranges from 0.05 (small UAV) to 2.5+ (fighter); transonic (0.8-1.2) is the most numerically difficult.

25.3 Boundary layers

Prandtl (1904) observed that for high Re, viscosity matters only in a thin layer of thickness δ next to the surface. The boundary layer equations are a reduced form of NS:

$$\partial u/\partial x + \partial v/\partial y = 0,\quad u\,\partial u/\partial x + v\,\partial u/\partial y = -(1/\rho)dp/dx + \nu\,\partial^2 u/\partial y^2,\quad \partial p/\partial y \approx 0$$

Pressure is impressed from the outer inviscid flow. The Blasius solution for a flat plate at zero pressure gradient gives the laminar growth law $\delta \approx 5.0\,x/\sqrt{Re_x}$, local skin friction $C_f \approx 0.664/\sqrt{Re_x}$.

Turbulent transition on a smooth flat plate at zero gradient occurs near $Re_x \approx 5\times10$■ but is highly sensitive to freestream turbulence, roughness, and pressure gradient. UAV-scale airfoils at Re < 5×10■ often have laminar separation bubbles that dominate the polar.

Turbulent profile: $u^+ = (1/\kappa)\ln y^+ + B$ in the log layer, with $\kappa \approx 0.41$ and $B \approx 5.0$. RANS CFD should resolve $y^+ < 1$ at the first cell for wall-resolved analysis.

25.4 Separation, stall, and post-stall

Flow separates when the wall shear vanishes under an adverse pressure gradient. On an airfoil this occurs at the stall angle α_{stall}, beyond which lift falls and drag rises. Three stall mechanisms (depending on airfoil thickness):

- **Trailing-edge stall** (thick airfoils >15% t/c): separation creeps upstream from the trailing edge; gentle lift break.
- **Leading-edge stall** (medium 9-15% t/c): a short laminar separation bubble bursts; sharp lift break.

• **Thin-airfoil stall** (<8% t/c): long bubble grows and reattaches further aft; lift curve flattens before breaking.

JSBSim's `<hysteresis_limits>` in `<aerodynamics>` implements the post-stall reattachment hysteresis explicitly.

Chapter 26: Lift Generation Theory

26.1 Bernoulli is not the answer (and equal-transit-time is wrong)

Bernoulli's equation along a streamline of incompressible inviscid steady flow is

$$p + \tfrac{1}{2}\rho V^2 + \rho g z = \text{const}$$

It is a **consequence** of momentum conservation, not a cause of lift. The popular "equal transit time" explanation — that air over the upper surface must traverse it in the same time as air below — is empirically wrong; smoke-line experiments show the upper-surface air arrives at the trailing edge much earlier. The real cause of pressure asymmetry is *circulation*, established by the Kutta condition.

26.2 Kutta-Joukowski: the right answer

$$L' = \rho_\infty V_\infty \Gamma$$

For 2-D inviscid incompressible flow around any closed body, lift per unit span equals density times freestream velocity times circulation $\Gamma = \oint \mathbf{V}\cdot d\mathbf{s}$. Drag in this framework is zero (d'Alembert's paradox); viscosity reintroduces drag and selects Γ uniquely via the Kutta condition (smooth flow off the sharp trailing edge).

26.3 Thin airfoil theory

For a thin airfoil, the lift-curve slope is the famous result

$$dC_l/d\alpha = 2\pi \ \text{(per radian)} \approx 0.110 \ \text{/deg}$$

and the aerodynamic centre (point of zero pitching moment with α) is at the quarter-chord. Camber shifts the zero-lift angle $\alpha_{L=0}$ but does not change the slope.

26.4 Finite wings: Prandtl's lifting line

A finite wing sheds trailing vorticity that induces a downwash, tilting the local lift vector backwards (induced drag) and reducing the effective angle of attack. Prandtl's lifting-line theory replaces the wing by a bound vortex of strength $\Gamma(y)$ and a sheet of trailing vortices. The classical results:

$$C_L = \pi \cdot AR \cdot A_1, \quad C_{D,i} = C_L^{\,2} / (\pi \cdot AR \cdot e)$$

with span efficiency $e \le 1$ ($e = 1$ for elliptic loading, the minimum-induced-drag distribution). Modern aircraft use winglets and span loading optimisation to approach $e \approx 0.9$-0.95.

The finite-wing lift-curve slope:

$$dC_L/d\alpha = a_0 / (1 + a_0/(\pi \cdot AR \cdot e))$$

with $a_0 \approx 2\pi$ the 2-D section slope. For AR=8, e=0.85 this gives $dC_L/d\alpha \approx 4.9$/rad — about 20% less than the 2π value an infinite wing would have.

26.5 Compressibility correction

$$C_{L,M} = C_{L,inc} / \sqrt{(1 - M_\infty^{\,2})} \ \text{(Prandtl-Glauert)}$$

Valid up to $M \approx 0.7$. Beyond, shock formation and the local transonic problem dominate; M_{crit} (where local Mach first reaches 1) typically occurs at $M_\infty \approx 0.7$-0.8 for transport airfoils.

Chapter 27: Drag — A Complete Breakdown

$$C_D = C_{D,f} + C_{D,p} + C_{D,i} + C_{D,int} + C_{D,w}$$

Five additive sources at subsonic speeds. Each is captured by a different XML function in JSBSim and each comes from a different physical mechanism.

27.1 Skin friction (parasitic)

Turbulent skin friction on a flat plate (ESDU/Schlichting):

$$C_f \approx 0.455 \,/\, (\log_{10} Re_L)^{2.58}$$

Modified by a form factor FF for curvature and an interference factor Q for component junctions. Summed over wetted area, $D_f = q \sum C_{f,i}\, FF_i\, Q_i\, S_{wet,i}$.

27.2 Form (pressure) drag

Result of boundary-layer thickening and mild separation that prevents the rear-body pressure from fully recovering its stagnation value. Together with skin friction, this is the *profile drag*. $C_{D,f} + C_{D,p}$ are combined into C_{D0} for the polar.

27.3 Induced drag (lift-induced)

$$C_{D,i} = C_L^{\,2} \,/\, (\pi \cdot AR \cdot e)$$

Increases quadratically with C_L. Span efficiency e is between 0.7 and 0.95 for conventional wings; Oswald's efficiency e_0 (which absorbs other $C_L^{\,2}$ effects) is used in the standard parabolic polar.

27.4 Interference drag

Junctions between components (wing-fuselage, pylon-wing) create additional vortical and pressure-induced drag. ESDU and Hoerner give empirical Q factors typically 1.0-1.3.

27.5 Wave drag

Above the critical Mach number M_{crit}, the flow accelerates locally to M=1 and a shock forms. Shock-boundary-layer interaction yields wave drag that grows rapidly past the drag-divergence Mach number M_{DD}:

$$\Delta C_{D,wave} \approx 20(M - M_{DD})^4$$

(Lock's fourth-power rule). Supersonic minimum wave drag is achieved by Sears-Haack body of revolution (area rule, Whitcomb).

27.6 The parabolic drag polar and L/D max

$$C_D = C_{D0} + k\, C_L^{\,2}, \quad k = 1/(\pi \cdot AR \cdot e)$$

Maximum L/D occurs at $C_L^{\,*} = \sqrt{(C_{D0}/k)}$, $C_D^{\,*} = 2\, C_{D0}$:

$$(L/D)_{max} = 1 \,/\, (2\, \sqrt{(k \cdot C_{D0})})$$

Typical values: sailplane 40-60, jet transport 17-22, small UAV 8-14, Wright Flyer 1903 ≈ 8.3 (Anderson).

Chapter 28: Stability & Control — Complete Derivative Table

28.1 Sign conventions and definitions

- **Longitudinal static stability**: $dC_m/d\alpha < 0$. Equivalent to the CG forward of the neutral point.
- **Directional ('weathercock') stability**: $dC_n/d\beta > 0$. Right sideslip yields nose-right moment, restoring nose to wind.
- **Lateral ('dihedral effect')**: $dC_l/d\beta < 0$. Right sideslip yields left rolling moment, raising the windward wing.

28.2 Complete derivative table

Force/Moment	α / $\dot{\alpha}$	Rate	Control
C_L	$C_{L\alpha}$, $C_{L\dot{\alpha}}$	C_{Lq}	$C_{L\delta e}$, $C_{L\delta f}$
C_D	$C_{D\alpha}$ ($\approx 2k \cdot C_L \cdot C_{L\alpha}$)	—	$C_{D\delta e}$, $C_{D\delta f}$, C_{Dgear}
C_Y	$C_{Y\beta}$	C_{Yp}, C_{Yr}	$C_{Y\delta a}$, $C_{Y\delta r}$
C_l (roll)	$C_{l\beta}$	C_{lp}, C_{lr}	$C_{l\delta a}$, $C_{l\delta r}$
C_m (pitch)	$C_{m\alpha}$, $C_{m\dot{\alpha}}$	C_{mq}	$C_{m\delta e}$
C_n (yaw)	$C_{n\beta}$	C_{np}, C_{nr}	$C_{n\delta r}$, $C_{n\delta a}$

Damping derivatives are non-dimensionalised by b/(2V) (lateral) or $\bar{c}/(2V)$ (longitudinal). The $\dot{\alpha}$ derivative captures the downwash lag between wing and tail. $C_{n\delta a}$ is the "adverse yaw" — a positive aileron command produces yaw away from the intended turn direction due to differential drag on the deflected ailerons.

28.3 Neutral point and static margin

$$x_{NP} = x_{AC,wing} - (\bar{q} \cdot S_h / \bar{q} \cdot S) \cdot (C_{L\alpha,h} / C_{L\alpha}) \cdot l_h / \bar{c}$$

Static margin: SM = $(x_{NP} - x_{CG}) / \bar{c}$. Civil transports SM ≈ 5-15%; fighters can run negative SM with active control law (relaxed static stability).

28.4 Maneuver point

The CG location at which $dC_m/dn_z = 0$ in a steady pull-up. Aft of x_{NP} by approximately $\rho S \bar{c} C_{mq} / (4 m)$, which moves x_{MP} a few percent of $\bar{c}$ behind x_{NP} for typical transports.

Chapter 29: Aircraft Eigenmodes & Linearisation

29.1 Linearisation around trim

Perturbations Δu, Δw, Δq, $\Delta\theta$ about trim with U_0, Θ_0:

```
m Δu̇  = X_u Δu + X_w Δw - m g cosθ_0 Δθ + ΔX_ctrl
m Δẇ  = Z_u Δu + Z_w Δw + (m U_0 + Z_q) Δq - m g sinθ_0 Δθ + ΔZ_ctrl
I_y Δq̇ = M_u Δu + M_w Δw + M_w_dot Δẇ + M_q Δq + ΔM_ctrl
Δθ̇   = Δq
```

The lateral-directional subsystem is $[\Delta v, \Delta p, \Delta r, \Delta\phi]$ driven by Y_v, Y_p, Y_r, L_v, L_p, L_r, N_v, N_p, N_r. To first order at symmetric trim the two subsystems decouple.

29.2 State-space form

$$\dot{x} = Ax + Bu, \quad y = Cx + Du$$

Eigenvalues of A give the modes; eigenvectors give the modal shape (which states participate in each mode). JSBSim's `simulation/do_linearization` extracts A, B, C, D at a trim point and writes them to log4cpp output. The Python module exposes the same via `jsbsim.utils.linearize`.

29.3 Longitudinal modes

Short period — fast pitch oscillation, primarily Δw and Δq motion, lightly affected by Δu. Heavily damped ($\zeta \approx 0.3$-0.7), ω_n typically 1-5 rad/s for transports, 5-15 rad/s for fighters.

$$\omega_{n,sp}^2 \approx Z_\alpha M_q / V_0 - M_\alpha$$

$$2\zeta_{sp}\omega_{n,sp} \approx -(M_q + M_{\dot{\alpha}} + Z_\alpha / V_0)$$

Phugoid — slow exchange of kinetic and potential energy: Δu and $\Delta\theta$ oscillate, Δw and Δq nearly zero. Lightly damped ($\zeta \approx 0.05$) and slow.

$$\omega_{n,ph} \approx \sqrt{2} \cdot g / V_0 \quad \text{(Lanchester)}$$

$$\zeta_{ph} \approx (1/\sqrt{2}) \cdot (C_D / C_L)$$

For an airliner at $V_0 = 250$ m/s, $\omega_{ph} \approx 0.055$ rad/s — period ≈ 115 s. L/D = 17 gives $\zeta_{ph} \approx 0.041$.

29.4 Lateral-directional modes

Roll mode — first-order, heavily damped exponential decay of roll rate:

$$\tau_{roll} \approx -I_x / (\bar{q} \cdot S \cdot b \cdot C_{lp} \cdot b/(2V))$$

Typical $\tau_{roll} = 0.3$-1.5 s. Felt by the pilot as the "rate response."

Spiral mode — first-order, very slow, often slightly unstable. Eigenvalue near zero. Determined by the ratio of dihedral effect to weathercock stability; stability requires

$$C_{l\beta} \cdot C_{nr} > C_{n\beta} \cdot C_{lr}$$

Dutch roll — coupled yaw-roll oscillation; the aircraft "wags its tail" while rocking its wings 90° out of phase. Moderately damped ($\zeta \approx 0.05$-0.3), $\omega_n \approx 0.5$-3 rad/s for transports.

$$\omega_{n,DR}^2 \approx (\bar{q} \cdot S \cdot b / I_z)\, C_{n\beta}$$

$$2\,\zeta_{DR}\,\omega_{n,DR} \approx -(\bar{q} \cdot S \cdot b^2 / (2V \cdot I_z))\, C_{nr}$$

Dutch roll requires a yaw damper on transport aircraft; the natural mode is typically too lightly damped for pilot comfort. JSBSim's FCS handles this with a scheduled-gain yaw-damper.

Chapter 30: Airfoil Aerodynamics

30.1 The NACA airfoil family

4-digit (NACA 2412): max camber 2% chord, position of max camber 0.4c, max thickness 12%.

5-digit (NACA 23012): design C_L = 0.3, position of max camber 0.15c, max thickness 12%.

6-digit (NACA 64_2-415, "laminar"): min-pressure position at 0.4c, half-width of low-drag CL bucket = 0.2, design C_L = 0.4, thickness 15%.

30.2 Typical performance

Airfoil	$C_{L,max}$	α_{stall}	$C_{d,min}$	$C_{m,ac}$
NACA 0012	1.45 (Re 3×10■)	14°	0.0060	0.000
NACA 2412	1.55	15°	0.0065	–0.045
NACA 23012	1.70	18°	0.0070	–0.014
NACA 65-415	1.50	16°	0.0045	–0.075
Selig S1223 (low-Re)	2.20 (Re 2×10■)	10°	0.0150	–0.27

30.3 Reynolds number effects on UAVs

Below Re ≈ 5×10■ laminar separation bubbles dominate, raising $C_{d,min}$ and lowering $C_{L,max}$. UAVs at Re ≈ 10■-3×10■ need specialised low-Re airfoils: Eppler 387, SD7037, Selig S1223. JSBSim aero tables for UAVs should be specific to the operational Re — do not extrapolate from wind-tunnel Re 10■ to flight Re 10■.

30.4 Critical Mach number

$$C_p^* = (2/\gamma M_\infty^2)[((1 + (\gamma-1)/2 \cdot M_\infty^2)/(1 + (\gamma-1)/2))^{\gamma/(\gamma-1)} - 1]$$

M_{crit} is solved simultaneously with the Prandtl-Glauert Cp = Cp,min,inc / $\sqrt{(1 - M_{crit}^2)}$. Drag divergence $M_{DD} > M_{crit}$, defined where $dC_D/dM = 0.1$.

Supercritical airfoils (Whitcomb's NASA SC(2) series) raise M_{DD} by 0.05-0.10 at the same thickness, or allow 30-60% greater thickness at the same M_{DD} — enabling structurally lighter wings.

Chapter 31: CFD Methods — Theory and Practice

31.1 Panel methods

For inviscid incompressible flow (or P-G-corrected) over arbitrary shapes, Hess-Smith distributes sources and doublets on body panels and enforces flow tangency. O(N²) in panel count. Doublet-lattice (Albano-Rodden 1969) extends to unsteady oscillatory flow and is the workhorse of flutter analysis.

31.2 Vortex lattice

Wings represented by horseshoe vortices on a flat camber surface, trailing legs aligned with freestream. Computes C_L, induced drag, span-loading. Mark Drela's AVL is the canonical open-source tool for preliminary design and stability-derivative estimation. Limitations: no thickness, no viscous effects, no compressibility beyond P-G, no separation.

31.3 RANS turbulence models

- **Spalart-Allmaras (1992)**: one transport equation for modified eddy viscosity ν■. Widely used in external aerodynamics. Tolerates higher y^+ via near-wall linearisation.
- **k-ε**: two equations. Poor at adverse pressure gradients and separation; usually requires wall functions.
- **k-ω SST (Menter, 1994)**: blends k-ω near walls with k-ε in the far-field via F1 blending. Best general-purpose RANS model for separated/adverse-pressure flows. Industry default for high-lift, wings, rotorcraft.

Wall resolution: $y^+ < 1$ on first cell, 30-40 cells across boundary layer, wall-normal growth rate < 1.2.

31.4 LES, DES, hybrid

LES resolves eddies down to grid scale; $y^+ \approx 1$ in all three directions, Δx^+, $\Delta z^+ \sim 50$. Cost ~ $Re^{2.5}$ for wall-resolved LES — prohibitive at flight Re.

DES/DDES/IDDES (Spalart 1997): RANS in attached boundary layers, LES in separated regions. Affordable for massively separated flows (high α, post-stall, store separation).

31.5 Validation cases

AGARD AR-303 and AR-138, NASA Turbulence Modeling Resource (turbmodels.larc.nasa.gov), DLR-F6/F11 high-lift, NASA Common Research Model (CRM), HiLiftPW workshops, AIAA Drag Prediction Workshops 1-7. CFL3D, FUN3D and OVERFLOW are the reference codes.

Chapter 32: Forced Oscillation and Damping Derivatives

32.1 Setup

Impose a sinusoidal motion at frequency ω. For pitch:

$$\alpha(t) = \alpha_0 + \Delta\alpha \sin(\omega t), \quad q(t) = \omega \Delta\alpha \cos(\omega t)$$

Time-accurate CFD runs ~5 cycles to dissipate startup transients; data from cycles 3-5. Best practice: ≥100 time steps per cycle, dual time-stepping with 30-50 inner iterations.

32.2 Reduced frequency

$$k = \omega L_{ref} / (2 V_{\infty})$$

Ratio of unsteady to convective time scales. $k \to 0$ is quasi-steady (static derivatives only); $k > 0.05$ includes circulation lag and added-mass effects. Typical CFD forced oscillation: $k = 0.02$-0.10.

32.3 Derivative extraction

$$C_m(t) \approx C_{m0} + C_{m\alpha}\Delta\alpha + (C_{mq} + C_{m\dot{\alpha}})(\bar{c}/(2V))\Delta\alpha \cdot \omega \cdot \cos(\omega t)/\sin(\omega t) \cdot \ldots$$

Fourier integration over one cycle:

```
C_mα   ≈ (1/(π Δα)) ∫₀^(2π/ω) C_m(t) sin(ωt) dt
C_mq + C_mα̇ ≈ (1/(π k Δα)) ∫₀^(2π/ω) C_m(t) cos(ωt) dt
```

The pure damping C_{mq} and $\dot{\alpha}$-lag $C_{m\dot{\alpha}}$ cannot be separated from a single pitch-only oscillation; one needs a plunge (varying α at fixed q) or a combined schedule. Many handbooks publish the sum $(C_{mq} + C_{m\dot{\alpha}})$ and split it roughly 70/30.

Chapter 33: Geodesy and the WGS-84 Ellipsoid

JSBSim integrates its equations of motion in the inertial frame and transforms to the WGS-84 ellipsoid for output. This chapter is the reference for the geodetic constants and conventions.

33.1 WGS-84 defining constants

Symbol	Value	Description
a	6 378 137.0 m (exact)	Semi-major axis
1/f	298.257 223 563 (exact)	Reciprocal flattening
GM	3.986 004 418 × 10¹■ m³/s²	Earth gravitational param.
ω	7.292 115 × 10■■ rad/s	Earth rotation rate
b	6 356 752.3142 m	Semi-minor axis, b = a(1–f)
e^2	6.694 379 990 14 × 10■³	First eccentricity² = 2f – f²
e'^2	6.739 496 742 28 × 10■³	Second eccentricity² = e²/(1–e²)
J2	1.082 626 7 × 10■³	Second zonal harmonic (un-normalised)
R_V	6 371 000.79 m	Mean (volumetric) radius
γ_e	9.780 325 m/s²	Normal gravity at equator
γ_p	9.832 185 m/s²	Normal gravity at pole
g_0	9.806 65 m/s²	Standard gravity (definitional)
Sidereal day	86 164.0905 s	Length of sidereal day

33.2 Geodetic vs geocentric latitude

$$\tan(\varphi_c) = (1 - e^2)\tan(\varphi_g)$$

The two differ by up to 11.55 arc-min ≈ 0.19° near $\phi = 45°$ — equivalent to 21.4 km of north-south distance on Earth's surface. Aviation always uses geodetic latitude (GPS, charts, autopilots). JSBSim internal calculations sometimes use geocentric; output is in both.

33.3 Altitude definitions

- **Ellipsoidal (geodetic) height h** — signed distance to WGS-84 ellipsoid along the ellipsoid normal. Native GPS output.
- **Orthometric height H** — height above the geoid (mean sea level equipotential). Water flows from high H to low H. Charts use this.
- **Geoid undulation N** — h = H + N. Globally $N \in$ [–105 m, +85 m] versus WGS-84. EGM96/EGM2008 give global geoid models.
- **Pressure altitude** — altitude in ISA at which the measured pressure occurs. Altimeter with 29.92 inHg setting.
- **Density altitude** — same concept for density. Predicts aircraft and engine performance.
- **Geopotential altitude** — $H_{geopot} = R \cdot H_{geom}/(R + H_{geom})$. Used by the standard atmosphere model so that the hydrostatic equation has constant g.

33.4 Radii of curvature

$$M(\varphi) = a(1-e^2) / (1 - e^2 \sin^2\varphi)^{3/2} \quad \text{(meridian)}$$

$$N(\varphi) = a / \sqrt{(1 - e^2 \sin^2\varphi)} \quad \text{(prime vertical)}$$

At equator $M = a(1-e^2)$, $N = a$; at the pole $M = N = a/\sqrt{(1-e^2)} \approx 6\ 399\ 593.6$ m. Distance per radian: north-south M; east-west $N \cos \phi$. One arc-minute of latitude at the equator equals 1843 m; one nautical mile is exactly 1852 m by definition.

Chapter 34: Coordinate Transformations in Detail

34.1 Geodetic → ECEF (closed form)

```
x = (N + h) cos(φ) cos(λ)
y = (N + h) cos(φ) sin(λ)
z = (N (1 - e²) + h) sin(φ)
where N(φ) = a / sqrt(1 - e² sin²φ)
```

Three multiplications and a square root. JSBSim uses this every tick to map the integrated inertial position to a geodetic (lat, lon, alt) triple for output.

34.2 ECEF → Geodetic (Bowring iteration)

```
p = sqrt(x² + y²)
φ_0 = atan2(z, p (1 - e²))
repeat:
    N_i = a / sqrt(1 - e² sin² φ_i)
    h_i = p / cos(φ_i) - N_i
    φ_{i+1} = atan2(z, p (1 - e² N_i / (N_i + h_i)))
until |φ_{i+1} - φ_i| < 1e-12
λ = atan2(y, x)
```

Convergence in 3 iterations to 10■¹¹ rad ≈ 0.06 mm anywhere on Earth. Heikkinen (1982) and Olson (1996) give iteration-free closed-form alternatives; Vermeille (2011) is accurate to nanometres within 5000 km of the ellipsoid.

34.3 ECEF → ECI rotation

```
| x_ECI |   |  cos(θ)  -sin(θ)  0 |   | x_ECEF |
| y_ECI | = |  sin(θ)   cos(θ)  0 | * | y_ECEF |
| z_ECI |   |    0        0     1 |   | z_ECEF |
```

θ is the Greenwich Mean Sidereal Time (GMST). For precision work you augment with precession P, nutation N, polar motion W: GCRF = $P^{T}N^{T}R^{T}W^{T}$ · ITRF. For aircraft sim the bare rotation is enough.

34.4 ECEF → NED at (ϕ■, λ■)

```
                | -sin(φ₀)cos(λ₀)  -sin(φ₀)sin(λ₀)   cos(φ₀) |
R_NED^ECEF =    | -sin(λ₀)           cos(λ₀)            0     |
                | -cos(φ₀)cos(λ₀)  -cos(φ₀)sin(λ₀)  -sin(φ₀) |
```

ENU follows by swapping the first two rows and negating the third. JSBSim caches both NED-from-ECEF and ECEF-from-NED matrices each tick.

34.5 NED → Body (3-2-1 Tait-Bryan)

$$R^{b}_{n} = R_x(\varphi)\, R_y(\theta)\, R_z(\psi)$$

Multiplied out:

```
          | cθcψ                   cθsψ                   -sθ    |
R_b_n =   | sφsθcψ - cφsψ          sφsθsψ + cφcψ          sφcθ   |
          | cφsθcψ + sφsψ          cφsθsψ - sφcψ          cφcθ   |
```

Singularity at θ = ±90° is the canonical gimbal lock — which is why JSBSim integrates the quaternion form and only exports Euler angles.

Chapter 35: Earth Rotation and Gravity

35.1 Earth rotation rate and fictitious forces

The rotating ECEF frame is non-inertial. Newton's law acquires Coriolis and centrifugal terms:

$$a_{inertial} = a_{ECEF} + 2\,\Omega \times v + \Omega \times (\Omega \times r)$$

At Mach 0.8 cruise (~250 m/s) at mid-latitudes, Coriolis is ~0.036 m/s² (0.0037 g) — small but accumulates to substantial heading and track errors over flight-hour timescales if neglected. The centrifugal term at the equator is ~0.034 m/s² outward, which is precisely why measured surface gravity at the equator (9.780 m/s²) is less than the pure gravitational pull (9.814 m/s²). Centrifugal is conventionally folded into the gravity model so only Coriolis remains explicit in the EOMs.

35.2 Spherical gravity (simple)

$$g(r) = GM / r^2 \quad \text{(outward radial direction)}$$

At sea level r = 6 378 km, g ≈ 9.798 m/s². Adequate for many aircraft simulations; JSBSim default.

35.3 Somigliana normal gravity (WGS-84 surface)

$$\gamma(\varphi) = \gamma_e\,(1 + k\,\sin^2\varphi) / \sqrt{(1 - e^2\,\sin^2\varphi)}$$

with γ_e = 9.7803254 m/s² and $k = (b\,\gamma_p - a\,\gamma_e)/(a\,\gamma_e) \approx 0.001932$. Free-air correction for altitude:

$$\gamma(\varphi, h) \approx \gamma(\varphi)[1 - (2/a)(1 + f + m - 2f\,\sin^2\varphi)\,h + (3/a^2)\,h^2]$$

with $m = \omega_E^2\,a^2\,b / GM \approx 3.45 \times 10■^{3}$.

35.4 J2 gravity perturbation

The leading-order departure of the Earth's gravity field from a spherical mass:

$$a_{J2} = -(3\,J2\,GM\,a^2)/(2\,r^4) \times [(1-5\sin^2\varphi_c)\hat{x}, (1-5\sin^2\varphi_c)\hat{y}, (3-5\sin^2\varphi_c)\hat{z}]$$

For aircraft below 20 km altitude on flights under 24 hours, Somigliana-with-altitude suffices. For launch vehicles, missiles and orbital reentry, J2 (and often J3, J4) are required.

Chapter 36: Magnetic Field and Navigation

36.1 The World Magnetic Model (WMM 2025)

The WMM is the joint NCEI / British Geological Survey / NGA standard for the geomagnetic main field. The current model is WMM2025, valid through 31 December 2029. The field is a spherical-harmonic series (degree/order 12 standard, 133 for the high-resolution WMMHR2025), each Gauss coefficient varying linearly in time over the model epoch.

At a given (ϕ, λ, h, t) the model outputs the three geocentric components (X north, Y east, Z down), from which:

- Total intensity $F = \sqrt{(X^2 + Y^2 + Z^2)}$
- Horizontal intensity $H = \sqrt{(X^2 + Y^2)}$
- Magnetic declination $D = \text{atan2}(Y, X)$ — angle from true north to magnetic north, positive east
- Magnetic inclination (dip) $I = \text{atan2}(Z, H)$ — angle from horizontal

Convert true to magnetic heading: $\psi_{mag} = \psi_{true} - D$. Magnetic compasses also need a deviation card (airframe-specific) and exhibit northerly-turning errors due to dip.

36.2 GPS in WGS-84

The GPS constellation broadcasts ephemerides in WGS-84 ECEF and satellite times in GPST. A receiver solves the four-unknown system (3 ECEF coords + clock bias) from pseudorange equations

$$\rho_i = |r_{sat,i} - r_{rx}| + c \cdot \Delta t_{rx} + \varepsilon_i$$

via weighted least squares or an extended Kalman filter. The ECEF solution is then converted to (ϕ, λ, h) by the closed-form or iterative algorithms of the previous chapter. Modern INS-aided GNSS systems achieve sub-meter accuracy in dynamic operation.

36.3 Haversine great-circle distance

```
a = sin²((φ₂-φ₁)/2) + cos(φ₁) cos(φ₂) sin²((λ₂-λ₁)/2)
c = 2 atan2(√a, √(1-a))
d = R_E · c
```

with $R_E \approx 6\,371$ km. Initial bearing:

```
θ_i = atan2(sin(Δλ) cos(φ₂), cos(φ₁) sin(φ₂) - sin(φ₁) cos(φ₂) cos(Δλ))
```

Errors below 0.3% versus the oblate Earth. Vincenty's iterative method (or Karney's variant) gives geodesic distances to submillimetre on WGS-84.

Chapter 37: Time Systems in Aerospace

Five clocks matter. Getting them straight is the difference between a sim that ties cleanly to GPS, ephemerides, and magnetic-field models — and one that doesn't.

Scale	Definition	Use
TAI	International Atomic Time. Uniform SI seconds.	Foundation; no discontinuities.
UTC	= TAI – N leap seconds; |UT1–UTC|<0.9 s.	Civil time. UTC = TAI – 37 s (2026).
UT1	Earth-rotation time (mean sun on meridian → noon).	Drifts; broadcast as DUT1 = UT1–UTC.
GPST	Atomic; started at UTC on 1980-01-06.	GPS satellite time; = TAI – 19 s.
TT	= TAI + 32.184 s (definitional).	Solar-system ephemerides.
GMST	Greenwich Mean Sidereal Time.	ECEF↔ECI rotation; +3min 56.6 s/day vs UTC.

GMST formula (approximate, degrees):

```
GMST = 280.46061837 + 360.98564736629·d + 0.000387933·T² − T³/38710000
with d = days from J2000.0, T = centuries from J2000.0.
```

Chapter 38: The Standard Atmosphere in Depth

JSBSim's `FGStandardAtmosphere` implements the 1976 US Standard Atmosphere (NASA-TM-X-74335). ICAO Standard Atmosphere is identical to 32 km.

38.1 Sea-level reference

Quantity	Symbol	Value
Temperature	T_0	288.15 K (15°C)
Pressure	p_0	101 325 Pa
Density	ρ_0	1.225 kg/m³
Speed of sound	a_0	340.294 m/s
Dynamic viscosity	μ_0	1.7894 × 10■■ Pa·s
Specific gas const	R	287.058 J/(kg·K)
Ratio of specific heats	γ	1.40
g■	g_0	9.806 65 m/s²

38.2 Layer structure (0 to 86 km)

Layer	Base alt (km)	Top alt (km)	Lapse rate L (K/km)
Troposphere	0	11	–6.5
Tropopause	11	20	0
Stratosphere 1	20	32	+1.0
Stratosphere 2	32	47	+2.8
Stratopause	47	51	0
Mesosphere 1	51	71	–2.8
Mesosphere 2	71	84.852	–2.0

Within each non-isothermal layer, the hydrostatic equation

$$dp/dh = -\rho\, g, \quad p = \rho R T, \quad T(h) = T_b + L(h - h_b)$$

integrates to

$$p(h) = p_b \cdot [T_b/(T_b + L(h-h_b))]^{g_0/(R\cdot L)}$$

For isothermal layers (L = 0), the solution is exponential:

$$p(h) = p_b \cdot \exp[\, -g_0(h - h_b)/(R\, T_b)\,]$$

38.3 Derived quantities

$$a = \sqrt{(\gamma R T)} \quad \text{(speed of sound)}$$

$$M = V/a \quad \text{(Mach number)}$$

$$\mu(T) = \mu_{ref}(T/T_{ref})^{1.5}(T_{ref} + S)/(T + S)$$

(Sutherland's law, with S = 110.4 K for air). Reynolds number Re = ρVL/μ. Total/stagnation pressure and temperature:

$$p_t = p(1 + (\gamma-1)/2 \cdot M^2)^{\gamma/(\gamma-1)}$$

$$T_t = T(1 + (\gamma - 1)/2 \cdot M^2)$$

Chapter 39: Wind, Turbulence, and Gusts

39.1 Wind profile near the surface

$$u(z) = (u_*/\kappa)\,\ln(z/z_0) \quad \text{(log law, } \kappa \approx 0.41\text{)}$$

$$u(h) = u_{ref}\,(h/h_{ref})^{\alpha} \quad \text{(power law, } \alpha \approx 1/7 \text{ over open terrain)}$$

39.2 Discrete gusts (MIL-F-8785C 1-cosine)

$$V_{gust}(t) = (V_m/2)\,(1 - \cos(\pi t/T_m))$$

Used for handling-qualities certification. T_m ranges from 0.5 s (small UAV) to several seconds (transports); V_m 5-15 m/s typical.

39.3 Continuous turbulence — Dryden PSD

$$\Phi_u(\omega) = \sigma_u^2 \cdot (2L_u/V) \,/\, (1 + (L_u\omega/V)^2)$$

$$\Phi_v(\omega) = \sigma_v^2 \cdot (L_v/V) \cdot (1 + 3(L_v\omega/V)^2) \,/\, (1 + (L_v\omega/V)^2)^2$$

$$\Phi_w(\omega) = \sigma_w^2 \cdot (L_w/V) \cdot (1 + 3(L_w\omega/V)^2) \,/\, (1 + (L_w\omega/V)^2)^2$$

Scale lengths $L_{u,v,w}$ and intensities $\sigma_{u,v,w}$ are specified by altitude band in MIL-F-8785C / MIL-HDBK-1797. Light/moderate/severe intensity probabilities decrease with altitude; turbulence above the tropopause is rare.

Von Kármán PSD is an alternative (more accurate at high frequencies) commonly used in flutter analysis.

39.4 Microburst (Vicroy)

A microburst is modelled as a three-component flow field: horizontal outflow + downdraft + vortex ring. Vicroy's analytical model parameterises the radial outflow profile by core radius, altitude of the vortex ring, and outflow peak velocity. Used for wind-shear-recovery training scenarios.

Chapter 40: Propulsion Theory

40.1 Piston engines — Otto cycle

Four-stroke engine: intake, compression (adiabatic), combustion (constant volume), expansion (adiabatic), exhaust. Ideal Otto efficiency $\eta = 1 - (1/r)^{\gamma-1}$ with compression ratio r. BSFC typical 0.4-0.5 lb/(hp·hr) for normally aspirated avgas engines.

Performance scales with manifold absolute pressure (MAP), RPM, and mixture. `FGPiston` uses a parabolic power-vs-MAP relation and linear scaling with throttle/RPM. Altitude derating accounts for ambient density decrease (unless supercharged or turbocharged).

40.2 Turbine — Brayton cycle

Continuous-flow cycle: compression, combustion (≈constant pressure), expansion. Ideal Brayton efficiency $\eta = 1 - (1/\pi)^{(\gamma-1)/\gamma}$ with pressure ratio π. Modern turbofans $\pi \approx 30\text{-}50$, TSFC 0.30-0.50 lb/(lbf·hr) cruise, 1.5-2.5 with AB on.

Bypass ratio: pure jet ≈ 0; military low-bypass ≈ 0.3; commercial high-bypass ≈ 10-12 (GE9X, Trent XWB). High-bypass = more mass flow at lower velocity = quieter and more efficient at subsonic speeds.

Two-spool architecture: N1 (low-pressure spool, fan) and N2 (high-pressure spool, compressor + turbine). N1 is the primary thrust-rating parameter on most commercial engines.

40.3 Rocket — $F = \dot{m} V_e + (p_e - p_a)A_e$

$$I_{sp} = F / (\dot{m} \cdot g_0) \quad \text{(seconds)}$$

Specific impulse measures fuel efficiency. Typical values: solid propellant 250 s, RP-1/LOX 350 s, LH2/LOX 450 s, ion >3000 s. Tsiolkovsky rocket equation:

$$\Delta v = I_{sp}\, g_0 \ln(m_0/m_f)$$

Orbital insertion requires $\Delta v \approx 9\text{-}10$ km/s including gravity and drag losses. Multi-staging is necessary because m_0/m_f is exponential in Δv.

40.4 Electric motors — BLDC

Brushless DC motor obeys (per phase):

$$V = K_e \cdot \omega + I \cdot R, \quad \tau = K_t \cdot I, \quad K_t = 1/K_v \text{ in SI units}$$

with K_v in rpm/V (hobby convention). Power $P = V \cdot I = \omega \cdot \tau + I^2 \cdot R$; the second term is resistive loss. Efficiency $\eta = \omega \cdot \tau / (V \cdot I)$ peaks at intermediate loads (~50-70% rated current).

Battery: LiPo cells nominal 3.7 V (4.2 V max, 3.0 V min); C-rating limits peak discharge (e.g. 25 C × 5000 mAh = 125 A max). State-of-charge integrates current over time: $SoC(t) = SoC(0) - \int I/Q \cdot dt$.

Chapter 41: Propeller and Rotor Theory

41.1 Propeller — momentum + blade element

Momentum theory (Froude): an ideal disc accelerates the flow from V to V+2v_i, producing thrust

$$T = 2\,\rho\,A\,(V + v_i)\,v_i$$

with v_i the induced velocity at the disc. Power $P = T\,(V + v_i)$; efficiency $\eta = TV/P \to 1$ only at zero v_i (infinite disc area, the actuator-disc limit).

Blade-element theory analyses each blade element as a 2-D airfoil with local α and $V_{resultant}$. JSBSim's `FGPropeller` uses tabulated $C_T(J)$ and $C_P(J)$ curves where the advance ratio is

$$J = V / (n \cdot D), \quad n \text{ in rev/s, } D = \text{diameter}$$

$$\text{Thrust} = C_T(J) \cdot \rho \cdot n^2 \cdot D^4$$

$$\text{Power} = C_P(J) \cdot \rho \cdot n^3 \cdot D^5$$

$$\text{Efficiency } \eta_p = J \cdot C_T / C_P$$

41.2 P-factor and prop-induced effects

- **P-factor**: at non-zero α the descending blade has a higher local angle of attack than the ascending blade, producing a lateral thrust offset that yaws the aircraft.
- **Slipstream rotation**: the prop wash spirals, hitting the vertical tail asymmetrically and producing a yaw moment.
- **Gyroscopic precession**: a pitching aircraft applies an input torque about the lateral axis; spinning prop reacts with a precessional yaw, and vice versa.
- **Torque reaction**: Newton's third law — the engine torquing the prop one way torques the aircraft the other way. Sense (clockwise vs counter-clockwise from cockpit) determines which.

41.3 Constant-speed propellers

A governor adjusts blade pitch to hold a commanded RPM regardless of throttle setting. JSBSim's `<constspeed>1</constspeed>` in the propeller XML enables this. Below the governor range, the prop reverts to fixed pitch at `<minpitch>`. Beta range (below flight idle) and reverse (negative pitch) are supported on turboprops.

41.4 Helicopter rotor

A rotor is a propeller that also generates lift. The thrust at hover (momentum theory):

$$T = 2\,\rho\,A\,v_i^2, \quad v_i = \sqrt{(T/(2\rho A))}$$

Translational lift: in forward flight the inflow becomes asymmetric; advancing blade sees $V + \omega \cdot r$, retreating sees $\omega \cdot r - V$. The retreating blade approaches stall at high forward speed — the helicopter's V_{NE} limit.

Cyclic pitch: blade pitch varies once per revolution to tilt the rotor disc, generating forces in any direction. **Collective pitch**: all blades change pitch together, controlling thrust magnitude. JSBSim's `FGRotor` implements blade-element + momentum theory with flapping dynamics; the X-15 and Pterosaur examples are good starting points.

Chapter 42: Signal Processing for FCS

42.1 Continuous-time filters

$$\text{Lag:}\quad H(s) = c_1/(s + c_1)$$

$$\text{Lead-lag:}\quad H(s) = (c_1 s + c_2)/(c_3 s + c_4)$$

$$\text{Washout:}\quad H(s) = s/(s + c_1)$$

$$\text{Second-order:}\quad H(s) = (c_1 s^2 + c_2 s + c_3)/(c_4 s^2 + c_5 s + c_6)$$

All four are the JSBSim filter components from Chapter 12. Coefficients $c_{1..6}$ are taken directly from the design in the continuous-time s-plane.

42.2 Discretisation: bilinear (Tustin) transform

$$s \leftarrow (2/T) \cdot (z - 1)/(z + 1)$$

Tustin maps the entire left half-plane (stable continuous) into the unit disc (stable discrete) so stability is preserved. Frequencies are warped: $\omega_d = (2/T)\tan(\omega_a T/2)$. For critical pole/zero frequencies, "prewarping" (replacing 2/T with $\omega_0/\tan(\omega_0 T/2)$) preserves the location exactly.

42.3 PID discretisation

```
u(z) = K_p · e(z)
     + K_i · T/2 · (1 + z⁻¹)/(1 − z⁻¹) · e(z)   (trapezoidal)
     + K_d · (1 − z⁻¹)/T · e(z)                  (backward Euler)
```

JSBSim's <pid> offers four integration methods (rect = backward Euler, trap = trapezoidal, ab2/ab3 = Adams-Bashforth). The <trigger> element provides anti-windup: when non-zero the integral is frozen, when negative it is reset.

42.4 PID tuning rules

Method	Kp	Ki	Kd
Ziegler-Nichols (oscillation)	$0.6 \cdot K_u$	$1.2 \cdot K_u/T_u$	$0.075 \cdot K_u \cdot T_u$
Lambda tuning (FOPDT, λ = closed-loop τ)	$\tau/(K(\lambda+\theta))$	Kp/τ	0 (no D)
IMC PI	$(2\tau+\theta)/(2K(\lambda+\theta))$	$Kp/(\tau+\theta/2)$	—

Chapter 43: Linear Complementarity and Contact Friction

43.1 The LCP formulation

A Linear Complementarity Problem seeks **z** ∈ ■■ such that

$$w = Mz + q, \quad w \geq 0, \quad z \geq 0, \quad w^{T}z = 0$$

i.e. for each i, either $w_i = 0$ or $z_i = 0$. Contact-with-friction is naturally an LCP: z_i is the contact impulse at the i-th contact, w_i is the relative velocity normal to the surface; either there is a contact ($z > 0$, $w = 0$) or there is not ($z = 0$, $w > 0$).

43.2 JSBSim's gear LCP

Each landing-gear contact contributes:

- A non-penetration constraint normal to the runway.
- Coulomb friction constraints tangent to the runway (forward / side / rolling resistance).
- Brake torque from the FCS as an additional friction-cone limit.

JSBSim's `FGAccelerations::CalculateFrictionForces()` solves the resulting LCP with Projected Gauss-Seidel iteration (Catto 2005), up to 50 inner iterations per tick. This is the same algorithm used in modern game-physics engines (Bullet, ODE, Box2D).

Why an LCP and not just an explicit force calculation? Because the stiff spring-damper-friction system is unconditionally unstable under explicit integration if you allow interpenetration. The LCP formulation projects the contact onto the admissible constraint manifold each tick, which is implicitly stable regardless of stiffness.

Chapter 44: The Trim Algorithm Inside Out

`FGTrim` finds the aircraft state and control settings that satisfy a steady-flight condition. It is implemented as a composition of one-dimensional secant root-finders coordinated by an outer loop.

44.1 FGTrimAxis: the unit cell

Each `FGTrimAxis` pairs a state variable to zero out (e.g. ■) with a control variable to vary (e.g. α). Given two control guesses with their corresponding state values, the secant update is

$$x_{n+1} = x_n - f(x_n) \cdot (x_n - x_{n-1}) / (f(x_n) - f(x_{n-1}))$$

JSBSim applies a 0.9 relaxation to dampen oscillations:

$$x_{n+1} = x_n - 0.9\, f(x_n)\, (\Delta x / \Delta f)$$

If the secant step leaves the bracket, JSBSim falls back to bisection on that axis. Each axis converges to a tolerance: 1e-3 for translational ■, 1e-4 (10× tighter) for angular ω̇.

44.2 Trim modes

Mode (TrimMode)	States zeroed	Controls varied
tLongitudinal	*u*■=0, ■=0, q̇=0	throttle, α, elevator
tFull	+ v̇=0, ■=0, ■=0, ψ-track	+ ϕ, aileron, rudder, β
tFullWingsLevel	tFull, but ϕ=0; β solves v̇=0	throttle, α, elevator, aileron, rudder, β
tGround	■=0, q̇=0, ■=0	altitude (AGL), θ, ϕ
tPullup	longitudinal, n_z = target	α, throttle, elevator
tTurn	coordinated turn at target bank	throttle, elevator, rudder
tTurnFull	coordinated turn + roll/yaw rates	full set
tCustom	user-built via AddState/RemoveState	user-chosen

44.3 Outer loop and convergence

Defaults: `SetMaxCycles(60)` passes through the axis list, `SetMaxCyclesPerAxis(100)` iterations per inner axis. Within each cycle, axes are trimmed in a fixed order. If all axes simultaneously meet their tolerances after a full cycle, the trim is declared converged and `simulation/trim-completed` is set to 1; otherwise the time history is restored and an error is raised.

Chapter 45: NASA NESC 2015 Verification Check Cases

JSBSim was the only open-source participant in the NASA Engineering and Safety Center's 6-DoF verification programme (NASA/TM-2015-218675). The other six tools were NASA in-house simulators (LaSRS++, SES, Marvin, POST-II, OSIRIS, MAVERIC). The NESC concluded that all seven simulators agreed to a publishable degree on the majority of cases, with the remaining differences explained and reducible.

45.1 Atmospheric flight cases

#	Case	Tests
1	Dropped sphere, dragless	Free fall in uniform gravity
2	Tumbling brick, dragless	Newton-Euler equations, inertia tensor
3	Tumbling brick + aero damping	Adds damping moments to rigid-body case 2
4	Dropped sphere, flat Earth	Tests gravity model selection
5	Dropped sphere, rotating spherical Earth	Adds Coriolis
6	Dropped sphere, rotating ellipsoidal Earth	Adds WGS-84 geodesy
7-8	Dropped sphere, steady / varying wind	Wind shear and gust profiles
9-10	Ballistic eastward / northward	Coriolis components
11	F-16 subsonic trim	Trim algorithm, short-period eigenstructure
12	F-16 supersonic trim	Transonic / supersonic aero, M>1 trim
13.1-13.4	F-16 disturbance manoeuvres	Altitude doublet, velocity step, heading step
15-16	F-16 global flights	Over the North Pole, around the equator
17	Two-stage rocket sea-level to orbit	Atmospheric → orbital transition

JSBSim's NASA test-case implementations live at `github.com/open-aerospace/jsbsim-nasa-test-cases`; running them is the recommended way to verify a JSBSim build against the published reference trajectories. The F-16 short-period and Dutch-roll eigenvalues match across simulators to the third significant digit; trim residuals are below the per-axis tolerances 1e-3 ft/s² and 1e-4 rad/s² that JSBSim defaults to.

Chapter 46: The Complete Property Tree

What follows is a near-complete enumeration of the JSBSim property namespace, distilled from the Doxygen API reference, the online manual, and the source-code `PropertyManager->Tie()` calls in each model.

46.1 simulation/

```
simulation/sim-time-sec        cumulative simulation time
simulation/dt                  integration step
simulation/frame               tick counter
simulation/do_simple_trim      0=long, 1=full, 2=ground, 3=pullup,
                                4=custom, 5=turn, 6=none
simulation/do_linearization    write nonzero to dump A,B,C,D
simulation/trim-completed      1 after successful trim
simulation/reset               reset to IC
simulation/pause               freeze model
simulation/terminate           stop script
simulation/integrator/rate/rotational
simulation/integrator/rate/translational
simulation/integrator/position/rotational
simulation/integrator/position/translational
simulation/gravity-model       0=spherical, 1=WGS-84
simulation/gravitational-torque  0/1 spacecraft tidal
simulation/randomseed          seeds random/urandom functions
simulation/disperse            Monte-Carlo enable
forces/hold-down               hold aircraft fixed at IC
```

46.2 ic/

```
ic/u-fps, ic/v-fps, ic/w-fps         body-axis velocity
ic/vn-fps, ic/ve-fps, ic/vd-fps      NED velocity
ic/vt-fps, ic/vt-kts                 true airspeed
ic/vc-kts                            calibrated airspeed
ic/ve-kts                            equivalent airspeed
ic/vg-kts                            ground speed
ic/mach
ic/phi-deg, ic/theta-deg, ic/psi-true-deg
ic/alpha-deg, ic/beta-deg, ic/gamma-deg
ic/lat-gc-deg, ic/lat-geod-deg
ic/long-gc-deg
ic/h-sl-ft, ic/h-agl-ft, ic/terrain-elevation-ft
ic/vw-dir-deg, ic/vw-mag-fps
ic/vw-north-fps, ic/vw-east-fps, ic/vw-down-fps
ic/p-rad_sec, ic/q-rad_sec, ic/r-rad_sec
ic/roc-fpm, ic/roc-fps
ic/targetNlf                          load factor target
```

46.3 position/, attitude/

```
position/h-sl-ft, h-sl-meters
position/h-agl-ft
position/lat-gc-deg, lat-gc-rad, lat-geod-deg
position/long-gc-deg, long-gc-rad
position/radius-to-vehicle-ft
position/distance-from-start-lat-mt
position/distance-from-start-lon-mt
position/distance-from-start-mag-mt
position/terrain-elevation-asl-ft
position/epa-rad                       Earth position angle
position/eci-x-ft, eci-y-ft, eci-z-ft
position/ecef-x-ft, ecef-y-ft, ecef-z-ft
attitude/phi-rad, theta-rad, psi-rad
attitude/phi-deg, theta-deg, psi-deg
attitude/heading-true-rad
attitude/roll-rad, pitch-rad
```

46.4 velocities/

```
velocities/u-fps, v-fps, w-fps              body
velocities/u-aero-fps, v-aero-fps, w-aero-fps    body, airmass-rel
velocities/p-rad_sec, q-rad_sec, r-rad_sec       body rates
velocities/p-aero-rad_sec, q-aero-rad_sec, r-aero-rad_sec
velocities/pi-rad_sec, qi-rad_sec, ri-rad_sec    inertial rates
velocities/vt-fps, vt-kts
velocities/vc-fps, vc-kts                  calibrated
velocities/ve-fps, ve-kts                  equivalent
velocities/vg-fps                          ground
velocities/mach, machU
velocities/h-dot-fps                       climb rate
velocities/v-north-fps, v-east-fps, v-down-fps
velocities/eci-velocity-mag-fps
```

46.5 aero/, atmosphere/

```
aero/qbar-psf, qbarUW-psf, qbarUV-psf
aero/alpha-rad, alpha-deg, beta-rad, beta-deg
aero/alphadot-rad_sec, betadot-rad_sec
aero/mag-beta-rad, mag-alpha-rad
aero/Re                                    Reynolds number
aero/bi2vel, ci2vel                        b/(2V), cbar/(2V)
aero/alpha-wing-rad                        incidence adjusted
aero/h_b-cg-ft, h_b-mac-ft                 ground-effect ratios
aero/stall-hyst-norm                       stall hysteresis
aero/cl-squared
aero/coefficient/<axis>/<name>             every defined coefficient

atmosphere/T-R, T-sl-R, delta-T
atmosphere/rho-slugs_ft3, rho-sl-slugs_ft3
atmosphere/P-psf, P-sl-psf
atmosphere/a-fps, a-sl-fps
atmosphere/delta, theta, sigma
atmosphere/density-altitude, pressure-altitude
atmosphere/wind-north-fps, wind-east-fps, wind-down-fps
atmosphere/total-wind-fps, psiw-rad
atmosphere/turbulence/milspec/severity   0..7
```

46.6 forces/, moments/, accelerations/

```
forces/fbx-aero-lbs, fby-aero-lbs, fbz-aero-lbs
forces/fwx-aero-lbs, fwy-aero-lbs, fwz-aero-lbs   wind-axis
forces/fbx-prop-lbs, fby-prop-lbs, fbz-prop-lbs
forces/fbx-gear-lbs, fby-gear-lbs, fbz-gear-lbs
forces/fbx-total-lbs, fby-total-lbs, fbz-total-lbs
forces/hold-down
moments/l-aero-lbsft, m-aero-lbsft, n-aero-lbsft
moments/l-prop-lbsft, m-prop-lbsft, n-prop-lbsft
moments/l-gear-lbsft, m-gear-lbsft, n-gear-lbsft
moments/l-total-lbsft, m-total-lbsft, n-total-lbsft
accelerations/pdot-rad_sec2, qdot-rad_sec2, rdot-rad_sec2
accelerations/udot-ft_sec2, vdot-ft_sec2, wdot-ft_sec2
accelerations/Nx, Ny, Nz                load factors (g)
accelerations/n-pilot-x-norm, n-pilot-y-norm, n-pilot-z-norm
accelerations/a-pilot-x-ft_sec2, a-pilot-y, a-pilot-z
accelerations/gravity-ft_sec2
```

46.7 fcs/, gear/

```
fcs/elevator-cmd-norm, aileron-cmd-norm, rudder-cmd-norm
fcs/flap-cmd-norm, speedbrake-cmd-norm, spoiler-cmd-norm
fcs/pitch-trim-cmd-norm, roll-trim-cmd-norm, yaw-trim-cmd-norm
fcs/steer-cmd-norm
fcs/throttle-cmd-norm[n], mixture-cmd-norm[n]
fcs/throttle-pos-norm[n], mixture-pos-norm[n]
fcs/advance-cmd-norm[n], feather-cmd-norm[n]
fcs/magneto-cmd[n], starter-cmd[n]
fcs/elevator-pos-rad, -deg, -norm
fcs/left-aileron-pos-..., right-aileron-pos-...
fcs/rudder-pos-...
fcs/flap-pos-deg, flap-pos-norm
fcs/speedbrake-pos-rad, spoiler-pos-rad
fcs/wing-fold-pos-norm
fcs/left-brake-cmd-norm, right-brake-cmd-norm, center-brake-cmd-norm
gear/gear-cmd-norm, gear-pos-norm, tailhook-pos-norm
gear/unit[i]/compression-ft, WOW, wheel-speed-fps,
             static-friction-coeff, pos-x, pos-y, pos-z
```

46.8 propulsion/, inertia/, metrics/

```
propulsion/engine[i]/thrust-lbs, power-hp, rpm
propulsion/engine[i]/fuel-flow-rate-pps, fuel-flow-rate-gph
propulsion/engine[i]/n1, n2
propulsion/engine[i]/egt-degF, oil-pressure-psi, oil-temperature-degF
propulsion/engine[i]/bsfc-lbs_hphr
propulsion/engine[i]/mp-osi, volumetric-efficiency
propulsion/engine[i]/set-running                  start flag
propulsion/tank[i]/contents-lbs, capacity-gal_us
propulsion/total-fuel-lbs, refuel
inertia/empty-weight-lbs, weight-lbs, mass-slugs
inertia/cg-x-in, cg-y-in, cg-z-in
inertia/ixx-slugs_ft2, iyy-, izz-, ixy-, ixz-, iyz-
metrics/Sw-sqft, bw-ft, cbarw-ft
metrics/Sh-sqft, lh-ft, Sv-sqft, lv-ft
metrics/aero-rp-x-in, aero-rp-y-in, aero-rp-z-in
```

Chapter 47: Function Language — Complete Operator Reference

Every operator usable inside an <aerodynamics>, <flight_control>, <system> or <external_reactions> <function> block, distilled from the Doxygen FGFunction reference. Shortcuts: <v> = <value>, <p> = <property>, <t> = <table>.

Operator	Description
sum	Adds all immediate children.
difference	First child minus sum of remaining children.
product	Multiplies all children.
quotient	First child divided by second.
pow	First child raised to second.
sqrt	Square root.
exp	e raised to child.
ln, log2, log10	Natural, base-2, base-10 logarithm.
abs, sign	Absolute value, sign.
sin, cos, tan	Trig (arg in radians).
asin, acos, atan	Inverse trig; result in radians.
atan2	atan2(Y, X); range –π..π.
toradians, todegrees	Angular unit conversion.
pi	Constant π. Use as .
lt, le, gt, ge, eq, nq	Returns 1 if relation holds else 0.
and, or, not	Boolean. and/or take n children.
ifthen	ifthen(cond, true, false). Default false = 0.
switch	switch(index, v0, v1, …); index zero-based.
min, max, avg	Aggregate over children.
floor, ceil, integer, fraction	Rounding operations.
mod, fmod, roundmultiple	Modulo, floating-point modulo, rounding to multiple.
random	Gaussian; attrs seed, mean, stddev.
urandom	Uniform; attrs seed, lower, upper.
value, v	Literal numeric.
property, p	Read property (string body).
table, t	1-D, 2-D or 3-D table.
interpolate1d	1-D inline interpolation.
rotation_alpha_local, rotation_beta_local, rotation_gamma_local	Specialty rotations (6 args).
rotation_bf_to_wf, rotation_wf_to_bf	Body↔wind rotations (7 args).

Chapter 48: Verification and Validation Procedures

48.1 V&V hierarchy

• **Verification**: are we solving the equations correctly? Test against analytical solutions, against published reference trajectories (NESC), against the simulator's own previous results after code change.

• **Validation**: are we solving the correct equations? Compare with flight test, wind tunnel, real aircraft handbook numbers. Validation is always against external truth.

• **Sensitivity analysis**: perturb each parameter (CG, I_{yy}, $C_{m\alpha}$, etc.) and quantify the response shift. Identifies the parameters most worth investing in.

48.2 Verification check list (per release)

```
1.  Build and run unit tests (CMake/CTest).
2.  Run NESC atmospheric cases 1-17. Compare against
    open-aerospace/jsbsim-nasa-test-cases reference CSVs.
    Acceptable error: <1e-4 in normalised state.
3.  Run all aircraft in aircraft/ through their reset00 IC and
    do_simple_trim=0. Every aircraft must trim within 60 cycles.
4.  For each trimmed aircraft, run a pitch doublet and check
    short-period damping ratio is in [0.2, 0.9]; phugoid in
    [0.005, 0.1].
5.  Run forced-oscillation about Y at +/-1 deg, k=0.05, on the
    F-16 cruise condition; extract C_mq + C_m_alphadot;
    compare to NASA-2015 reference -22.0 +/- 1.0 1/rad.
```

48.3 Validation against flight data

When flight-test data exists (Cooper-Harper-rated step responses, stick-fixed releases, doublets), the procedure is:

• Match the trim condition exactly: weight, CG, altitude, Mach, fuel state. Document the aircraft configuration (gear, flaps, stores).

• Drive the simulator with the recorded pilot inputs (digital stick/throttle traces).

• Plot p, q, r, α, β, n_z, altitude, IAS overlaid with flight-test signals. Compute RMS error per channel.

• Update aero coefficients to minimise error using least-squares system ID — but always against a *held-out* validation manoeuvre to avoid overfitting.

48.4 Handling qualities (MIL-F-8785C / MIL-HDBK-1797)

Modern HQ rating predicts Cooper-Harper pilot rating from open-loop and closed-loop parameters. Key boundaries:

• Short-period ω_n vs n/α (Category A/B/C, Level 1/2/3 boundaries).

• Phugoid damping ratio (Level 1 $\zeta > 0.04$).

• Roll mode time constant τ_{roll} (Level 1 < 1 s).

• Dutch roll $\zeta \cdot \omega_n > 0.15$ (Level 1).

• Bandwidth and dropback (modern criteria; supplements MIL-F-8785C).

Chapter 49: Further Reading and Authoritative Sources

The list below is the curated set of references that this manual and the JSBSim source code itself cite most often.

49.1 Aerodynamics & airfoil theory

- Anderson, J. D. Jr. *Fundamentals of Aerodynamics*, 6th ed., McGraw-Hill, 2017.
- Houghton, E. L., Carpenter, P. W. *Aerodynamics for Engineering Students*, 7th ed., Butterworth-Heinemann, 2017.
- Drela, M. *Flight Vehicle Aerodynamics*, MIT Press, 2014.
- McCormick, B. W. *Aerodynamics, Aeronautics and Flight Mechanics*, 2nd ed., Wiley, 1994.
- Schlichting, H. and Gersten, K. *Boundary-Layer Theory*, 9th ed., Springer, 2017.
- Abbott, I. H., von Doenhoff, A. E. *Theory of Wing Sections*, Dover, 1959. — the NACA-airfoil reference.

49.2 Flight dynamics & control

- Stevens, B. L., Lewis, F. L., Johnson, E. N. *Aircraft Control and Simulation*, 3rd ed., Wiley, 2015.
- Etkin, B., Reid, L. D. *Dynamics of Flight: Stability and Control*, 3rd ed., Wiley, 1996.
- Nelson, R. C. *Flight Stability and Automatic Control*, 2nd ed., McGraw-Hill, 1998.
- Cook, M. V. *Flight Dynamics Principles*, 3rd ed., Butterworth-Heinemann, 2012.
- Roskam, J. *Airplane Flight Dynamics and Automatic Flight Controls*, DARcorp, 2003. Roskam's eight-volume *Airplane Design* set is the engineering bible.
- USAF Stability and Control DATCOM, AFFDL-TR-79-3032 (1978). Semi-empirical estimation methods for every derivative.

49.3 CFD & turbulence modelling

- Wilcox, D. C. *Turbulence Modeling for CFD*, 3rd ed., DCW Industries, 2006.
- Pope, S. B. *Turbulent Flows*, Cambridge, 2000.
- Hirsch, C. *Numerical Computation of Internal and External Flows*, 2nd ed., Butterworth-Heinemann, 2007.
- Spalart, P. R., Allmaras, S. R. "A One-Equation Turbulence Model for Aerodynamic Flows." AIAA 92-0439.
- Menter, F. R. "Two-Equation Eddy-Viscosity Turbulence Models for Engineering Applications." AIAA Journal 32, 1994.
- NASA Turbulence Modeling Resource — turbmodels.larc.nasa.gov.

49.4 Atmosphere & geodesy

- NASA-TM-X-74335 / NOAA-S/T 76-1562. *U.S. Standard Atmosphere, 1976.*
- NIMA / NGA TR 8350.2. *Department of Defense World Geodetic System 1984.*
- Torge, W., Müller, J. *Geodesy*, 4th ed., de Gruyter, 2012.
- Hofmann-Wellenhof, B., Lichtenegger, H., Wasle, E. *GNSS — Global Navigation Satellite Systems*, Springer, 2008.
- Vallado, D. A. *Fundamentals of Astrodynamics and Applications*, 4th ed., Microcosm Press, 2013.
- NCEI, BGS, NGA. *World Magnetic Model 2025 Technical Report*, 2025.

49.5 Propulsion

- Mattingly, J. D. *Aircraft Engine Design*, 2nd ed., AIAA, 2002.
- Hill, P. G., Peterson, C. R. *Mechanics and Thermodynamics of Propulsion*, 2nd ed., Addison-Wesley, 1992.
- Sutton, G. P., Biblarz, O. *Rocket Propulsion Elements*, 9th ed., Wiley, 2017.
- Leishman, J. G. *Principles of Helicopter Aerodynamics*, 2nd ed., Cambridge, 2006.

49.6 Numerical methods

- Hairer, E., Nørsett, S. P., Wanner, G. *Solving Ordinary Differential Equations I* (non-stiff), 2nd ed., Springer, 1993.
- Diebel, J. "Representing Attitude." Stanford Univ. report, 2006.
- Shoemake, K. "Animating Rotation with Quaternion Curves." SIGGRAPH 1985.
- Buss, S. "Accurate and Efficient Simulation of Rigid Body Rotations." UCSD, 1999.
- Catto, E. "Iterative Dynamics with Temporal Coherence." Crystal Dynamics tech. report, 2005.

49.7 JSBSim-specific resources

- Berndt, J. S. "JSBSim: An Open Source Flight Dynamics Model in C++." AIAA-2004-4923. PDF at jsbsim.sourceforge.net.
- Online manual: jsbsim-team.github.io/jsbsim-reference-manual.
- Doxygen API: jsbsim-team.github.io/jsbsim.
- NASA NESC 6-DoF check-cases: nescacademy.nasa.gov/flightsim/2015.
- JSBSim NASA test-case implementations: github.com/open-aerospace/jsbsim-nasa-test-cases.
- DeepWiki summary: deepwiki.com/JSBSim-Team/jsbsim.
- FlightGear wiki (lots of practical JSBSim notes): wiki.flightgear.org/JSBSim.

Chapter 50: Glossary and Symbol Index

Symbol	Meaning
α (alpha)	Angle of attack — angle between the body X axis and the projection of relative wind onto the body X-Z plane.
β (beta)	Sideslip angle — angle between relative wind and the body X-Z plane.
$\dot{\alpha}$ (alphadot)	Time rate of change of α.
$\bar{q}$ (qbar)	Dynamic pressure, $\frac{1}{2}\rho V^2$.
ρ (rho)	Atmospheric density.
V_t	True airspeed (magnitude of velocity relative to the air mass).
V_c	Calibrated airspeed (compressibility-corrected indicated airspeed).
V_e	Equivalent airspeed (density-corrected indicated airspeed).
M	Mach number, V_t/a.
a	Speed of sound.
p, q, r	Body-frame roll, pitch, yaw angular rates.
u, v, w	Body-frame translational velocity components.
ϕ, θ, ψ	Bank, pitch, heading Euler angles (3-2-1 sequence).
γ (gamma)	Flight path angle (positive climb).
S	Wing reference area.
b	Wing span.
$\bar{c}$ (cbar)	Mean aerodynamic chord.
AR	Aspect ratio, b^2/S.
e	Oswald efficiency factor (0.7-0.95 typical).
AERORP	Aerodynamic reference point — origin for aerodynamic moments.
VRP	Visual reference point — origin used by external viewers.
EYEPOINT	Pilot eye position.
AGL	Above ground level.
MSL	Mean sea level.
WGS-84	World Geodetic System 1984 — Earth ellipsoid model used by GPS and JSBSim.
ECEF	Earth-centred, Earth-fixed (rotates with the planet).
ECI	Earth-centred inertial (non-rotating).
NED	North-East-Down local tangent plane.
LCP	Linear complementarity problem — formulation of contact friction.
FDM	Flight Dynamics Model.
FCS	Flight Control System.
IC	Initial conditions.
BSFC	Brake specific fuel consumption.
MAP	Manifold absolute pressure (piston engines).
N1, N2	Low-pressure and high-pressure spool speeds (turbine engines).
TSFC	Thrust specific fuel consumption (turbines).
CFD	Computational Fluid Dynamics.
RANS	Reynolds-Averaged Navier-Stokes.
URANS	Unsteady RANS.
LES	Large-eddy simulation.

AVL	Athena Vortex Lattice (Drela).
DATCOM	USAF Stability and Control DATCOM handbook.
POH	Pilot's Operating Handbook.
TAS	True airspeed (=V_t).
IAS	Indicated airspeed.
CAS	Calibrated airspeed (=V_c).
EAS	Equivalent airspeed (=V_e).
KCAS / KTAS / KIAS	Knots calibrated/true/indicated airspeed.

End of the Ultimate JSBSim Reference. Bug reports and improvements welcome at https://github.com/JSBSim-Team/jsbsim.